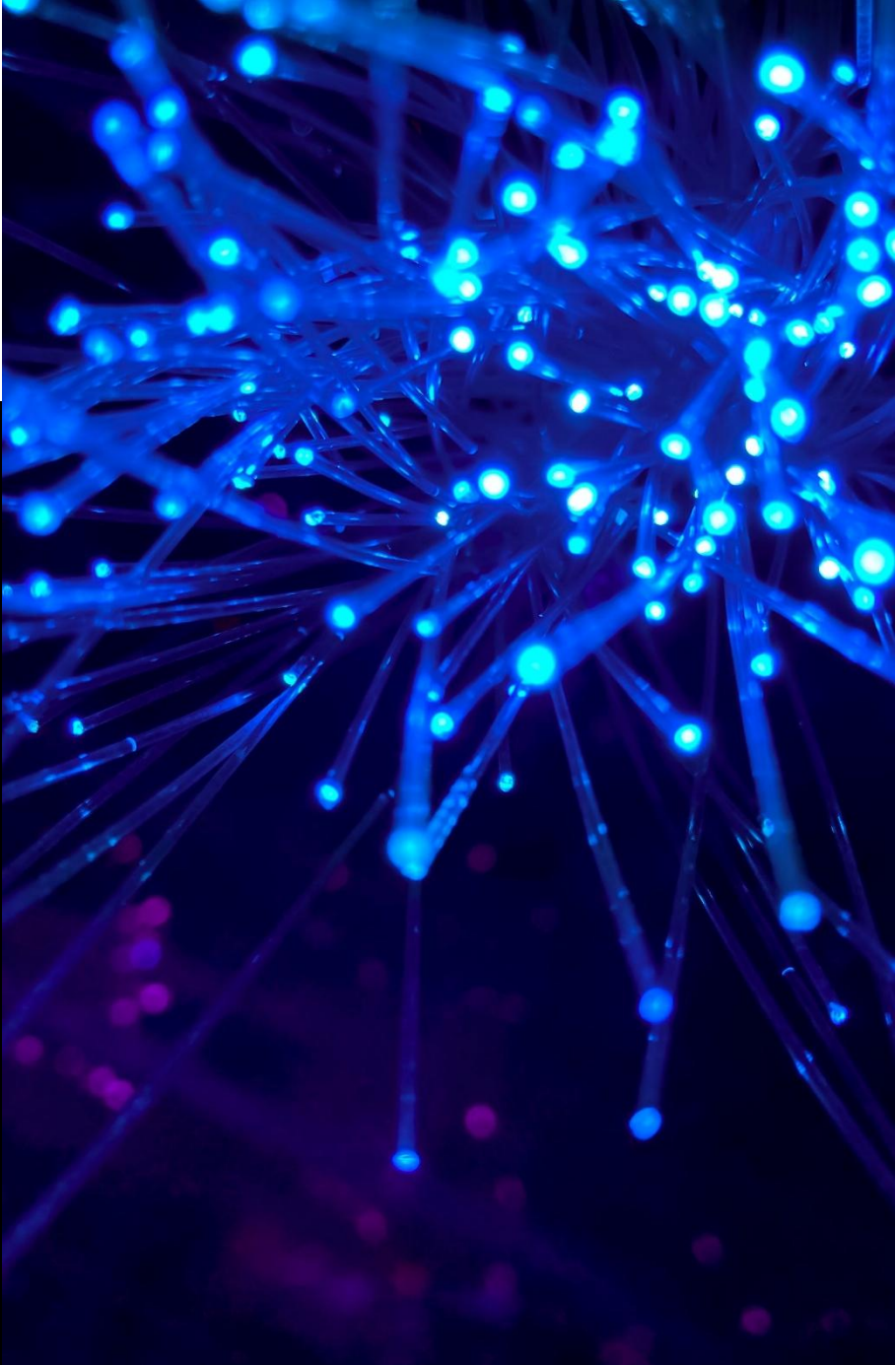




Gestión de Datos

Reducción de la Dimensionalidad



Reducción de la Dimensionalidad

- Análisis de Componentes Principales (PCA)
- Incrustación estocástica de vecinos distribuida en t (t-SNE)
- Análisis Discriminante Lineal (LDA)

- Análisis de Componentes Principales (PCA)
- Incrustación estocástica de vecinos distribuida en t (t-SNE)
- Análisis Discriminante Lineal (LDA)

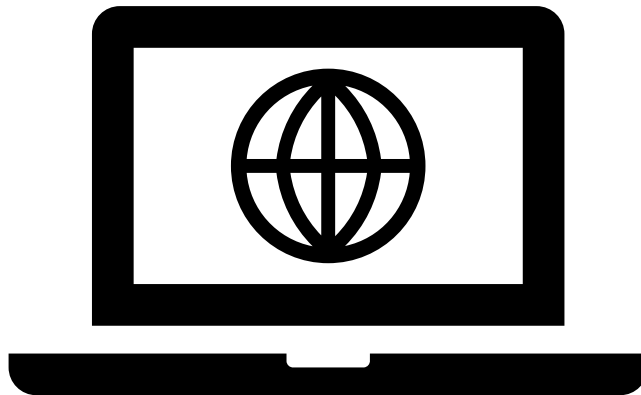
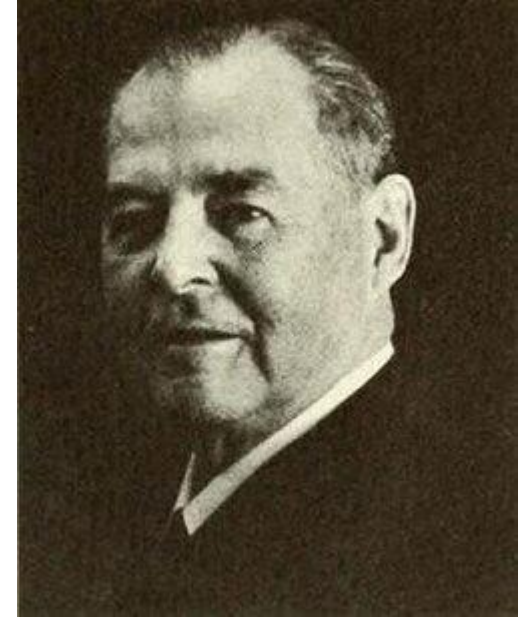
Análisis de Componentes Principales (PCA, Principal Component Analysis)

Es un método de reducción de dimensionalidad y aprendizaje automático **no** supervisado, que se utiliza para simplificar grandes conjuntos de datos en conjuntos más pequeños conservando al mismo tiempo la máxima varianza (información).



El PCA fue inventado en 1901
por Karl Pearson
Inglaterra

desarrollado y nombrado por
Harold Hotelling (1930)
EEUU

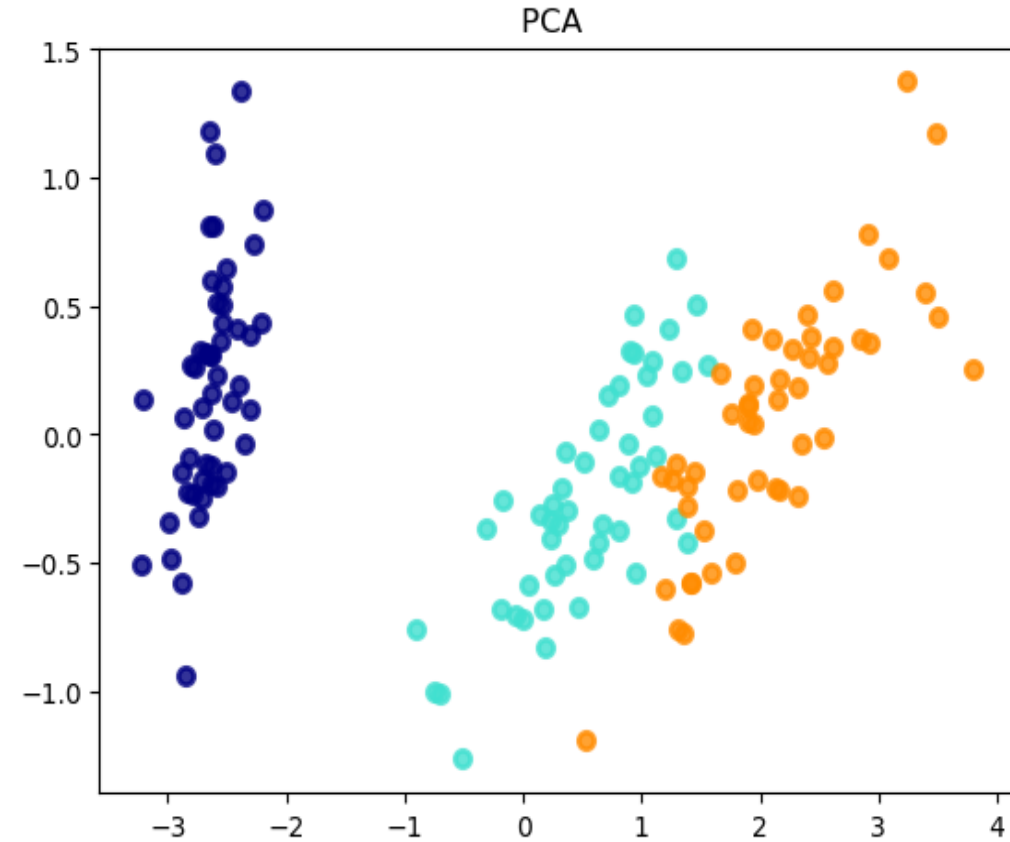
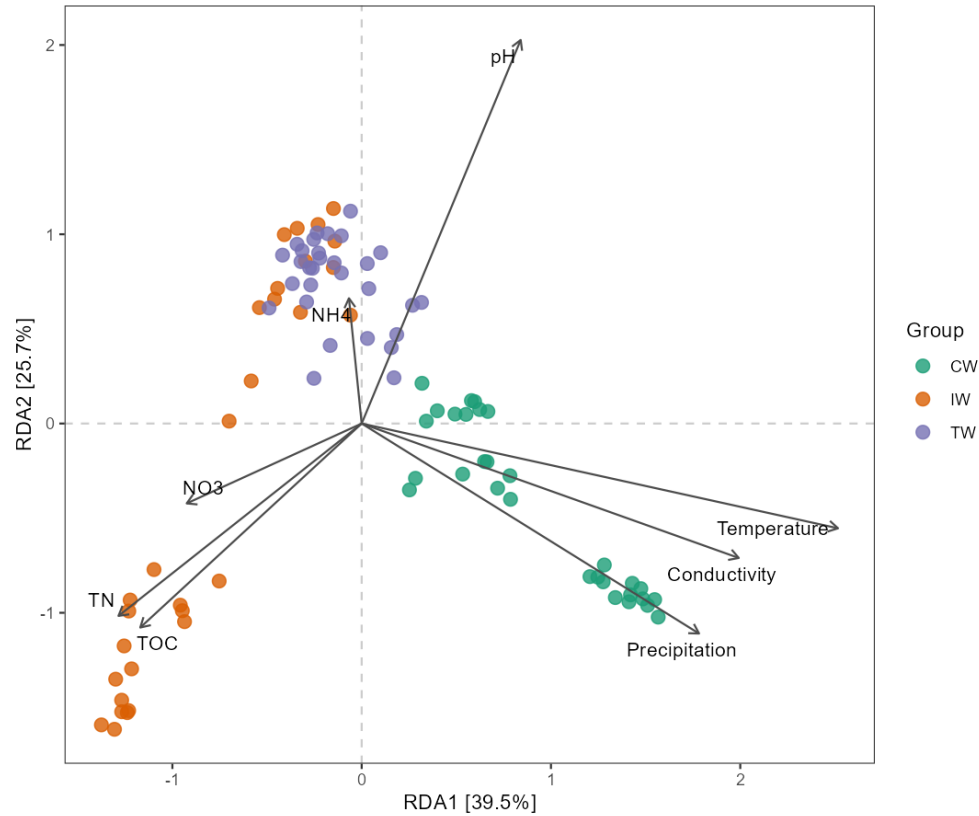


Reducir la dimensionalidad significa representar la información original utilizando un menor número de variables, conocidas como componentes principales.



Extraer información de un espacio de alta dimensión, proyectándola en un subespacio de dimensión inferior:

- Conservar partes esenciales: con alta variación.
- Eliminar partes no esenciales: con menor variación.



Facilita la visualización:
proyecta los datos en una dimensión inferior, permitiendo así
visualizarlos en un espacio 2D o 3D a simple vista.

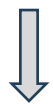
< ruido
> compresión

Reducir la dimensionalidad significa representar la información original utilizando un menor número de variables, conocidas como componentes principales.



Extraer información de un espacio de alta dimensión, proyectándola en un subespacio de dimensión inferior:

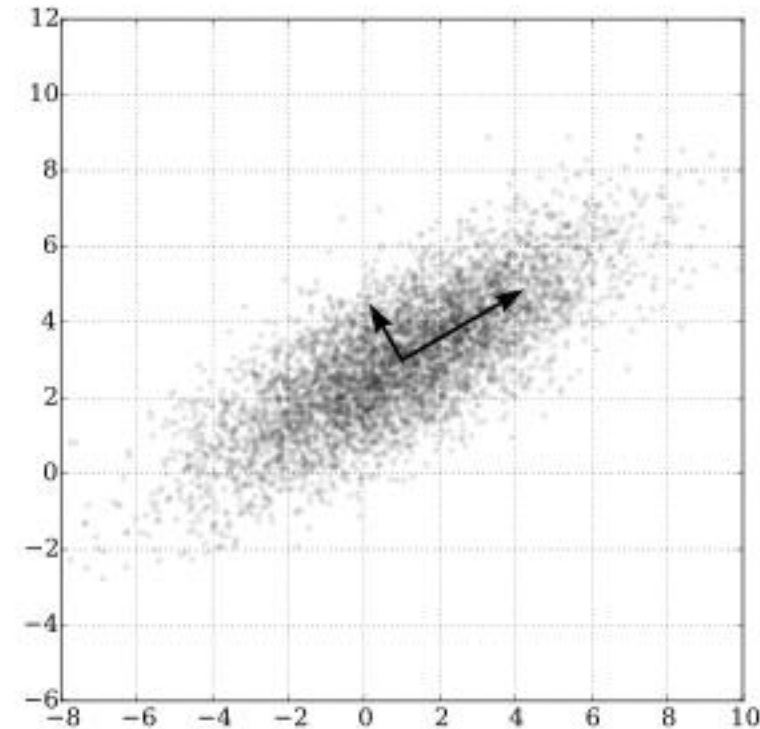
- Conservar partes esenciales: con alta variación.
- Eliminar partes no esenciales: con menor variación.



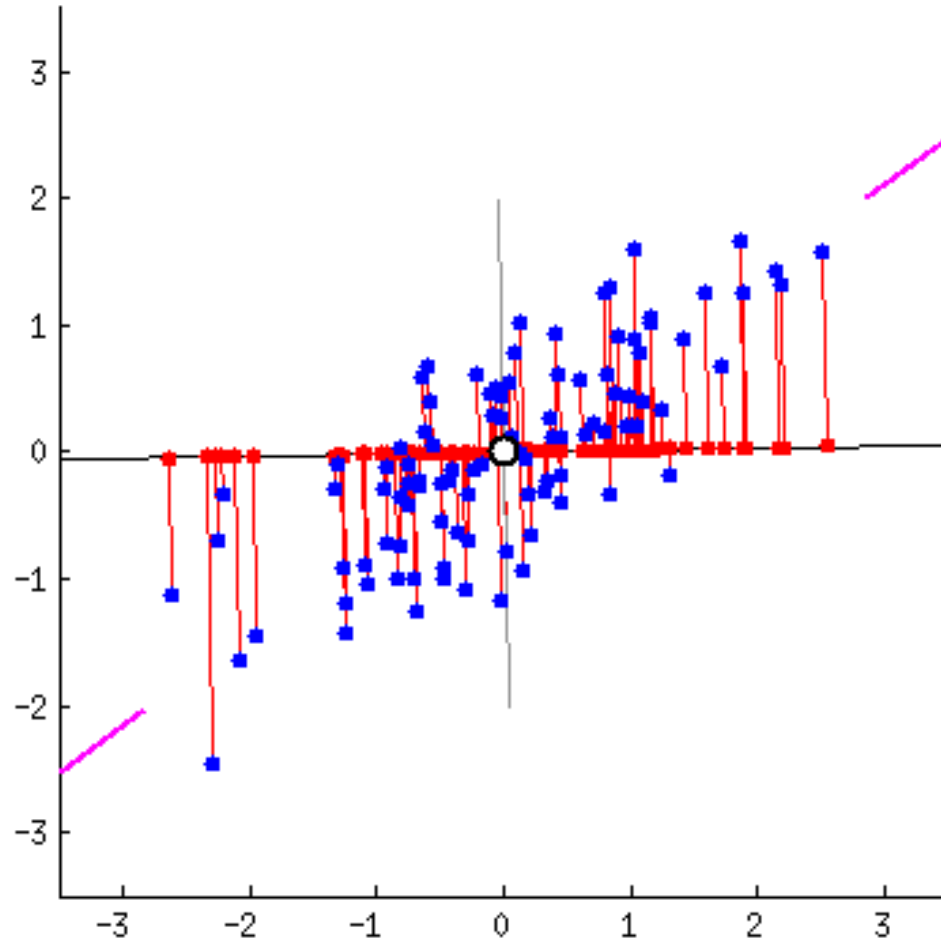
Problemas
comunes

- Multicolinealidad: cuando 2 o más variables independientes están altamente correlacionadas -> dificulta el modelo causal.
- Sobreajuste: los modelos sobreajustados se adaptan mal a los nuevos datos.

El PCA es un procedimiento estadístico:
utiliza una transformación ortogonal para convertir un conjunto de observaciones de variables posiblemente correlacionadas (cada una de las cuales adopta diversos valores numéricos) en un conjunto de valores de variables linealmente no correlacionadas denominadas componentes principales.



Vectores basados en los vectores propios de la matriz de covarianza para la distribución.

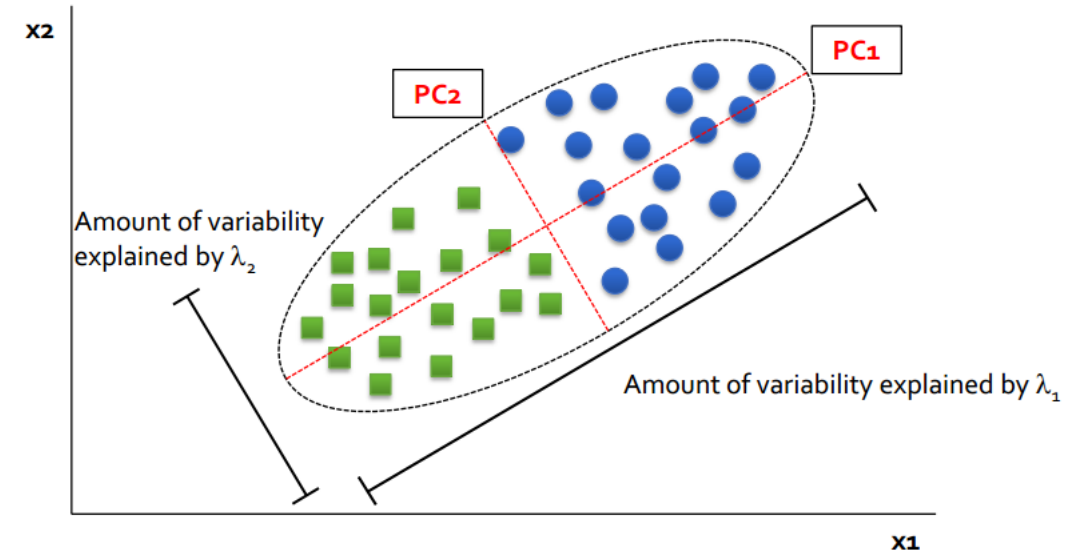


PCA como una rotación de ejes:
gira el sistema de coordenadas para
alinearlo con las direcciones de
máxima variación

¿Qué es una componente principal?

Son las dimensiones en que se proyectan los datos tras el PCA, y que contienen la mayor parte de la información (varianza) de los datos de partida.

- Las componentes principales son combinaciones lineales de las variables originales: representan un conjunto de características correlacionadas con distintas cantidades de variación.
- Cada componente principal es ortogonal a la anterior y tiene una varianza menor -> la primera componente principal es la que tiene mayor variabilidad.



- Tienen dirección y magnitud:
 - dirección: ejes principales a través de los que se extienden mayoritariamente los datos o estos tienen mayor varianza
 - magnitud: cantidad de varianza que capta de los datos el componente principal cuando se proyecta sobre ese eje.

Pasos:

0. Explorar los datos

1. Estandarizar los datos

2. Calcular la matriz de covarianza o matriz de correlación

3. Descomponer la matriz

4. Seleccionar las componentes principales

5. Transformar los datos en el nuevo sistema de coordenadas

1. Estandarizar los datos

$$x_{n,p}^N = \frac{x_{np} - \overline{x_{np}}}{\sigma}$$

Normalizar las variables para que tengan la misma escala.

Los datos deben reflejar una distribución normal.

Se calculan los valores medios de las variables y se restan del conjunto de datos original, para que todas contribuyan por igual al análisis.

Este valor se divide por la desviación típica de cada variable para que todas utilicen la misma escala.

2. Calcular la matriz de covarianza o matriz de correlación

$$S = \frac{1}{n} \cdot \sum_{i=1}^N (x_{np} - \bar{x}_p)^T \cdot (x_{np} - \bar{x}_p)$$

- S : La matriz de dispersión resultante (o matriz de covarianza si se ajusta a $n - 1$).
- n (o N): El número de muestras no prototípicas (tamaño de la muestra).
- $x_{np,i}$: El i -ésimo vector de muestra no prototípico ($i = 1, \dots, N$).
- $\bar{x}_p$: El vector medio de las muestras prototípicas (media del prototipo).
- $(x_{np,i} - \bar{x}_p)^T (x_{np,i} - \bar{x}_p)$: El producto externo del vector de diferencia, creando una matriz que representa la dispersión espacial de la i -ésima muestra del prototipo.

La matriz de covarianzas resume las covarianzas asociadas a todas las combinaciones de pares de las variables iniciales del conjunto de datos.

Calcular la matriz de covarianza ayuda a identificar las relaciones entre las variables, es decir, cómo las variables varían de la media entre sí.

La correlación puede ser positiva, negativa o cero.

La matriz de covarianza se usa cuando las variables tienen escalas similares, mientras que la de correlación es preferible si las variables tienen diferentes unidades o escalas.

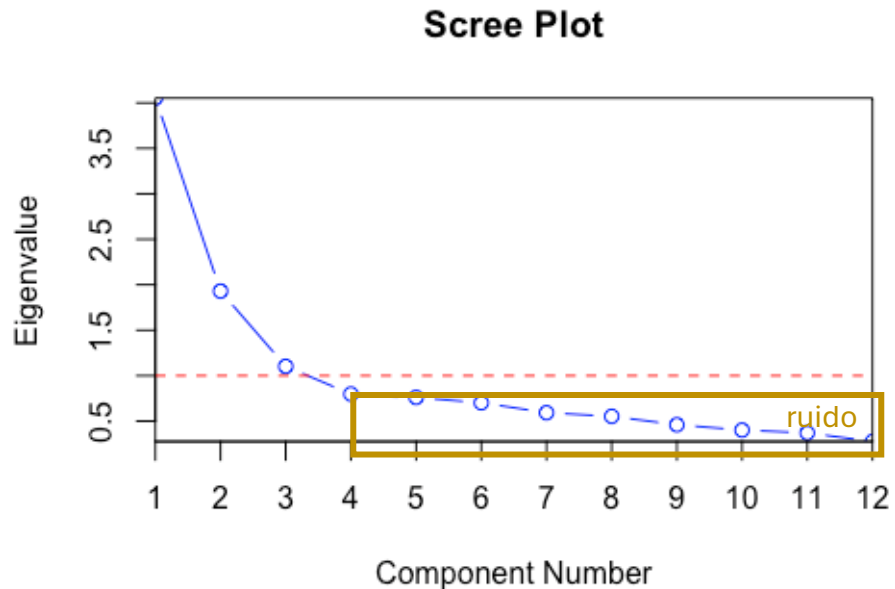
3. Descomponer la matriz

$$\begin{cases} |S - \lambda \cdot I| = 0 \\ |S - \lambda \cdot I| \cdot V = 0 \end{cases}$$

Calcular los vectores propios (componentes principales) y los valores propios de la matriz de covarianza.

- Como vectores propios, los componentes principales representan las direcciones de máxima varianza en los datos.
- Los valores propios representan la cantidad de varianza de cada componente. La clasificación de los vectores propios por valor propio identifica el orden de los componentes principales.

4. Seleccionar las componentes principales



By Staticshakedown - Own work, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=75715167>

Buscar el equilibrio entre la máxima información retenida y la menor cantidad de variables.

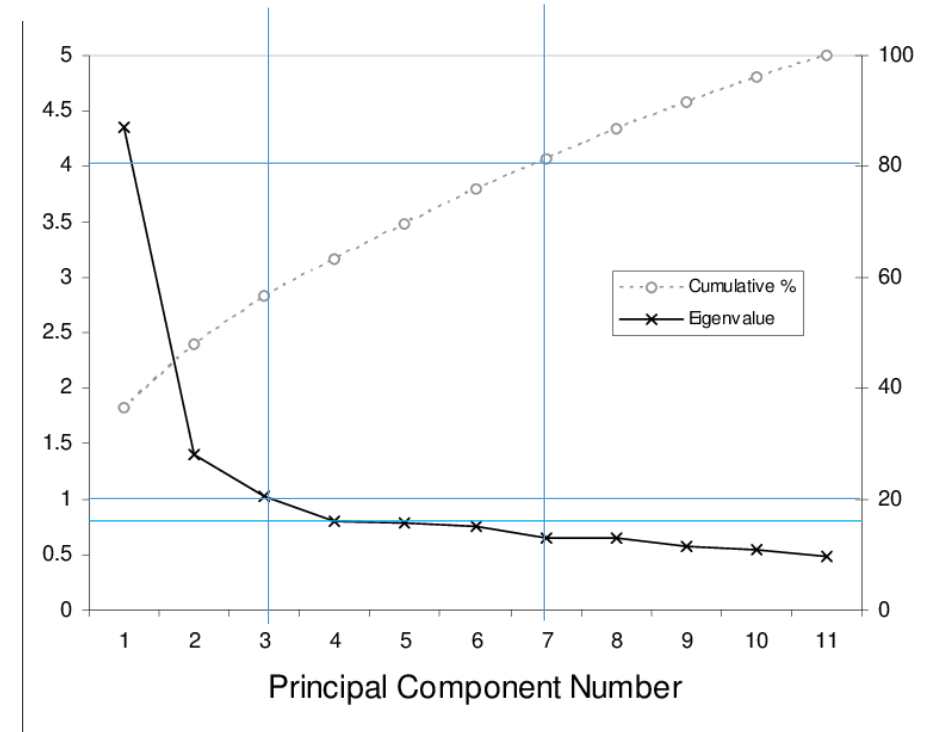
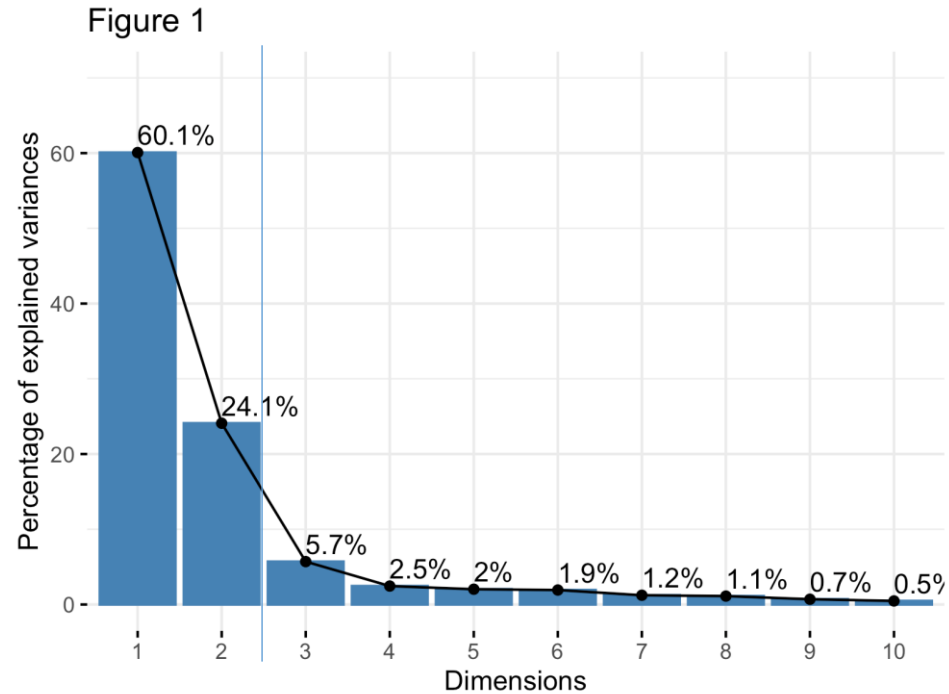
Se ordenan los autovalores de mayor a menor y se eligen los primeros k componentes principales que explican la mayor parte de la varianza en los datos.

Se decide qué valores conservar y cuáles descartar. El **gráfico de codo** o sedimentación (diagrama de Scree) muestra los componentes principales (x) frente a sus valores propios o % (y).

=> Regla del codo: seleccionar los componentes anteriores a una pendiente plana.

- Regla de kaiser: PCs con valores propios de al menos 1.
- Gráfico de proporción de varianza: los PCs seleccionados deben explicar al menos el 80% de la varianza.

4. Seleccionar las componentes principales



¿?

El **gráfico de varianza acumulada** ayuda a determinar el número de componentes. Representa gráficamente la suma acumulada de la varianza explicada frente al número de componentes principales.

4. Seleccionar las componentes principales

El análisis de un PCA se basa en la interpretación de la varianza explicada y las cargas factoriales.

- La varianza explicada por cada PC => los PCs seleccionados retener aprox. 80% varianza total.
- Las cargas factoriales muestran la influencia de cada variable en cada componente, donde valores absolutos altos indican una fuerte relación con esa dimensión.

La proporción de varianza explicada por cada PC se obtiene dividiendo cada autovalor por la suma total de los autovalores:

$$\text{Varianza Explicada} = \frac{\lambda_j}{\sum_{i=1}^p \lambda_i}$$

Las cargas de cada variable en cada componente principal se calculan como:

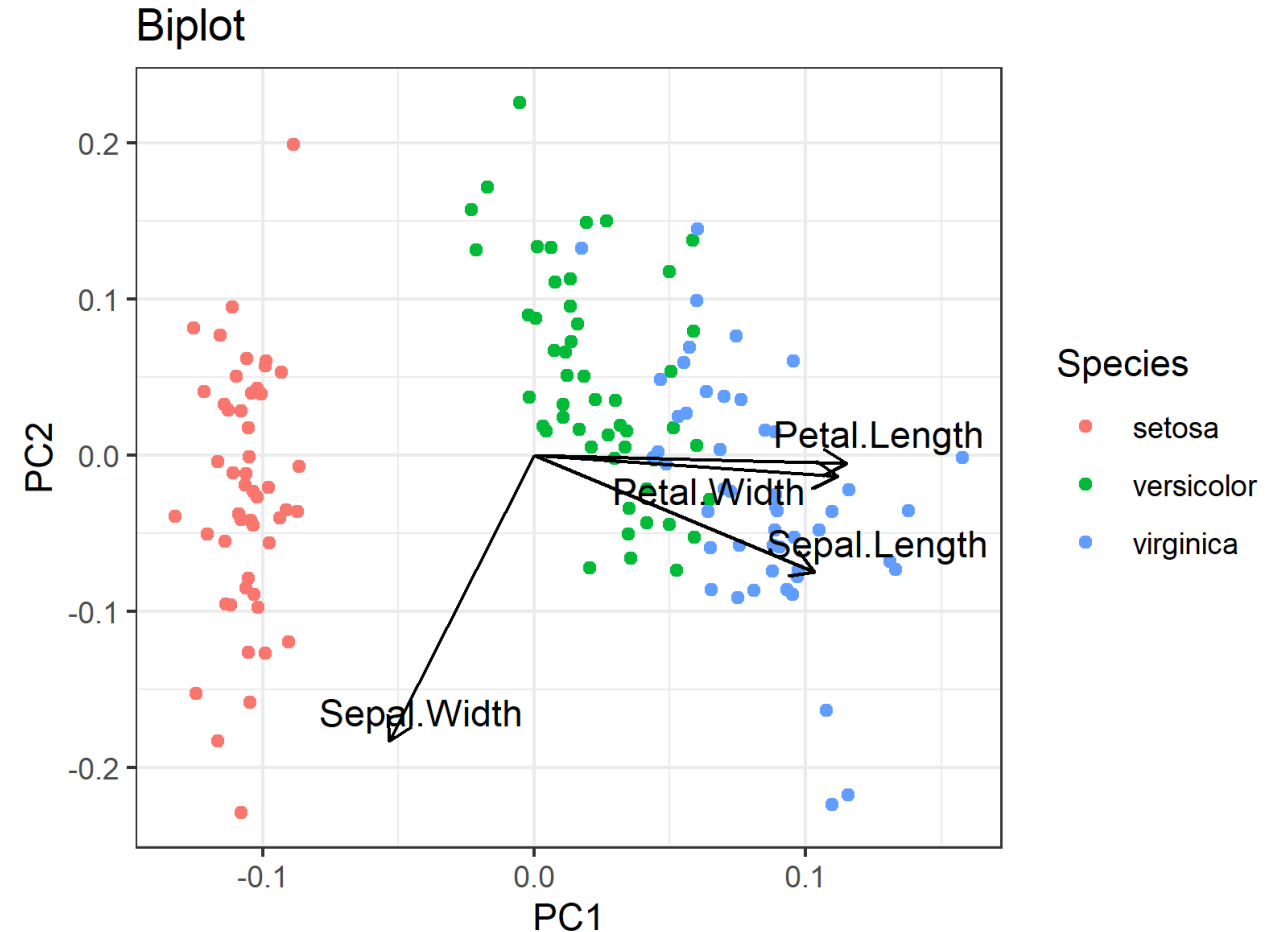
$$\text{Carga} = v_{ij} \cdot \sqrt{\lambda_j}$$

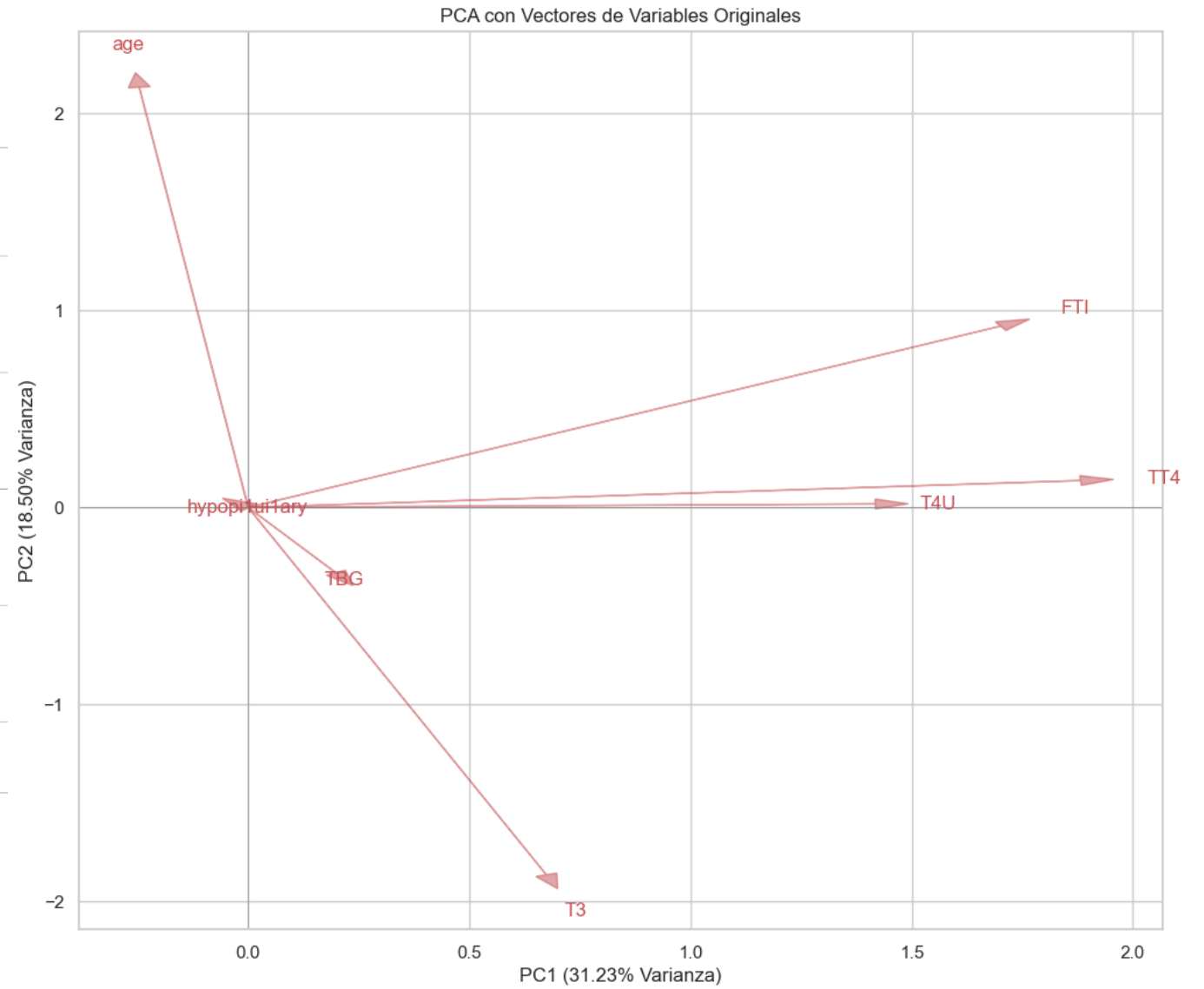
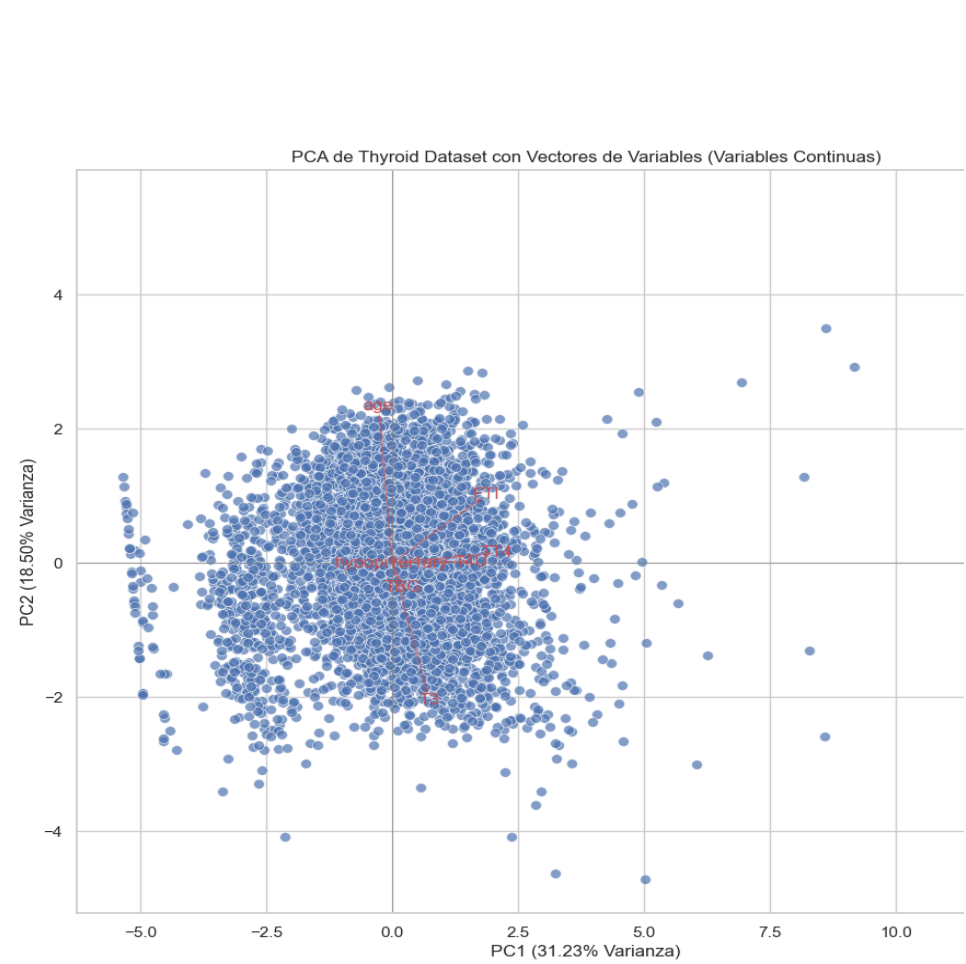
Las cargas factoriales indican cuanto contribuye cada variable a una PC. Cuanto mayor es el valor absoluto de una carga, mayor es la influencia de la variable en esa componente.

4. Seleccionar las componentes principales

Además de los gráficos de codo, los **biplots** ayudan a visualizar la distribución de los datos y las relaciones entre variables en el espacio transformado.

Si variables similares se agrupan en una misma dirección, significa que están correlacionadas en el nuevo espacio de representación.



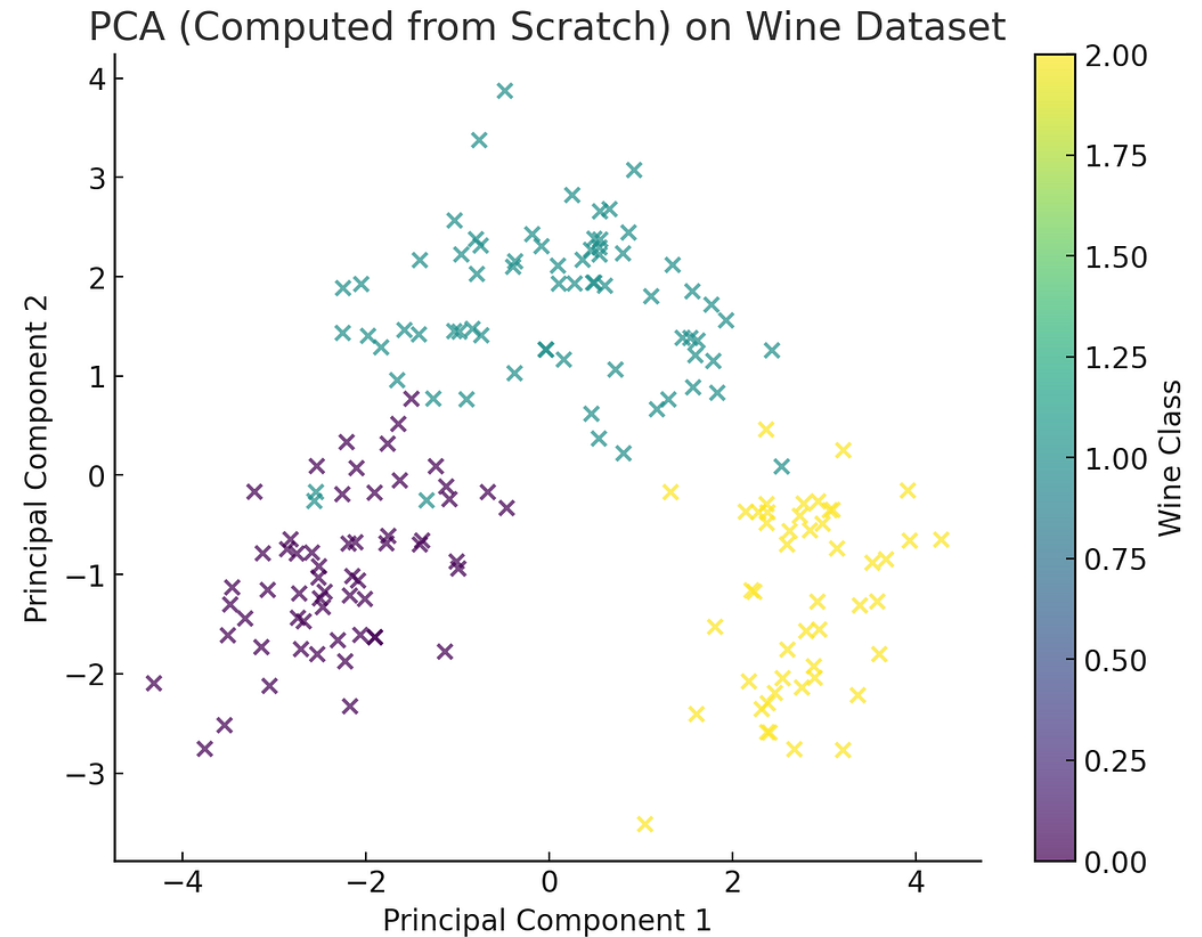


5. Transformar los datos en el nuevo sistema de coordenadas

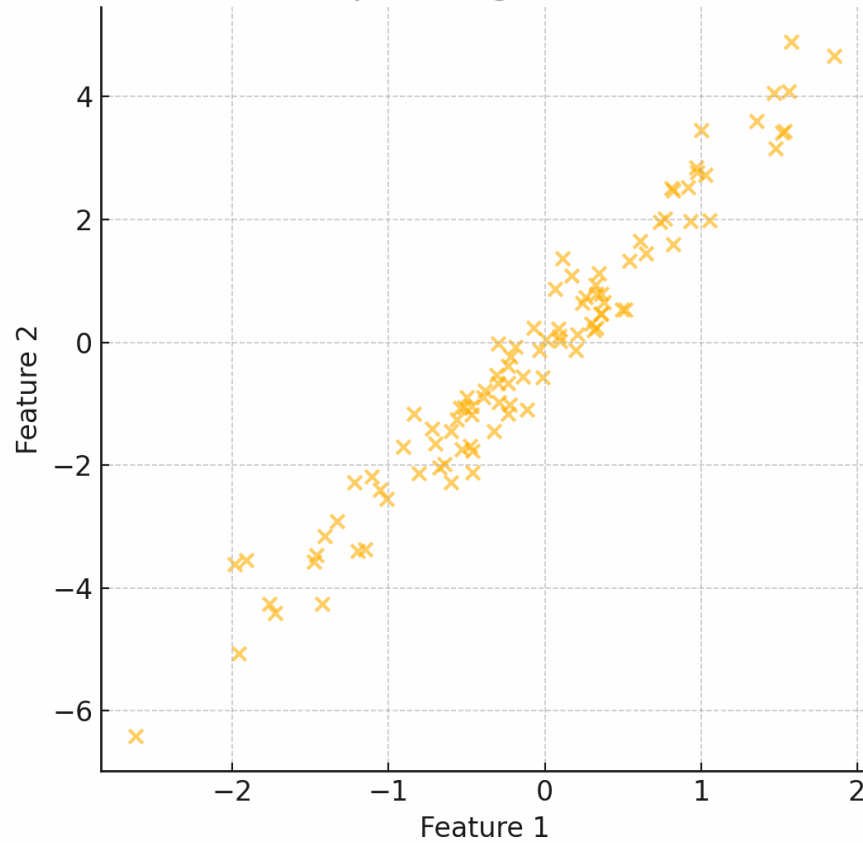
$$A_N \cdot V = PC$$

El vector de características creado a partir de los vectores propios de la matriz de covarianza proyecta los datos sobre los nuevos ejes definidos por los componentes principales.

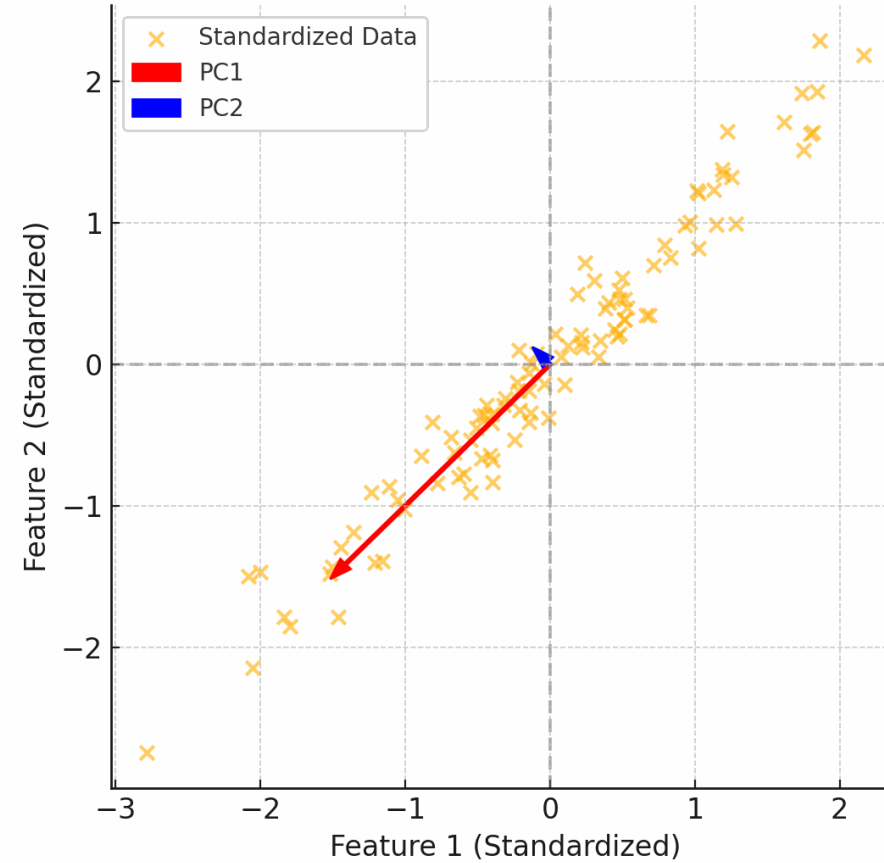
Esto crea nuevos datos, que capturan la mayor parte de la información, pero con menos dimensiones que el conjunto de datos original.

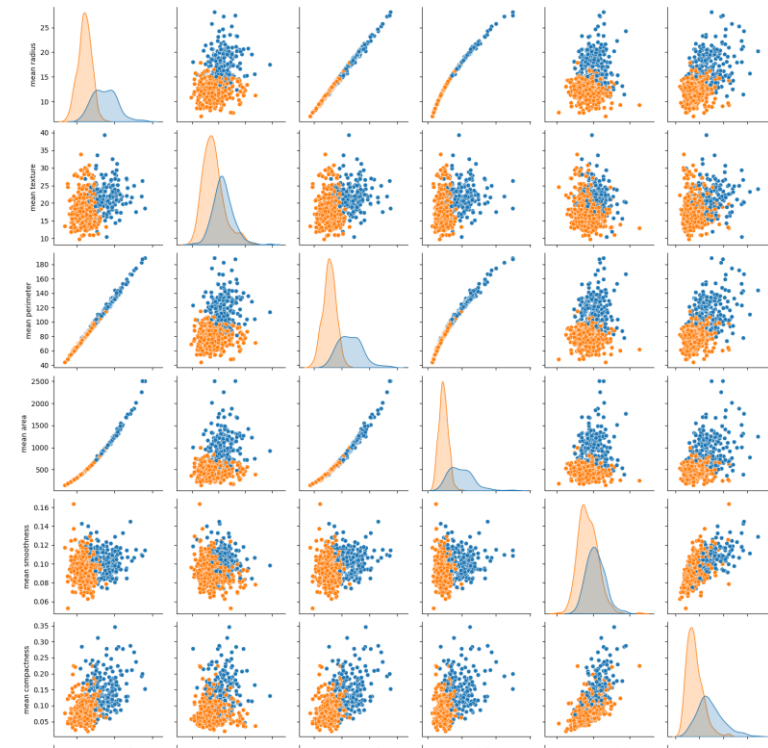


Step 1: Original Data



PCA: Data Projection onto Principal Components





Pairplot, by label (benign, malignant)

Ejemplo: breast-cancer dataset

Estandarizar los datos

```
scaler = StandardScaler()
df_for_scaler = breast_dataset.loc[:, features].values
x_scaled = scaler.fit_transform(df_for_scaler)

#Comprobar tamaño (569 filas, 30 características)
print(f"Forma datos: {x_scaled.shape}")
#Comprobar media (aprox. 0) y desvest (aprox. 1) de datos normalizados
print(f"Media datos estandarizados: {np.mean(x_scaled)}")
print(f"DE datos estandarizados: {np.std(x_scaled)}")

#convirtiendo a formato tabular
feat_cols = ['feature'+str(i) for i in range(x_scaled.shape[1])]
df_scaled = pd.DataFrame(x_scaled, columns=feat_cols)
```

El uso de StandardScaler equivale a usar la matriz de correlación para estandarizar.

Aplicar PCA

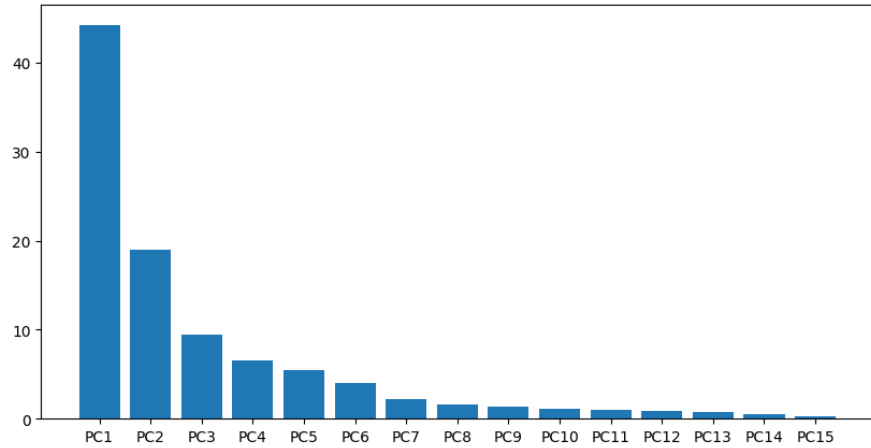
```
n_comp = 15 #se puede ajustar según se necesite
pca = PCA(n_components=n_comp)
principalComponents = pca.fit_transform(x_scaled)

#Generar un nuevo dataframe con los valores transformados de las 569 muestras
pca_df = pd.DataFrame(data=principalComponents, columns=[f'PC{i}' for i in range(1, n_comp+1)])

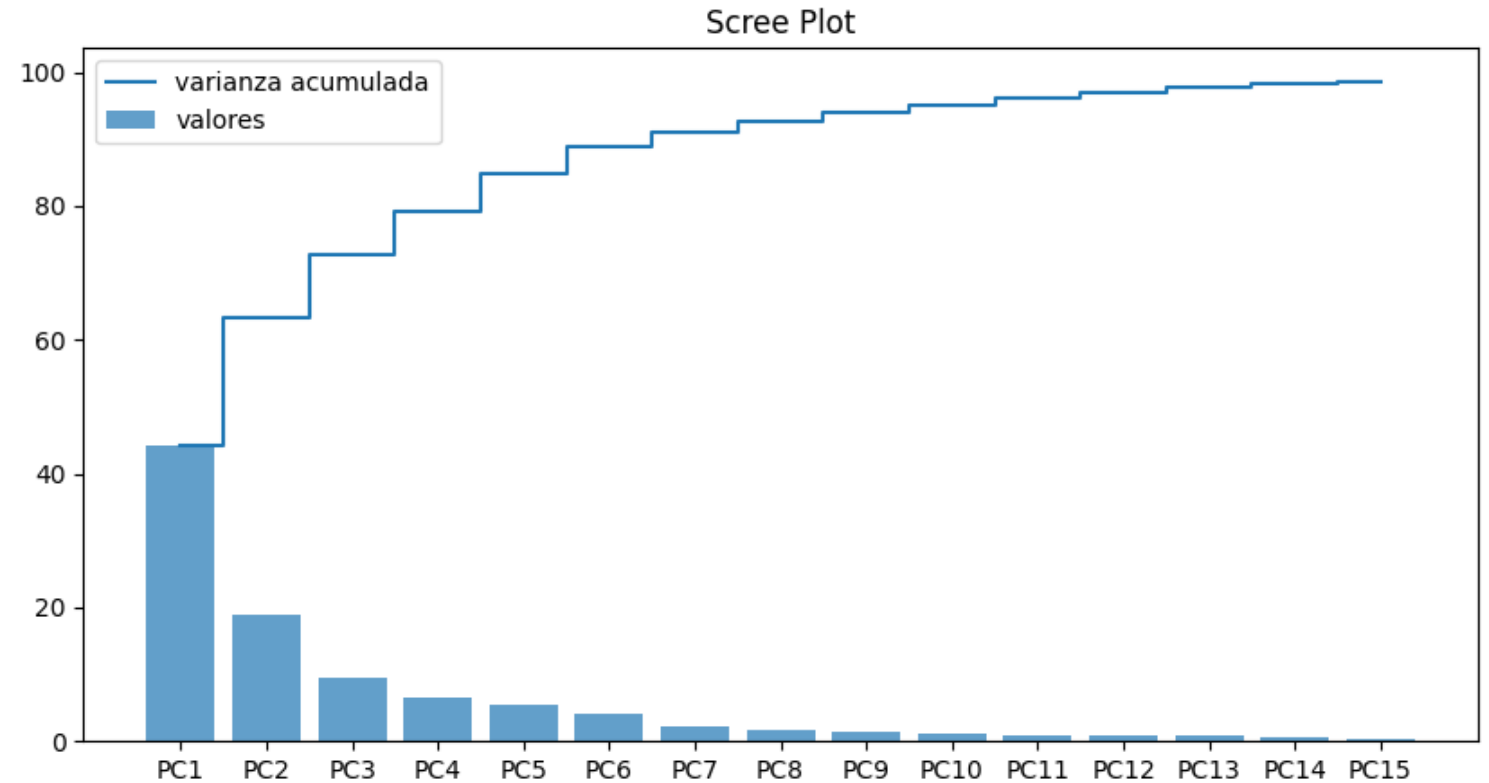
# Varianza explicada
explvar = pca.explained_variance_ratio_
data_expvar = {}
for i, var in enumerate(explvar, start=1):
    print(f"PC{i}: {var*100:.2f}%")
    data_expvar[f"PC{i}"] = var*100
```

pca.explained_variance_ratio_ devuelve un array con el porcentaje de varianza explicada por cada componente principal. Cada valor indica cuánta información del dataset original se conserva en esa componente.

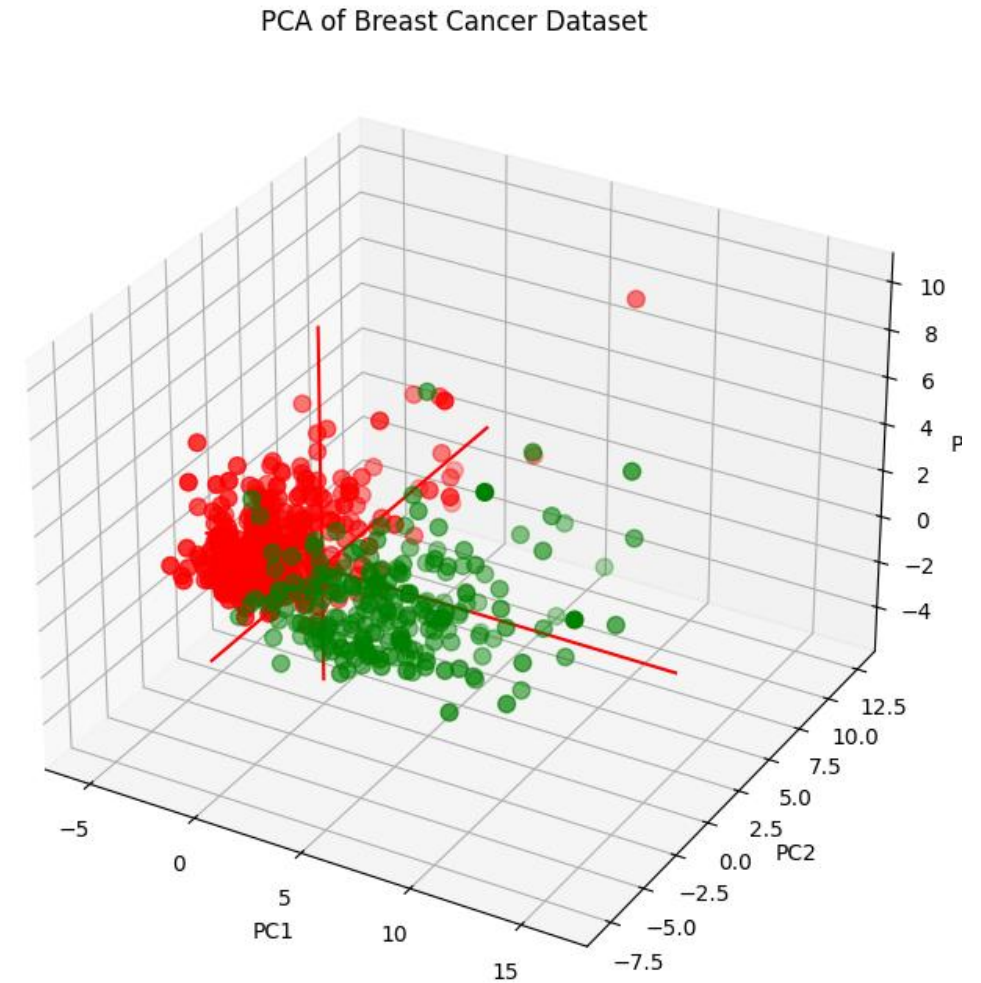
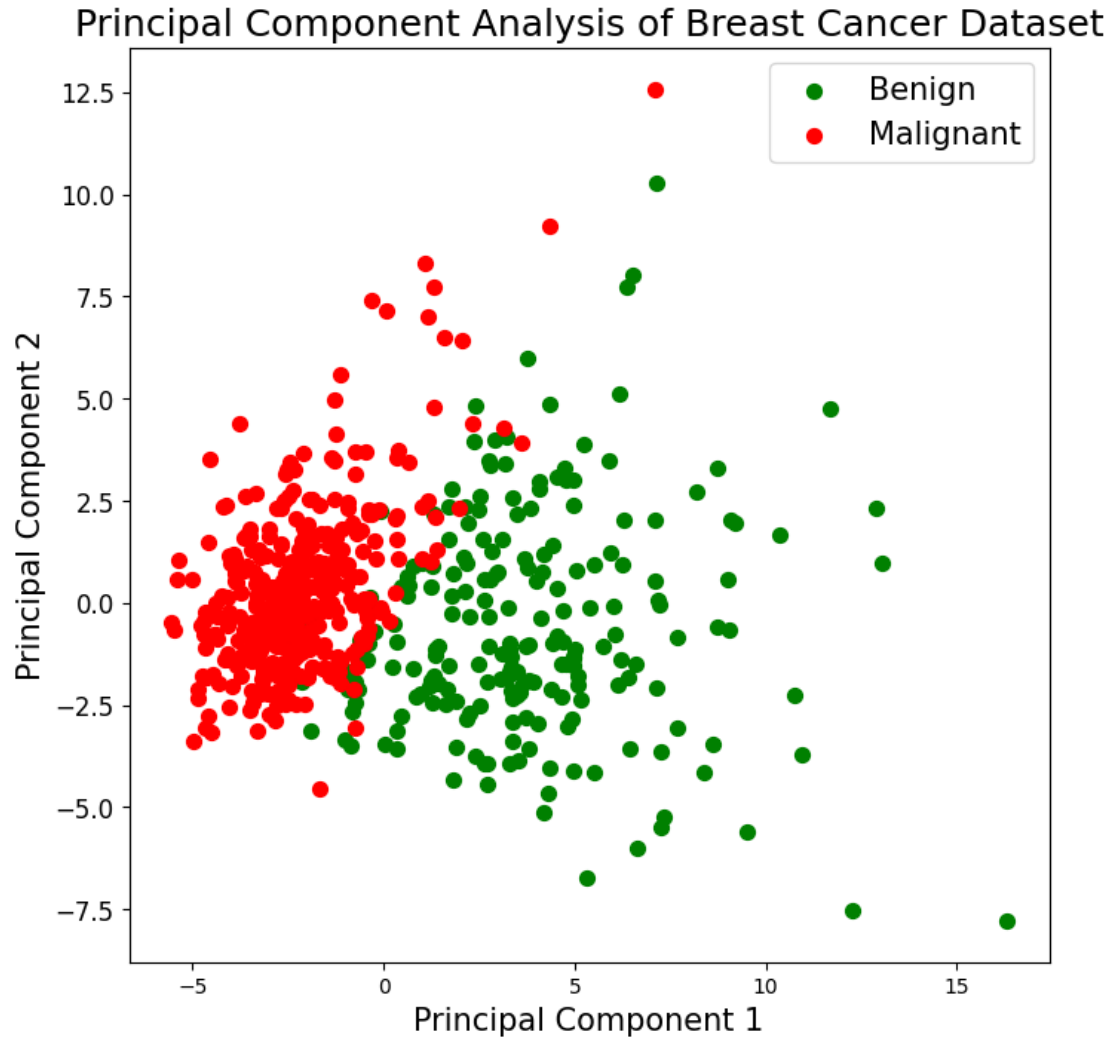
Ejemplo: breast-cancer dataset



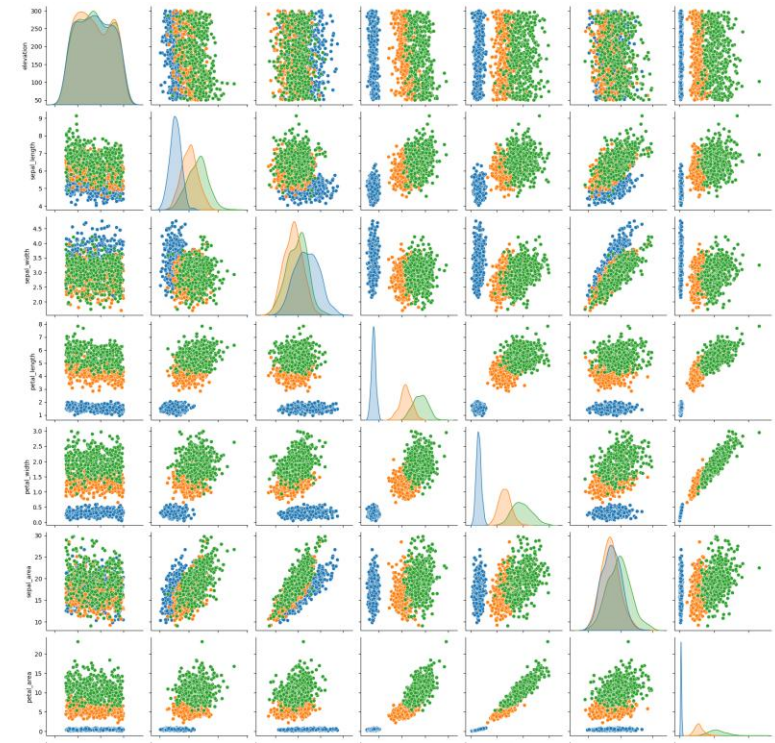
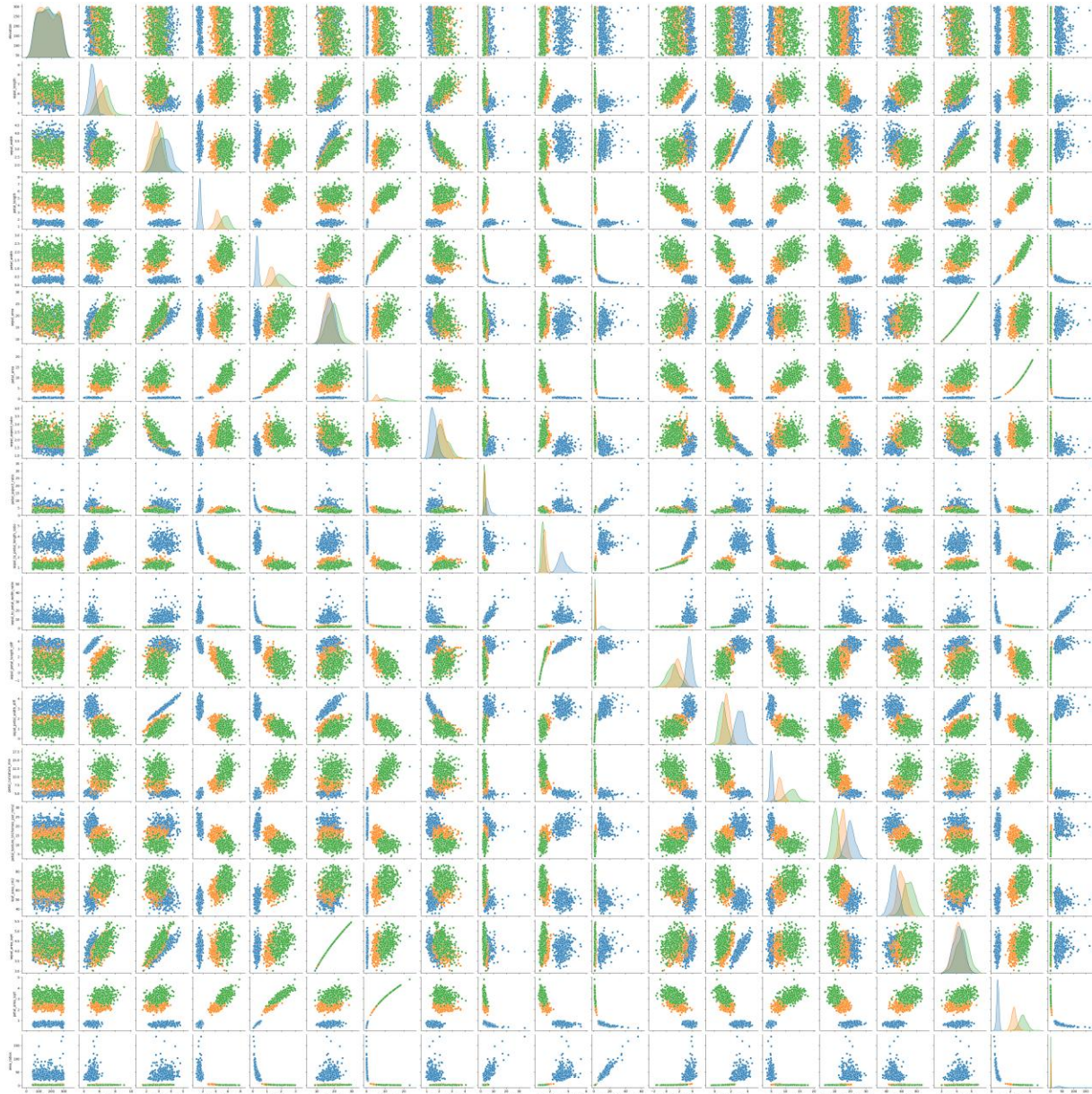
PC1: 44.27%	PC9: 1.39%
PC2: 18.97%	PC10: 1.17%
PC3: 9.39%	PC11: 0.98%
PC4: 6.60%	PC12: 0.87%
PC5: 5.50%	PC13: 0.80%
PC6: 4.02%	PC14: 0.52%
PC7: 2.25%	PC15: 0.31%
PC8: 1.59%	



Ejemplo: breast-cancer dataset



Ejemplo: iris-extended dataset



Pairplot, by species

Ejemplo: iris-extended dataset

Estandarizar los datos

```
scaler = StandardScaler()
df_for_scaler = iris_numdata.values
x_scaled = scaler.fit_transform(df_for_scaler)

#Comprobar tamaño (1200 filas, 19 características)
print(f"Forma datos: {x_scaled.shape}")
#Comprobar media (aprox. 0) y desvest (aprox. 1) de datos normalizados
print(f"Media datos estandarizados: {np.mean(x_scaled)}")
print(f"DE datos estandarizados: {np.std(x_scaled)}")

#convirtiendo a formato tabular
feat_cols = ['feature'+str(i) for i in range(x_scaled.shape[1])]
df_scaled = pd.DataFrame(x_scaled, columns=feat_cols)
```

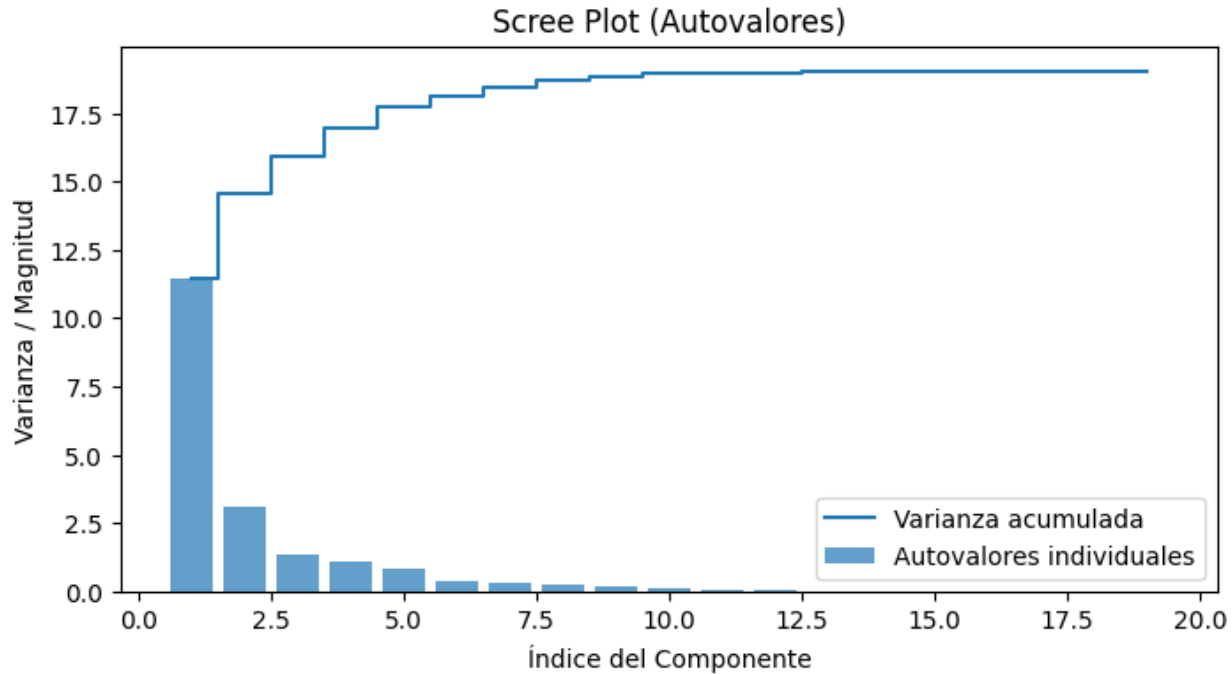
Aplicar PCA

```
n_comp = 10 #se puede ajustar según se necesite
pca = PCA(n_components=n_comp)
principalComponents = pca.fit_transform(x_scaled)

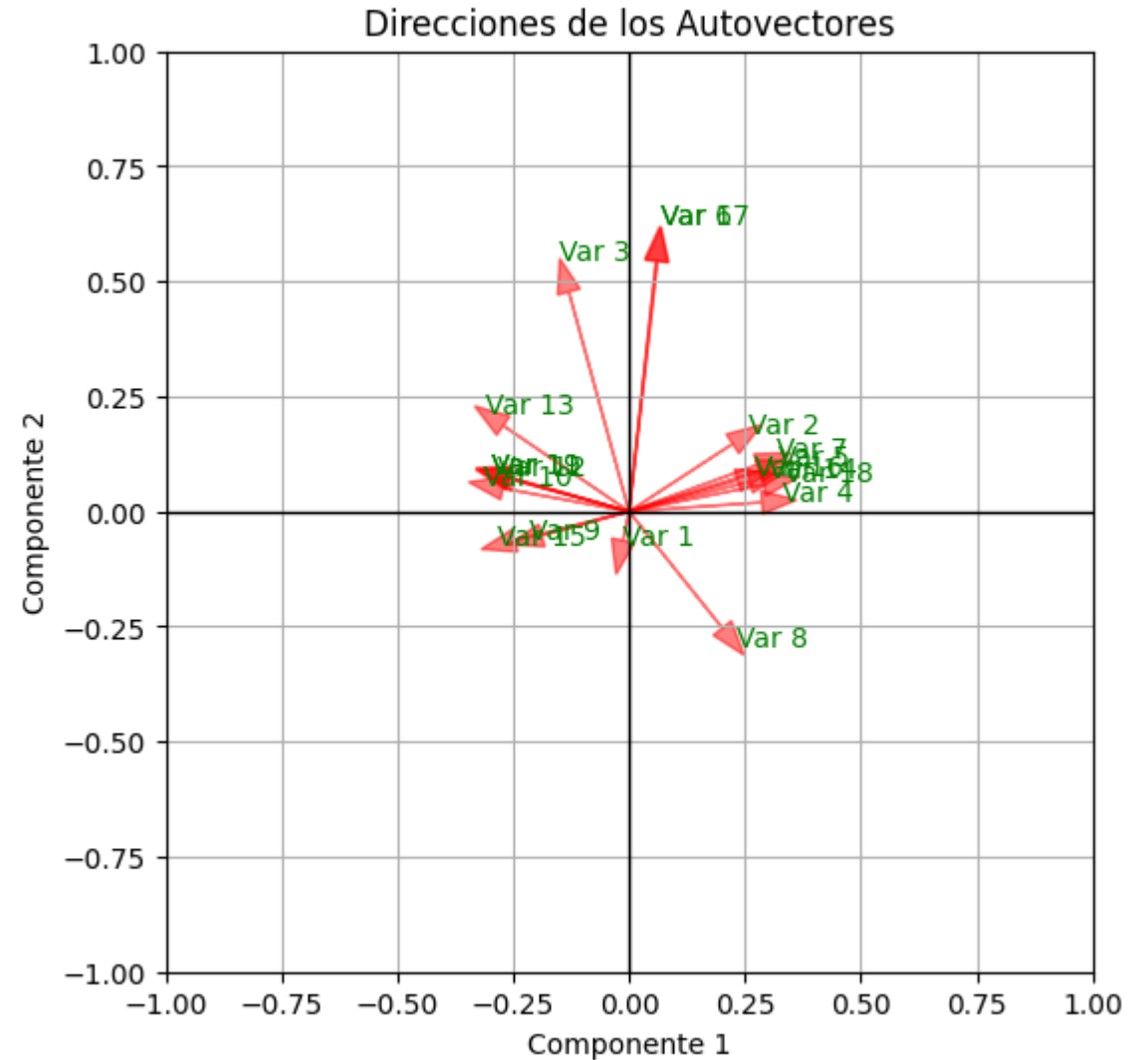
#Generar un nuevo dataframe con los valores transformados de las 1200 muestras
pca_df = pd.DataFrame(data=principalComponents, columns=[f'PC{i}' for i in range(1, n_comp+1)])

# Varianza explicada
explvar = pca.explained_variance_ratio_
data_expvar = {}
for i, var in enumerate(explvar, start=1):
    print(f"PC{i}: {var*100:.2f}%")
    data_expvar[f"PC{i}"] = var*100
```

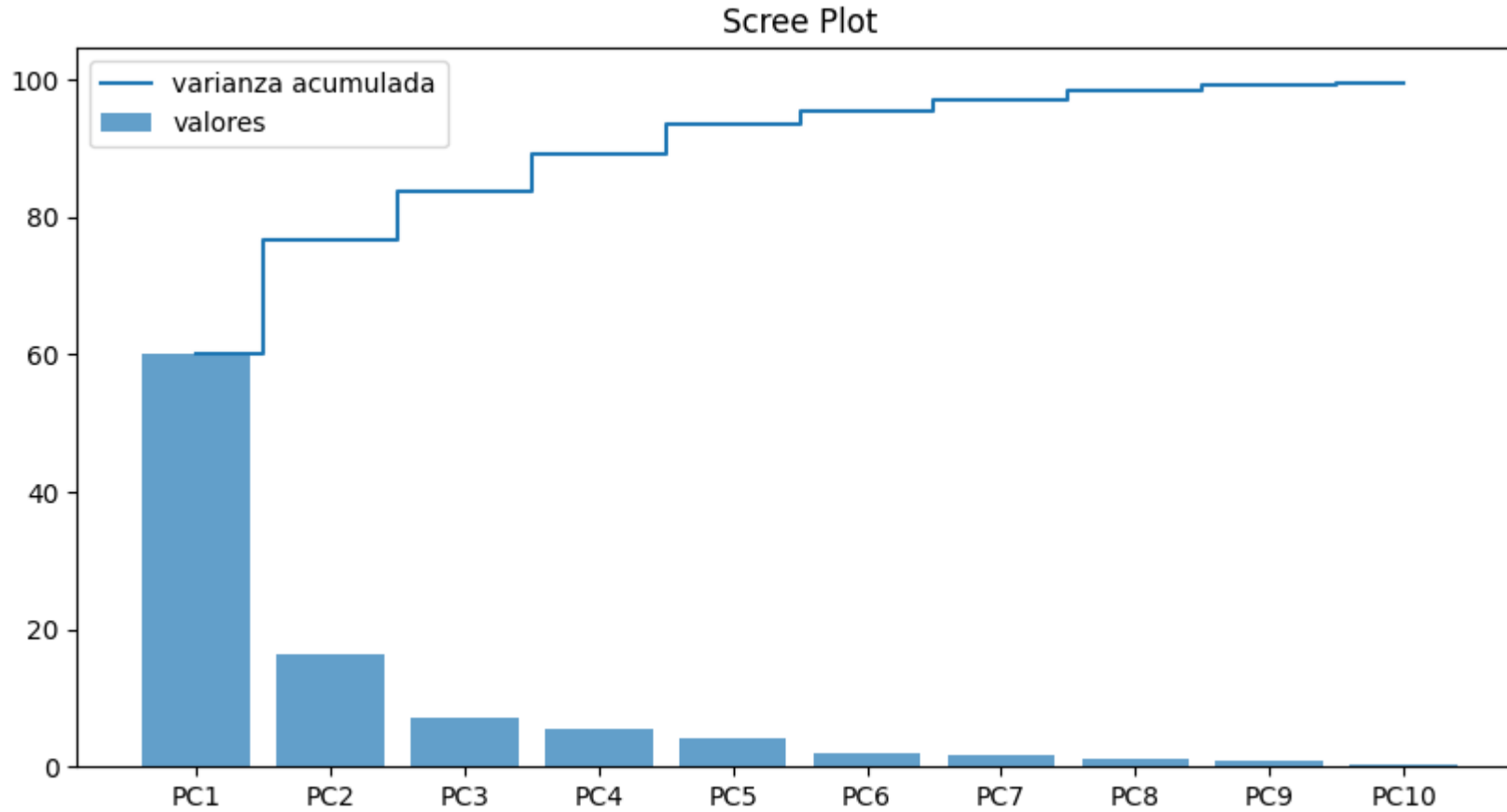
Ejemplo: iris-extended dataset



Paso no incluido en PCA: Eigendescomposición (autovalores y autovectores) calculada a partir de la matriz de correlación de las variables del dataset.

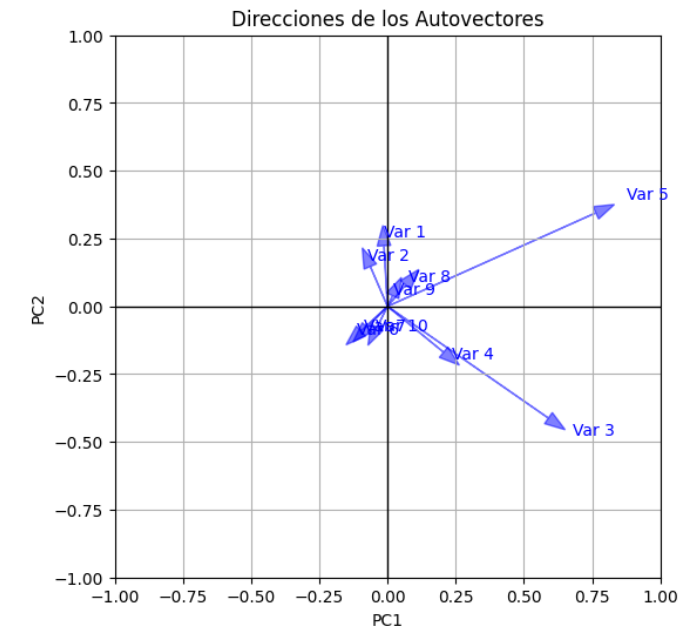
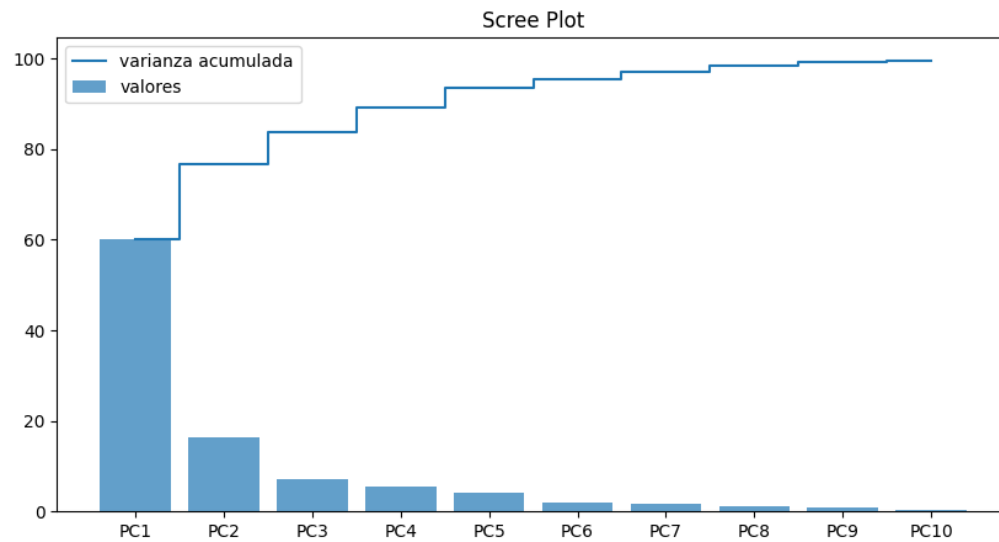
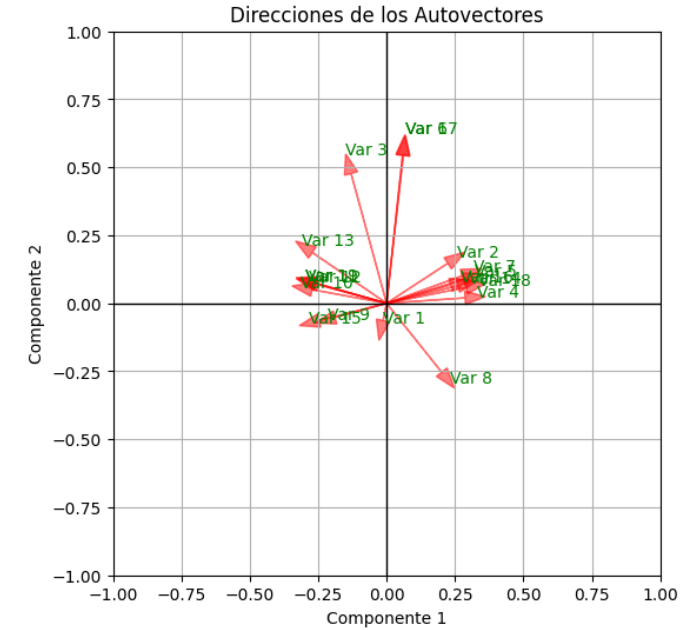
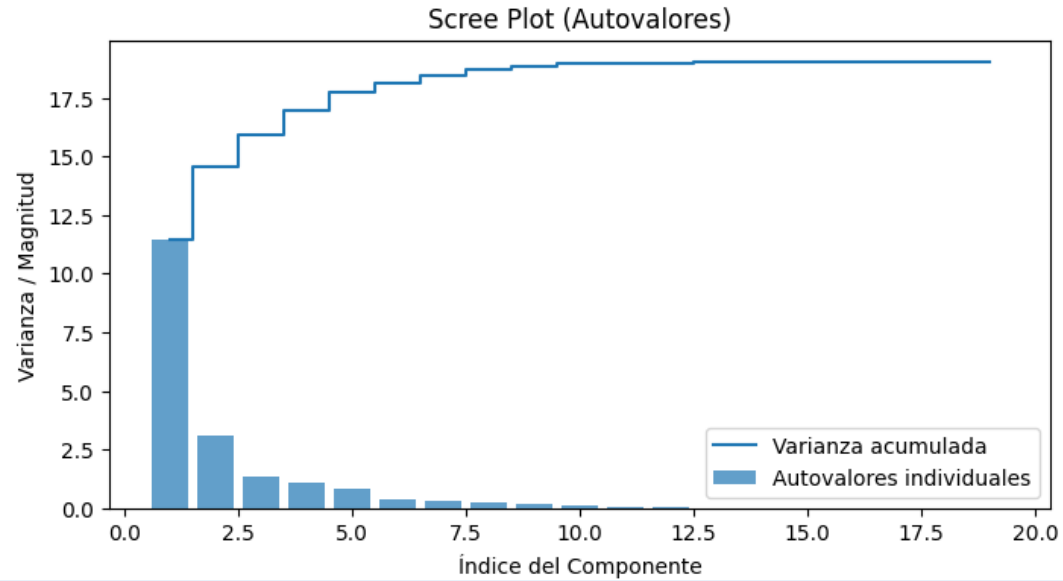


Ejemplo: iris-extended dataset

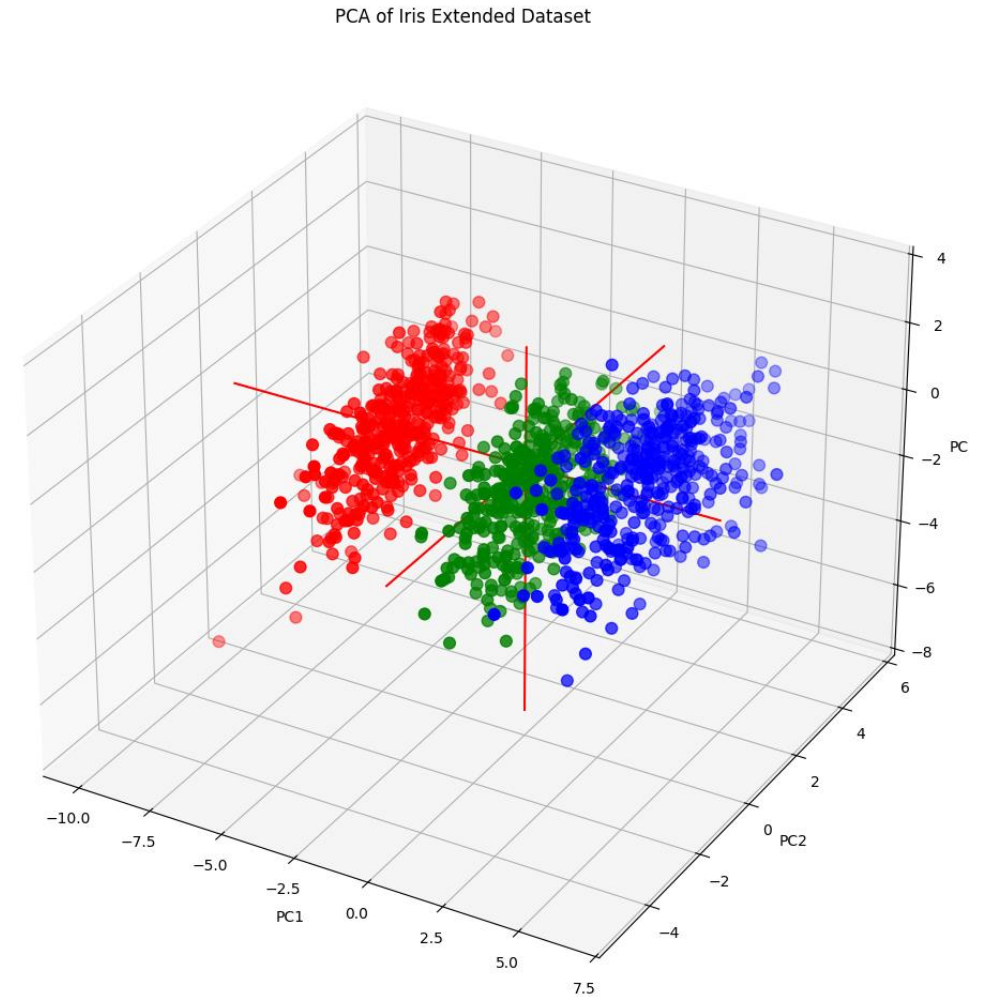
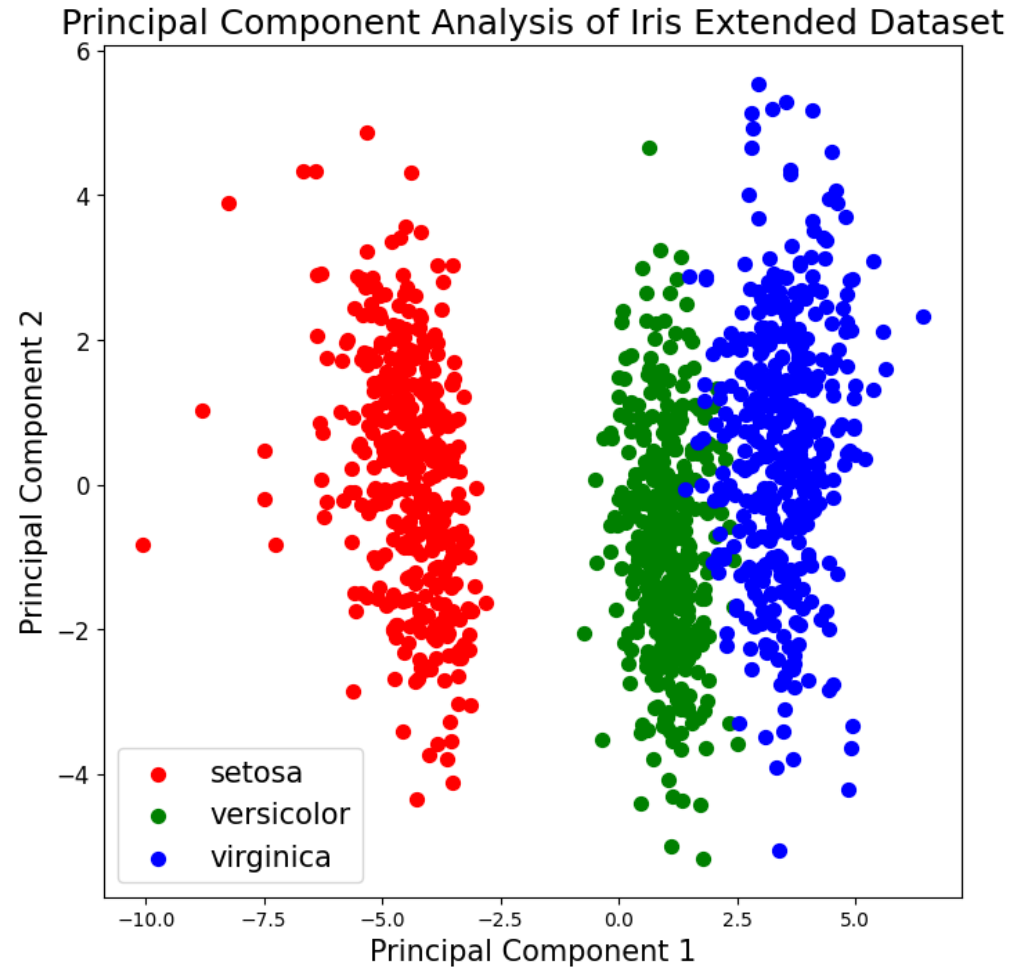


PC1: 60.23%
PC2: 16.39%
PC3: 7.01%
PC4: 5.54%
PC5: 4.26%
PC6: 1.96%
PC7: 1.68%
PC8: 1.27%
PC9: 0.85%
PC10: 0.43%

Ejemplo: iris-extended dataset



Ejemplo: iris-extended dataset



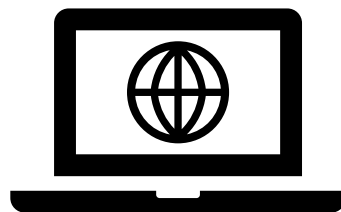
- Análisis de Componentes Principales (PCA)
- Incrustación estocástica de vecinos distribuida en t (t-SNE)
- Análisis Discriminante Lineal (LDA)

Incrustación estocástica de vecinos distribuida en t (t-SNE, t-Distributed Stochastic Neighbor Embedding)

Es un método de reducción de dimensionalidad y aprendizaje automático **no** lineal, que se utiliza para visualizar datos de alta dimensión proyectándolos en un espacio 2D o 3D.

Preserva estructuras locales, asegurando que puntos de datos similares estén cerca unos de otros, lo cual lo hace ideal para detectar clusters naturales en los datos.

SNE



Variante con distribución t

Stochastic Neighbor Embedding

Geoffrey Hinton and Sam Roweis
Department of Computer Science, University of Toronto
10 King's College Road, Toronto, M5S 3G5 Canada
{hinton,roweis}@cs.toronto.edu

Hinton, Geoffrey; Roweis, Sam (January 2002). Stochastic neighbor embedding (PDF). Neural Information Processing Systems.

https://papers.nips.cc/paper_files/paper/2002/file/6150ccc6069bea6b5716254057a194ef-Paper.pdf

Journal of Machine Learning Research 9 (2008) 2579-2605

Submitted 5/08; Revised 9/08; Published 11/08

Visualizing Data using t-SNE

Laurens van der Maaten
*TiCC
Tilburg University
P.O. Box 90153, 5000 LE Tilburg, The Netherlands*

LVDMAATEN@GMAIL.COM

Geoffrey Hinton
*Department of Computer Science
University of Toronto
6 King's College Road, M5S 3G4 Toronto, ON, Canada*

HINTON@CS.TORONTO.EDU

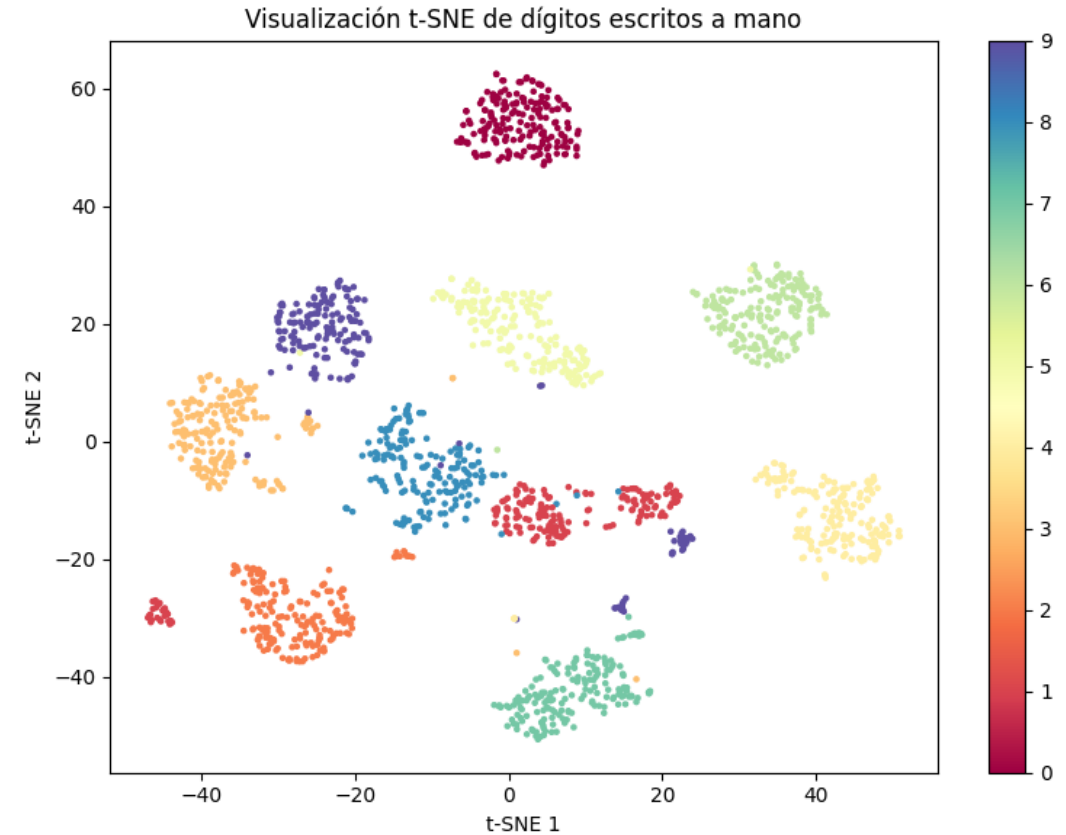
Van der Maaten, L.J.P.; Hinton, G.E. (Nov 2008). "Visualizing Data Using t-SNE" (PDF). Journal of Machine Learning Research. 9: 2579–2605.

<https://jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf>

t-SNE proyecta los datos en un espacio de menor dimensión (usualmente 2D o 3D) de manera que:

- Los puntos que estaban cerca en el espacio original sigan estando cerca en la nueva representación.
- Los puntos lejanos en el espacio original se separen, preservando la estructura de los clusters.

Convierte distancias euclidianas de alta dimensión entre puntos de datos en probabilidades condicionales que representan similitudes y luego crea un mapa similar de baja dimensión.



1. Mide cómo de cercanos son los puntos entre sí en el espacio original

Se define una probabilidad de similitud entre los puntos x_i o x_j usando una distribución Gaussiana centrada en cada punto. La probabilidad condicional de que x_j sea un vecino de x_i con un ancho de banda σ .

Los puntos que están muy separados tienen menos probabilidad de que se elijan en comparación con los puntos que están muy juntos.

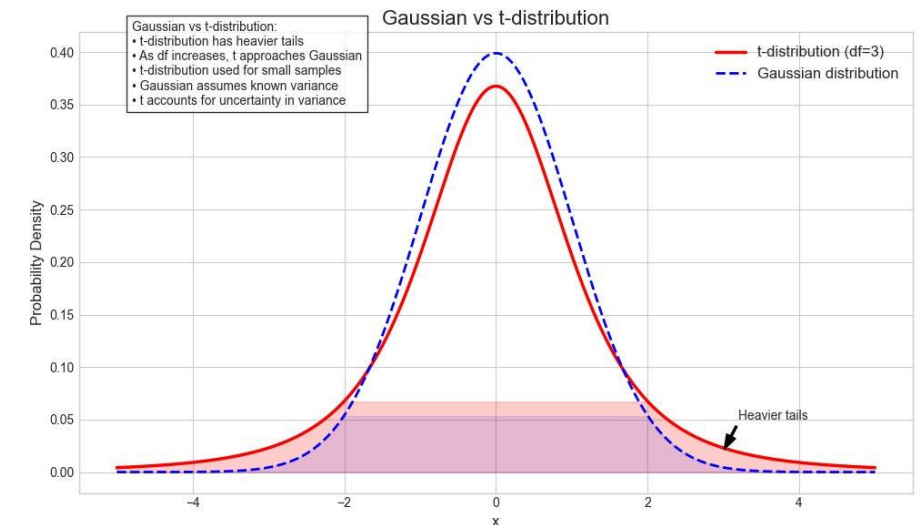
$$P_{j|i} = \frac{\exp\left(\frac{-\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(\frac{-\|x_i - x_k\|^2}{2\sigma_i^2}\right)} \rightarrow p_{ij} = \frac{P_{j|i} + P_{i|j}}{2N}$$

2. Mapear puntos de datos de mayor dimensión en un espacio de menor dimensión

Para que las relaciones entre los puntos en el espacio reducido sean similares a las del espacio original, las similitudes se modelan con una distribución t-Student con un grado de libertad en lugar de una Gaussiana.

Esta aproximación permite minimizar el problema de aglomeración (*crowding problem*), ya que la t-Student asigna mayor probabilidad a distancias más largas en el espacio reducido, debido a que tiene colas más pesadas => mejor separación de clústeres.

$$q_{i|j} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$



3. Minimizar la divergencia entre la distribución de probabilidad de la dimensión original y la de la dimensión inferior.

El objetivo final es encontrar las posiciones en el nuevo espacio de tal manera que la distribución de similitud en el espacio reducido (q) sea lo más parecida posible a la del espacio original (p). Para medir qué tan diferentes son, se usa la divergencia de Kullback-Leibler.

Para minimizar C , necesitamos calcular su gradiente respecto a los puntos en el espacio de baja dimensión.

Se usa en descenso de gradiente estocástico (SGD) para actualizar iterativamente las posiciones de los puntos en el espacio reducido.

La incrustación de dimensiones inferiores se optimiza hasta alcanzar un estado estable.

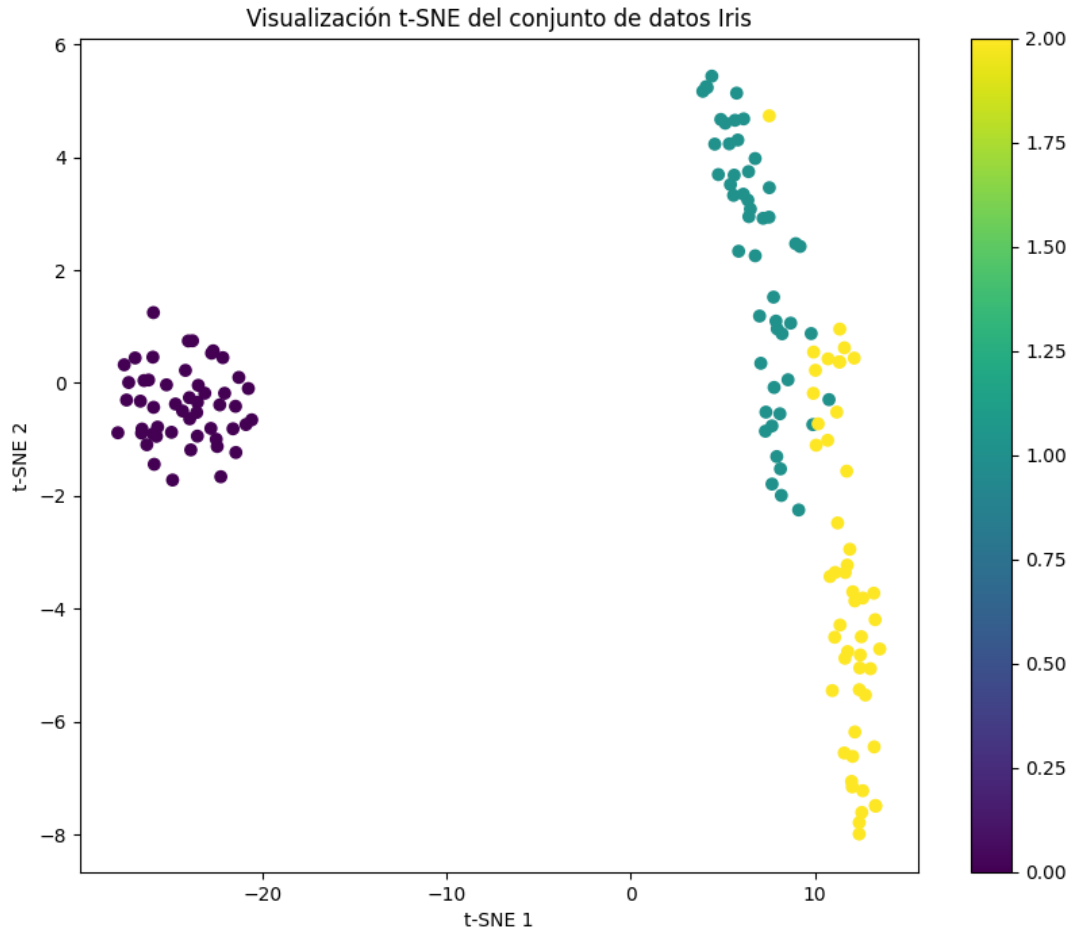
$$C = \sum_{i \neq j} p_{ij} \cdot \log \left| \frac{p_{ij}}{q_{ij}} \right|$$

La divergencia de Kullback-Leibler mide cómo una distribución de probabilidad q diverge de una segunda distribución de probabilidad esperada p . Se define como la esperanza del logaritmo del cociente de las dos distribuciones de probabilidad.

El gradiente de C con respecto a y es:

$$\frac{\partial C}{\partial y_i} = 4 \sum_{i \neq j} (p_{ij} - q_{ij}) \cdot q_{ij} \cdot Z(y_i - y_j)$$

donde Z es el factor de normalización de q (la suma de todos los denominadores en la fórmula de q). El factor 4 proviene de la derivada de la distribución t de Student utilizada en el espacio reducido.



El mapa de calor de la derecha, también conocido como barra de color o leyenda, indica la relación entre los colores en el diagrama de dispersión t-SNE y los valores numéricos que representan.

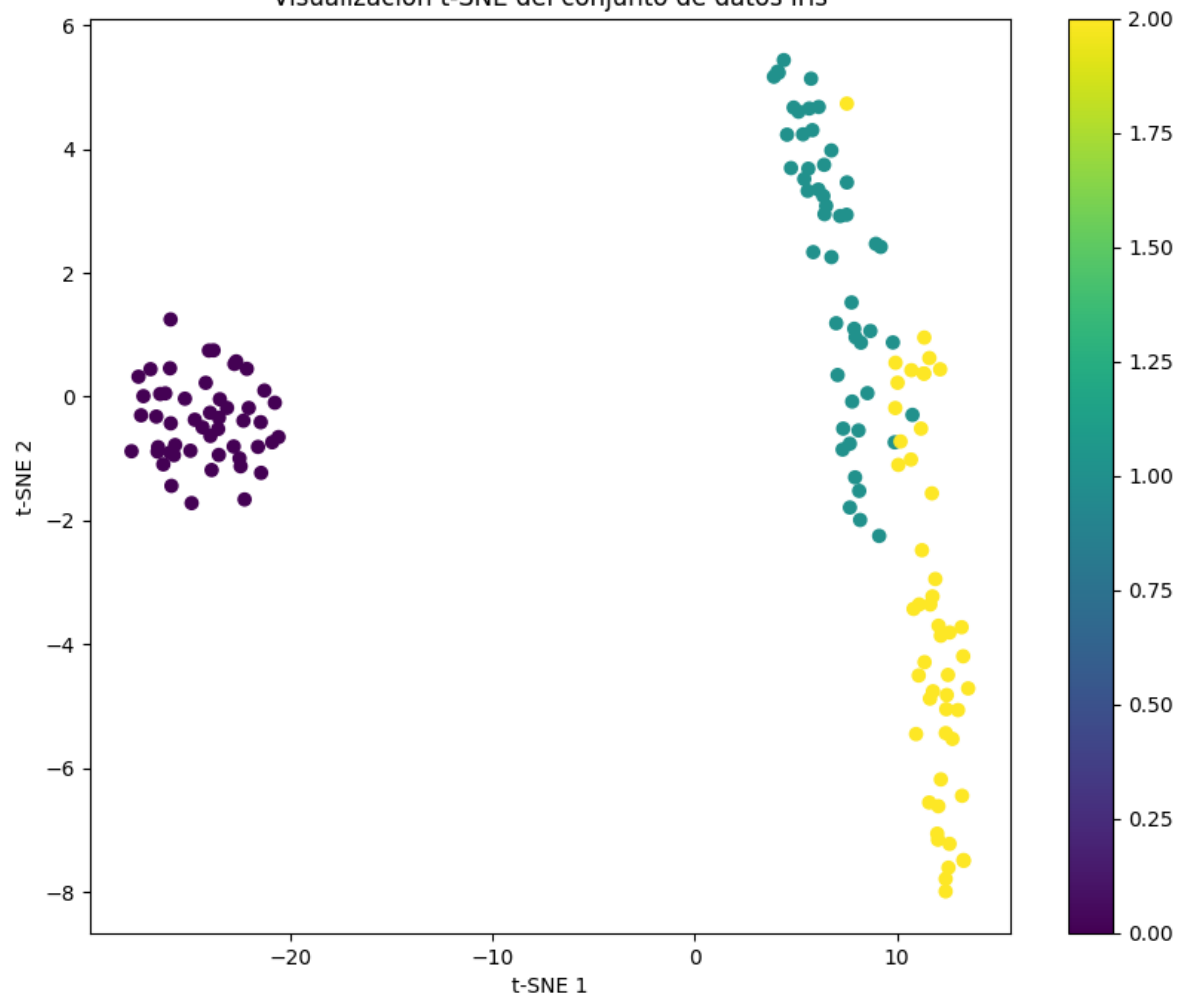
Eje t-SNE 1 y t-SNE 2: Son las dos dimensiones resultantes de la aplicación del algoritmo t-SNE. Cada punto en el gráfico representa una flor Iris, y su posición en estos ejes se basa en las características originales de las flores (longitud y ancho del sépalo y pétalo).

Colores: Los colores representan las tres especies de flores Iris en el conjunto de datos.

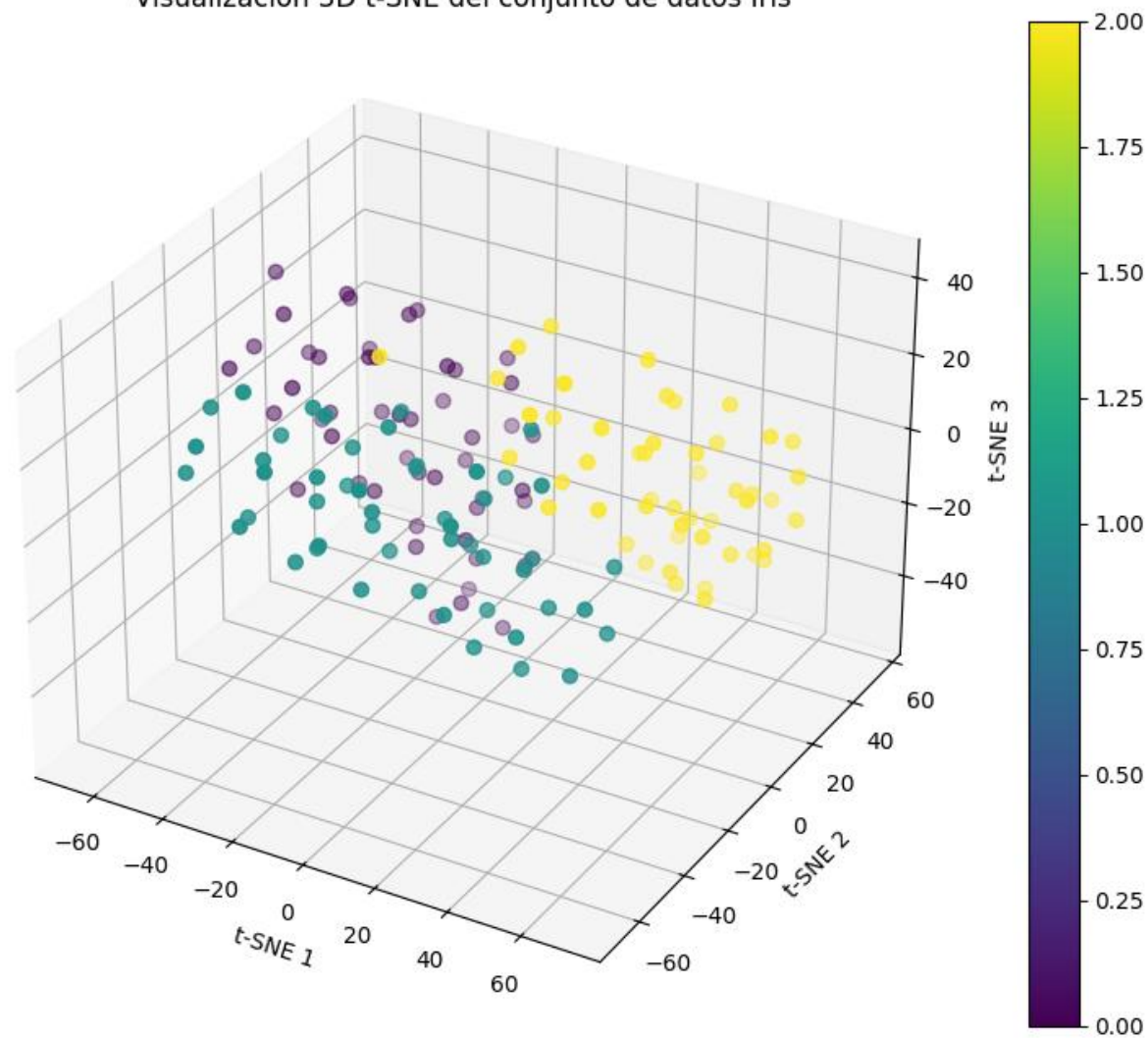
Clusters: Hay al menos dos clusters distintos, lo que indica que t-SNE ha logrado separar bien las algunas especies basándose en sus características.

Interpretación general: El gráfico muestra cómo t-SNE ha reducido la dimensionalidad de los datos originales (que tenían cuatro características) a dos dimensiones, al tiempo que conserva la estructura de los datos. En este caso, las especies de flores Iris que son más similares entre sí se agrupan en el mismo cluster en el gráfico t-SNE.

Visualización t-SNE del conjunto de datos Iris



Visualización 3D t-SNE del conjunto de datos Iris



Ejemplo: breast-cancer dataset

Estandarizar los datos

```
scaler = StandardScaler()
df_for_scaler = breast_dataset.loc[:, features].values
x_scaled = scaler.fit_transform(df_for_scaler)
```

El uso de StandardScaler equivale a usar la matriz de correlación para estandarizar.

Aplicar tSNE

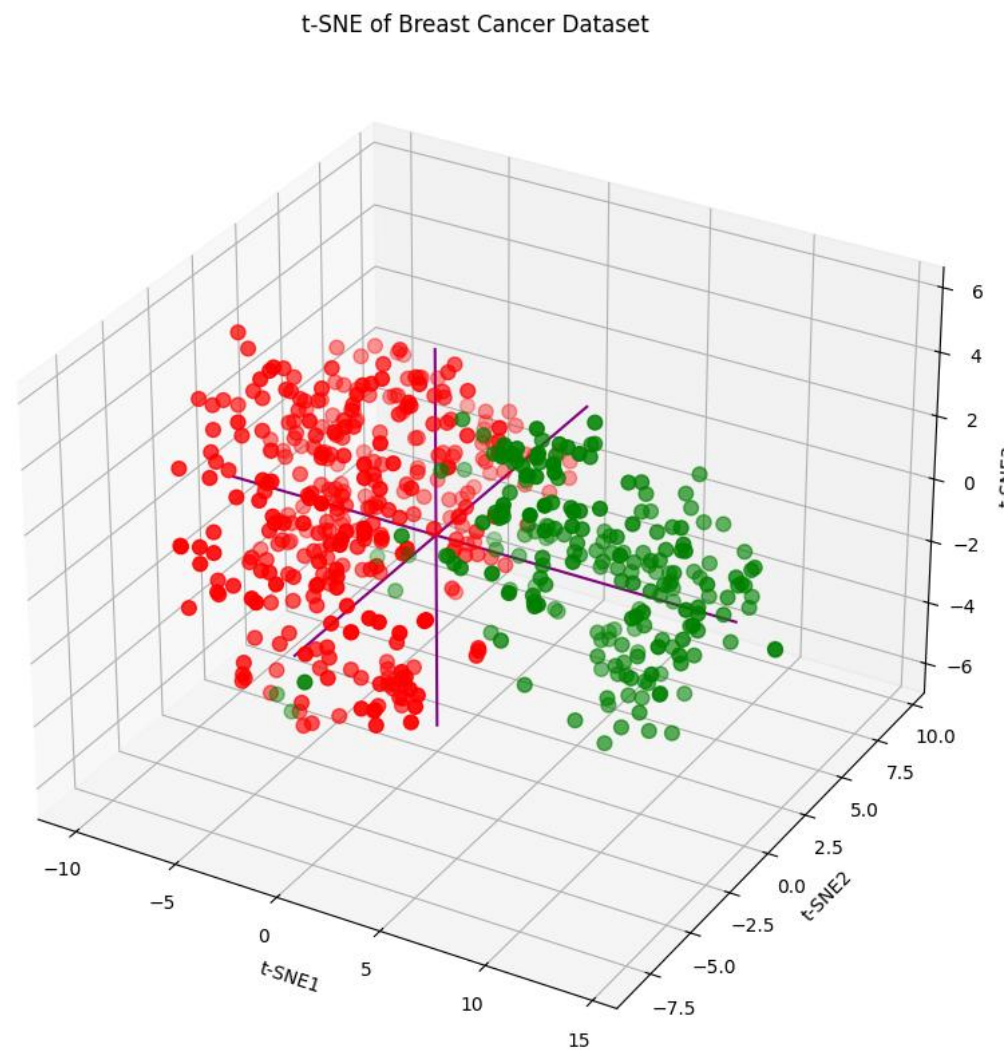
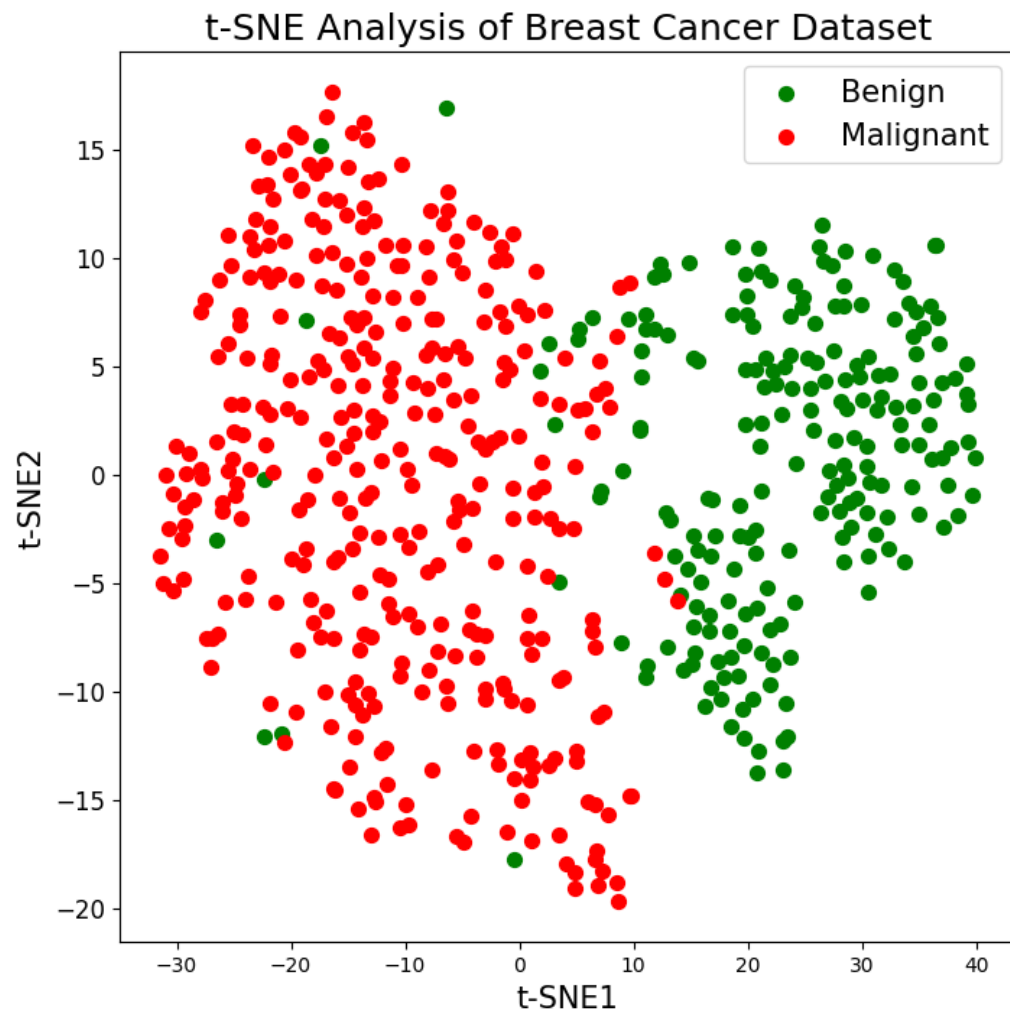
```
n_comp = 2 #2-3
tsne = TSNE(n_components=n_comp, perplexity=30, random_state=42)
tsne_components = tsne.fit_transform(x_scaled)

#Generar un nuevo dataframe con los valores transformados de las 569 muestras
tsne_df = pd.DataFrame(data=tsne_components, columns=[f't-SNE{i}' for i in range(1, n_comp+1)])
```

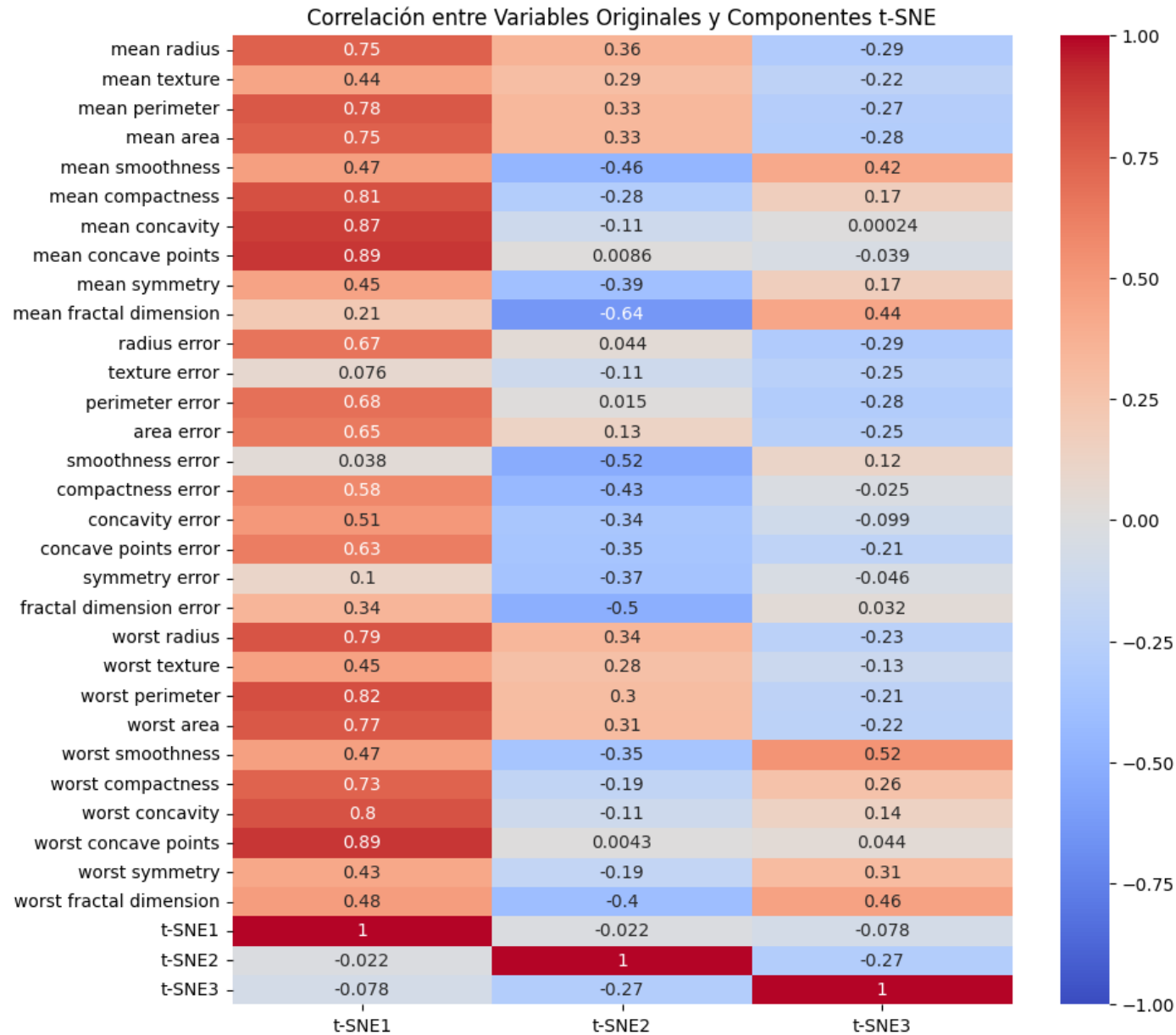
Hiperparámetro importante: perplexity define cuántos vecinos considera al mapear los puntos.

Parámetro	Descripción	Valores recomendados
n_components	Dimensionalidad de la salida	2 o 3
perplexity	Balance entre estructuras locales y globales	5 - 50 (por defecto 30)
learning_rate	Velocidad de actualización en el SGD	10 - 1000 (por defecto 200)
n_iter	Número de iteraciones	250 - 1000 (por defecto 1000)
metric	Distancia usada en el espacio original	"euclidean", "cosine", "correlation"
method	Algoritmo de optimización	"barnes_hut" (rápido) o "exact" (preciso)
early_exaggeration	Separación inicial entre clusters	12 - 32 (por defecto 12)

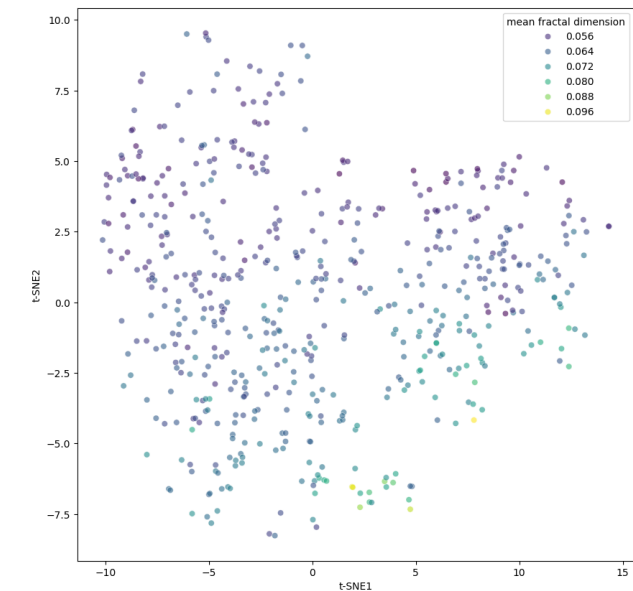
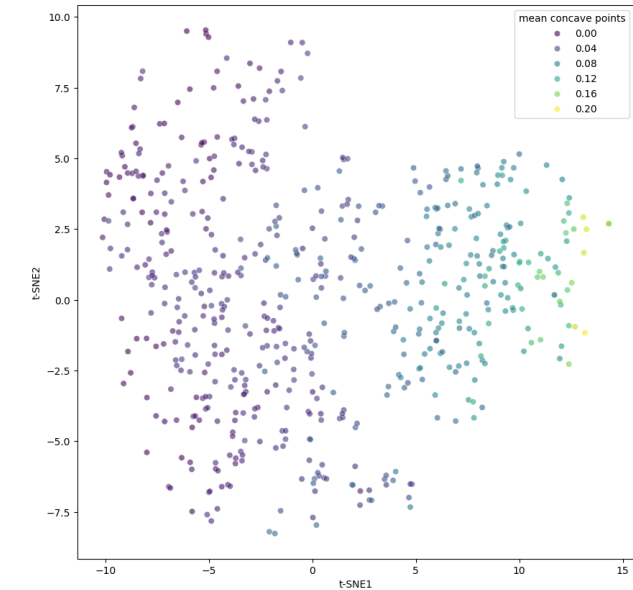
Ejemplo: breast-cancer dataset



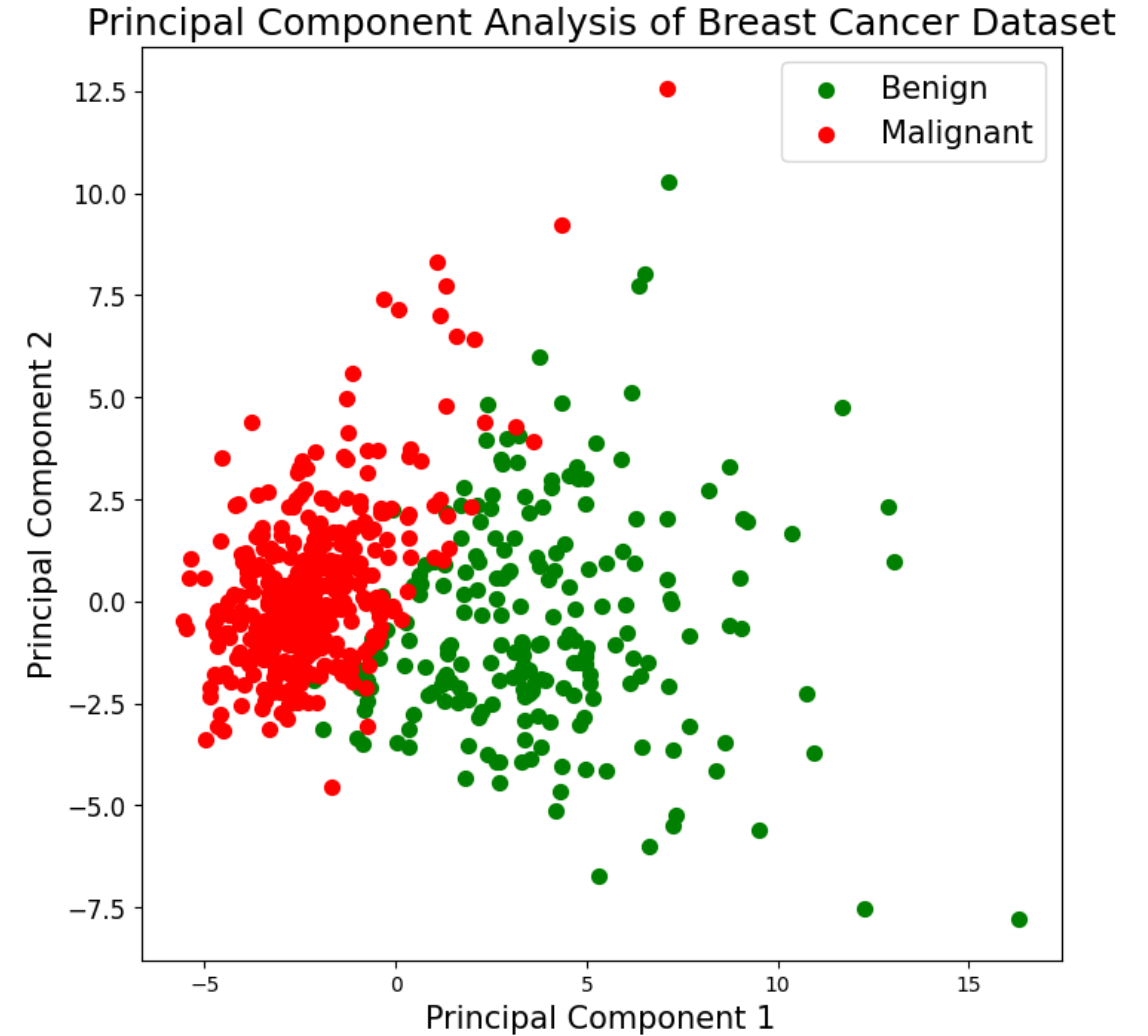
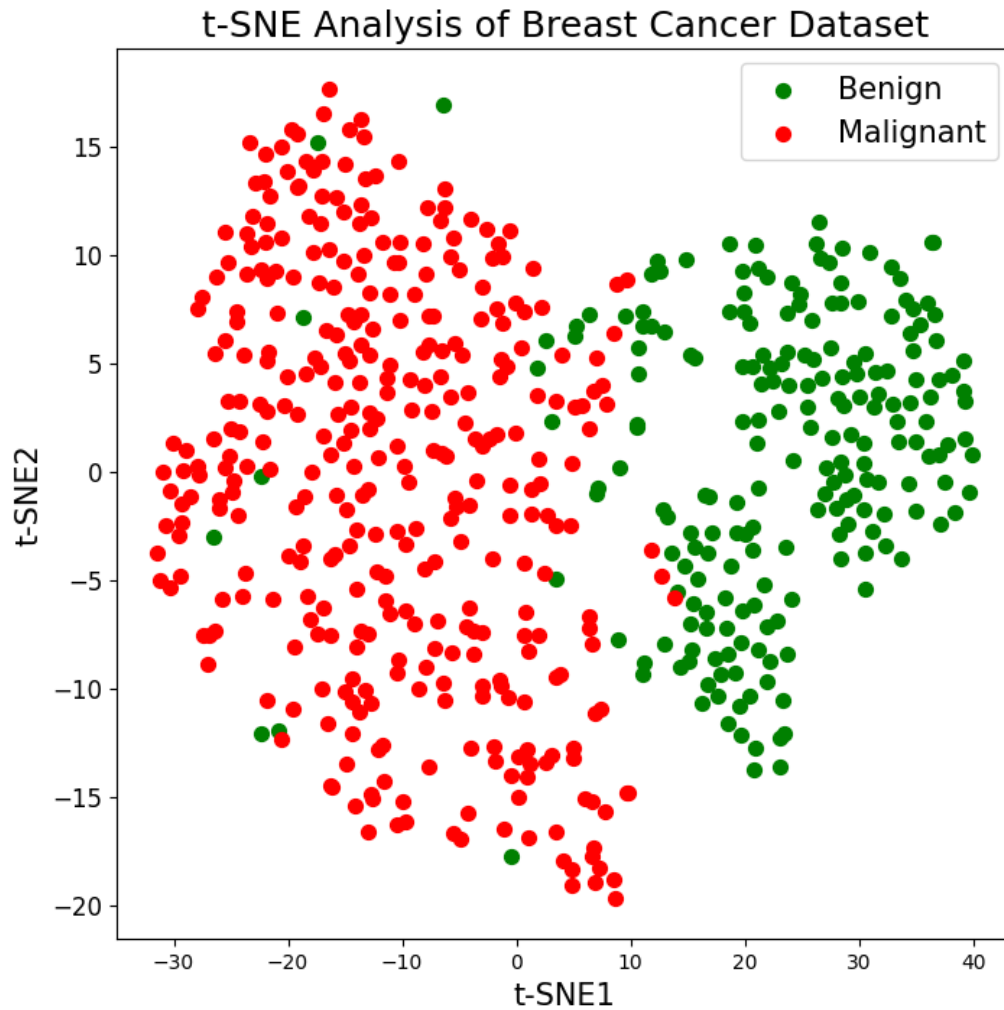
Ejemplo: breast-cancer dataset



Coloreado por variable



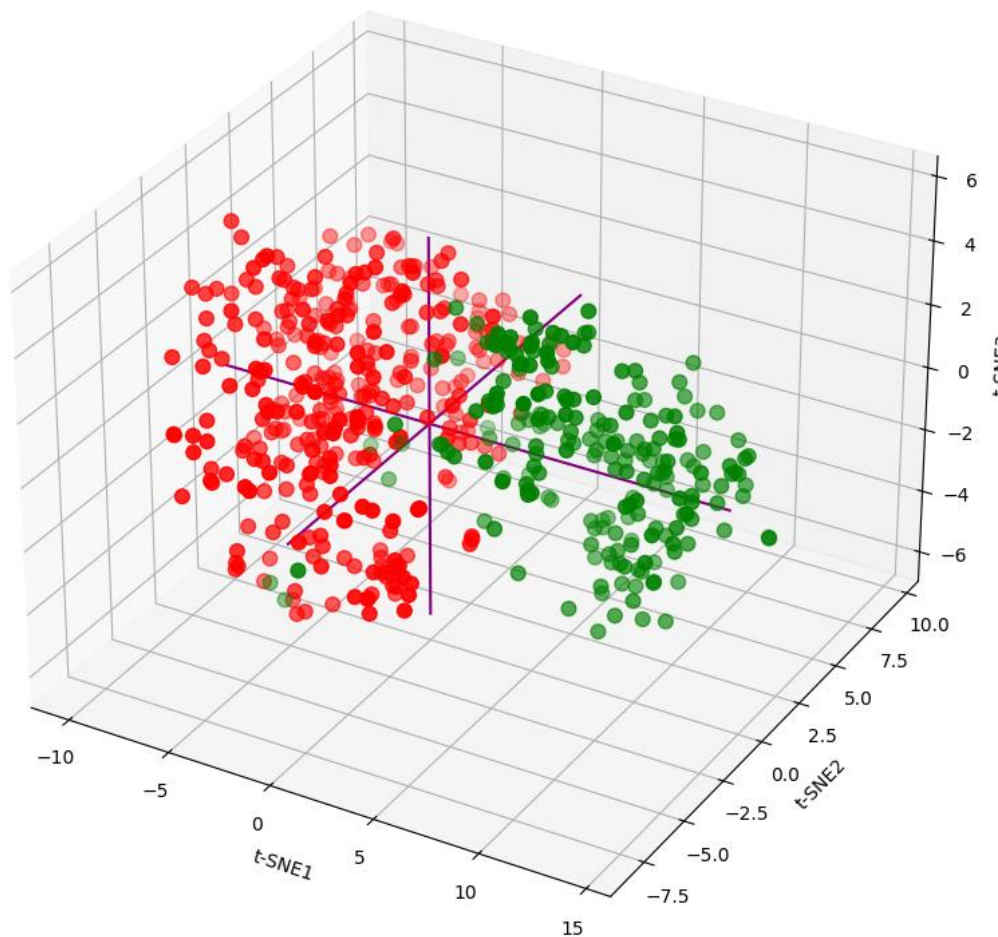
Ejemplo: breast-cancer dataset



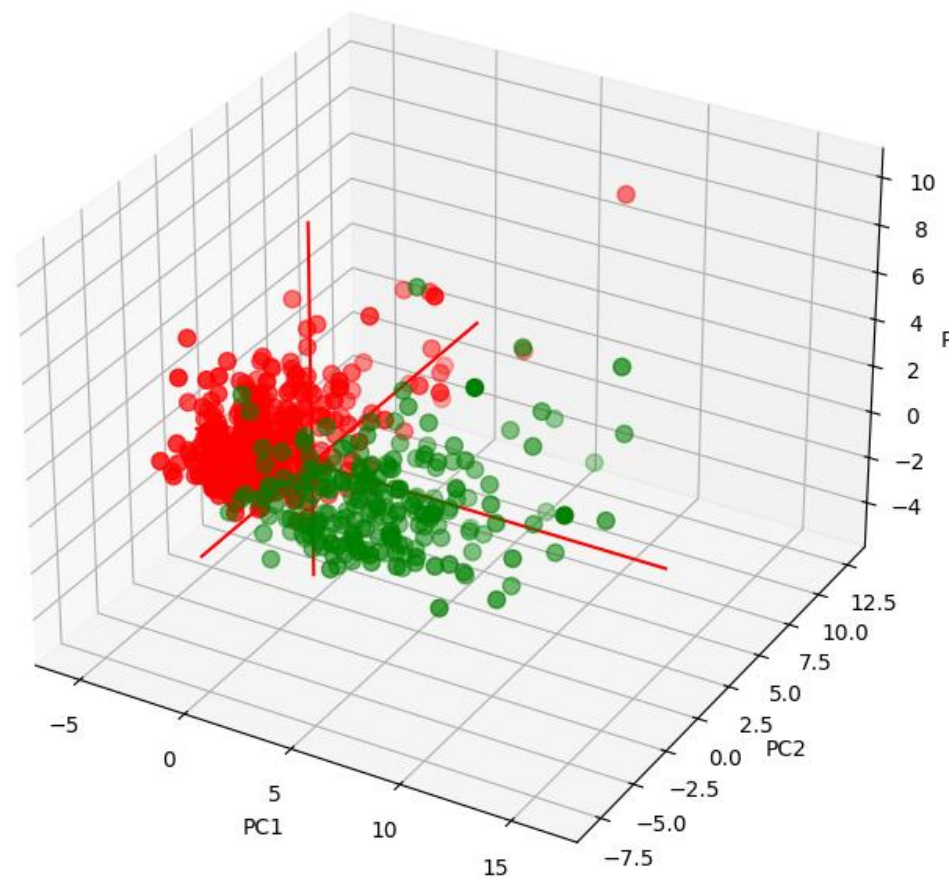
T-SNE vs PCA

Ejemplo: breast-cancer dataset

t-SNE of Breast Cancer Dataset



PCA of Breast Cancer Dataset



T-SNE vs PCA

Ejemplo: iris-extended dataset

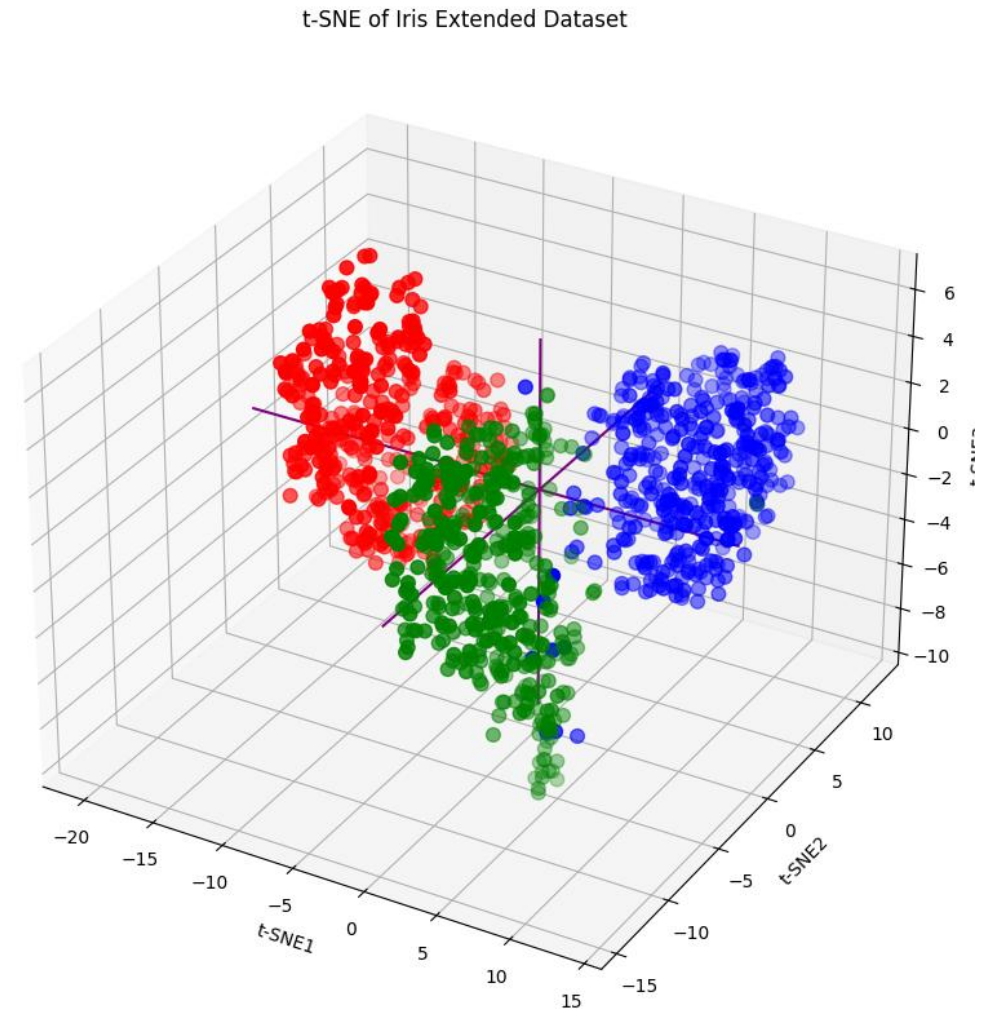
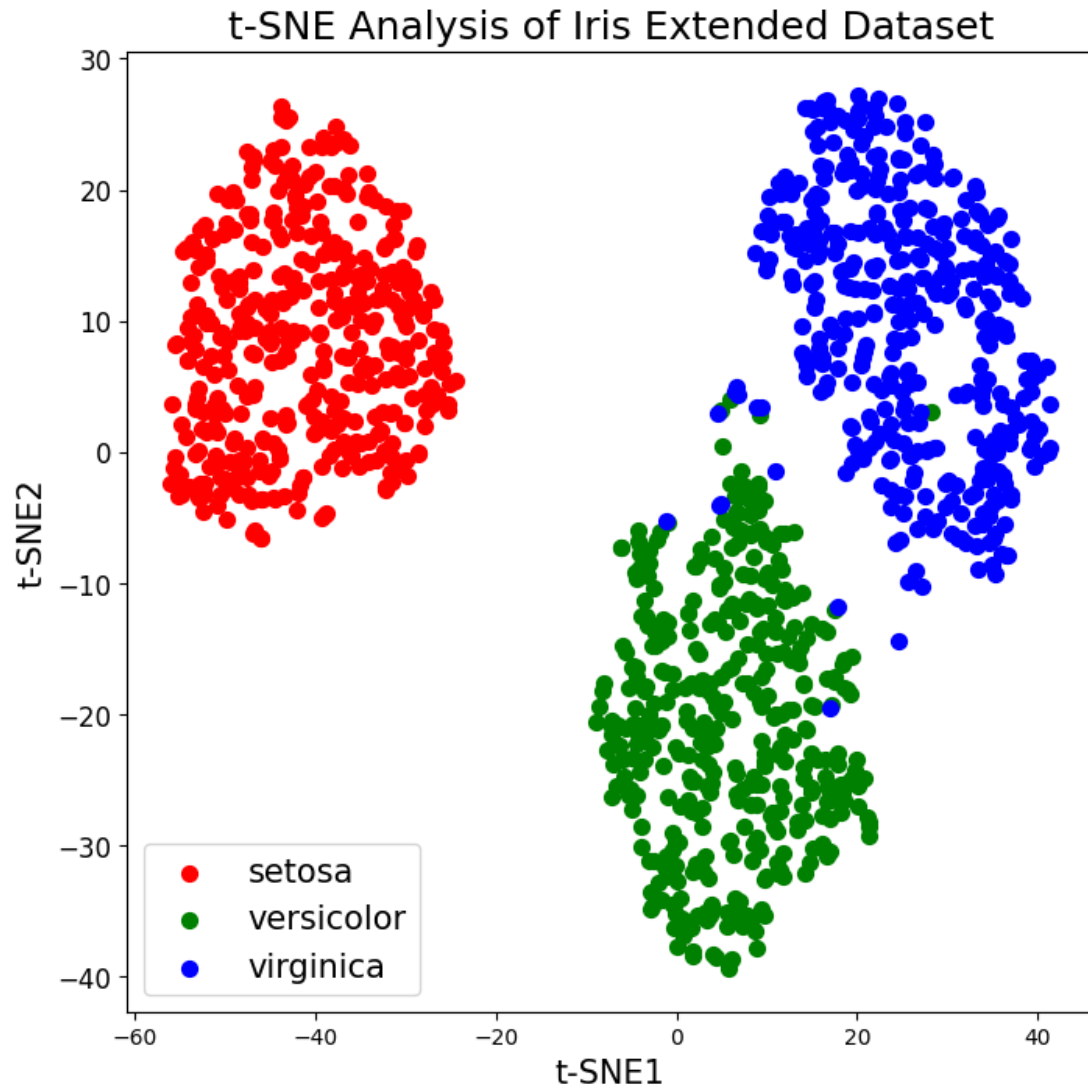
Estandarizar los datos

```
scaler = StandardScaler()  
df_forscaler = iris_numdata.values  
x_scaled = scaler.fit_transform(df_forscaler)
```

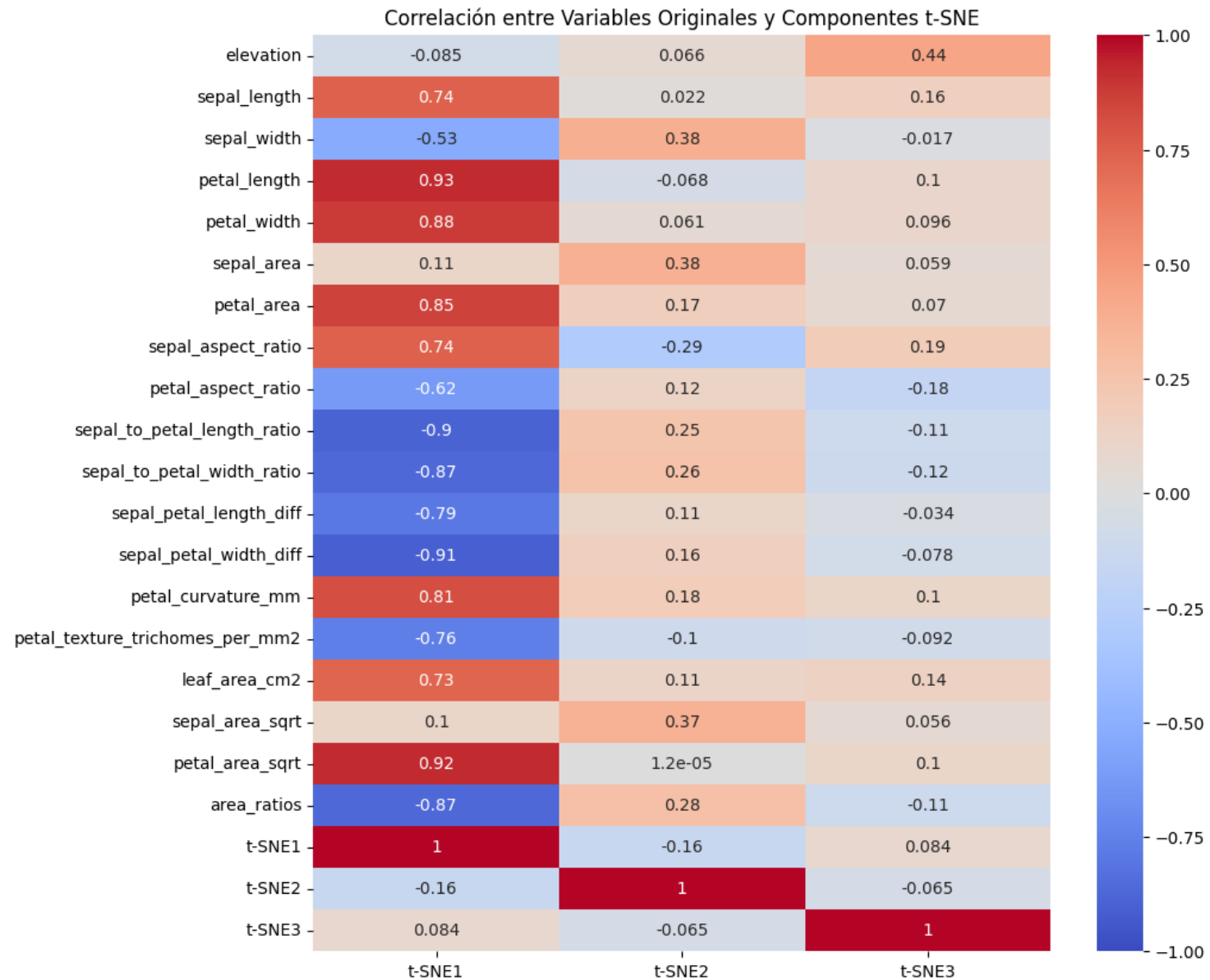
Aplicar tSNE

```
n_comp = 2 #2-3  
tsne = TSNE(n_components=n_comp, perplexity=30, random_state=42)  
tsne_components = tsne.fit_transform(x_scaled)  
  
#Generar un nuevo dataframe con los valores transformados de las 569 muestras  
tsne_df = pd.DataFrame(data=tsne_components, columns=[f't-SNE{i}' for i in range(1, n_comp+1)])
```

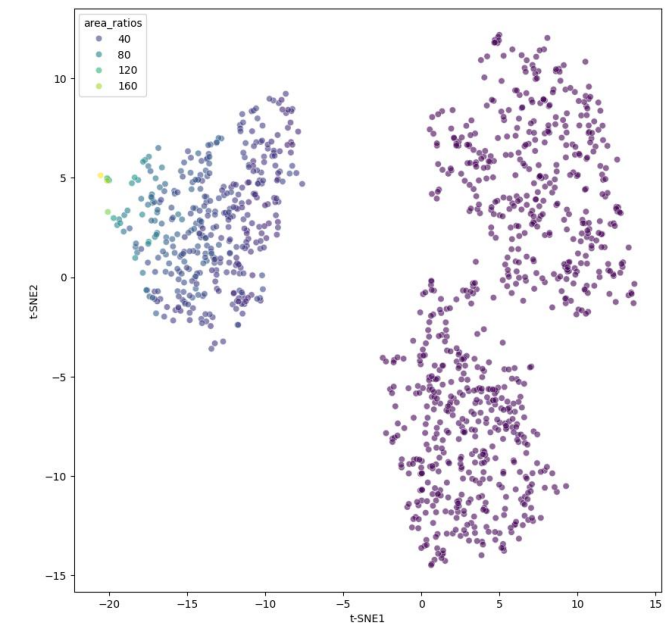
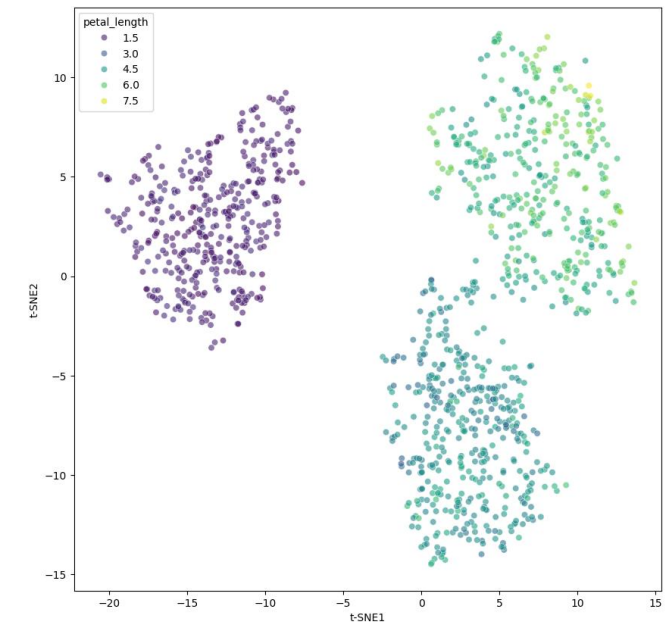
Ejemplo: iris-extended dataset



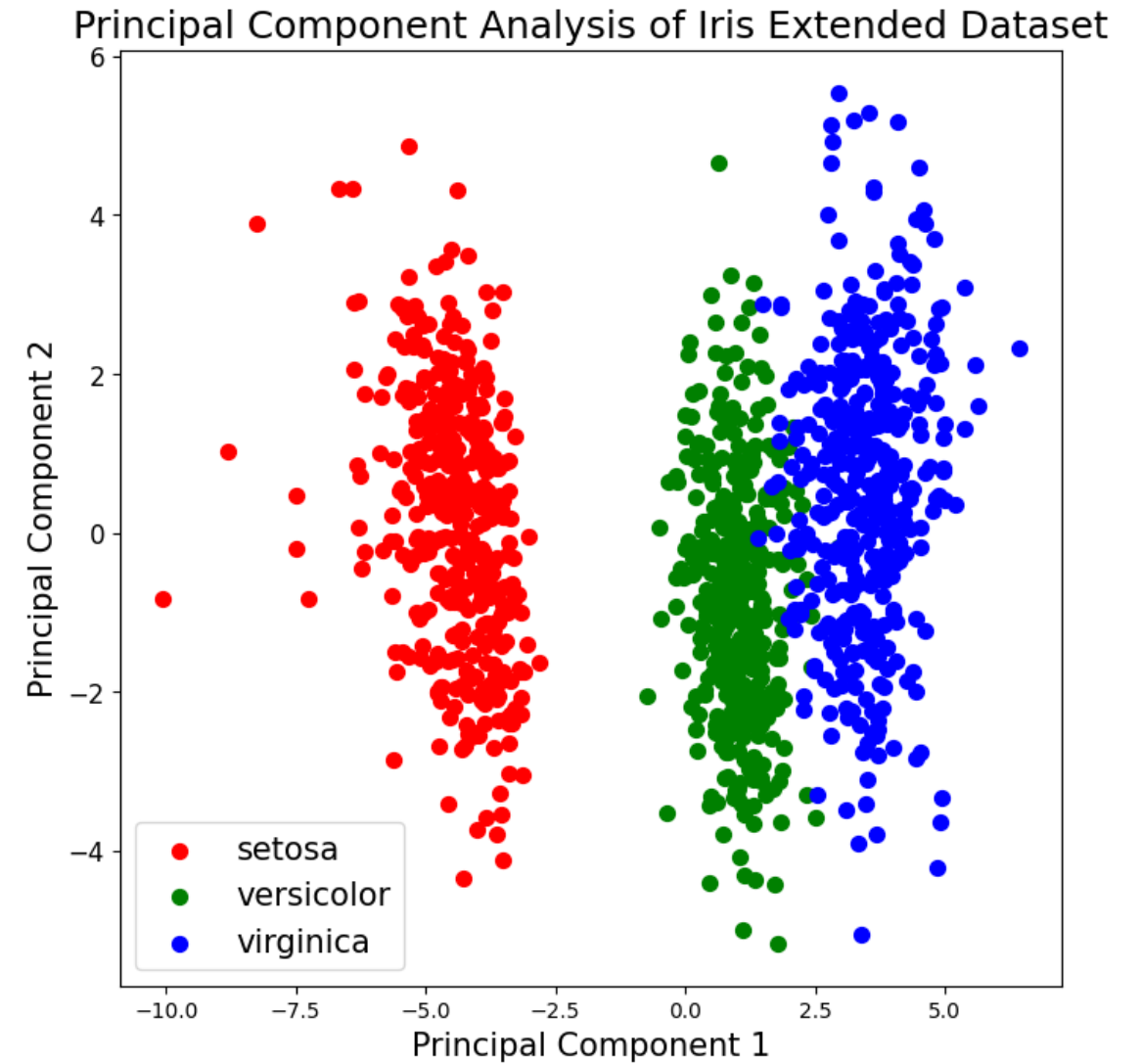
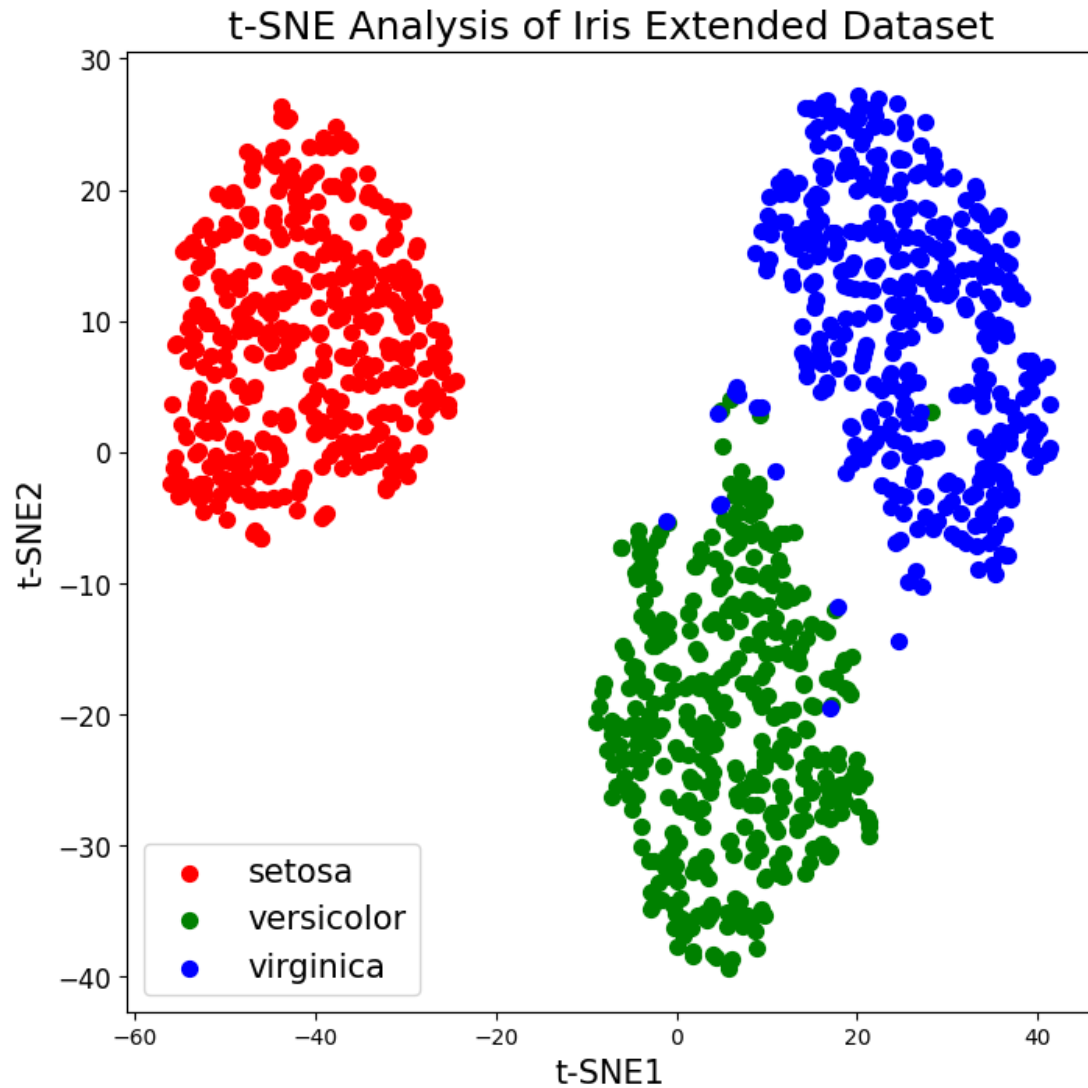
Ejemplo: iris-extended dataset



Coloreado por variable



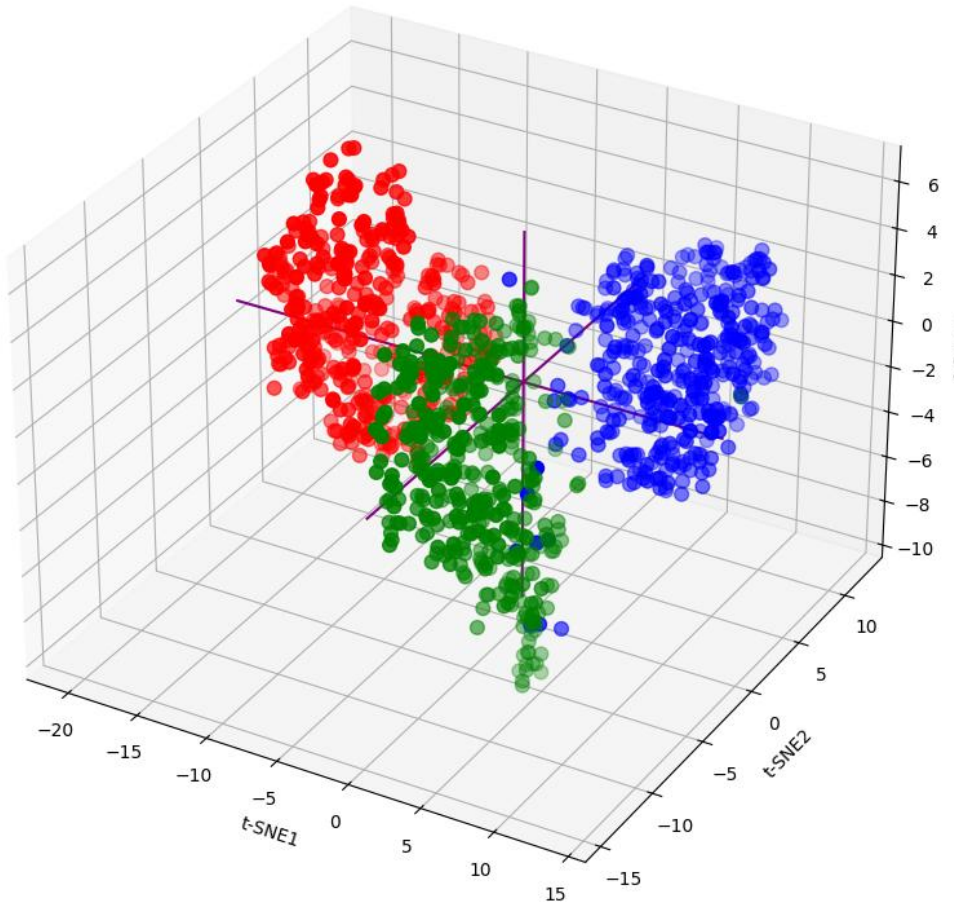
Ejemplo: iris-extended dataset



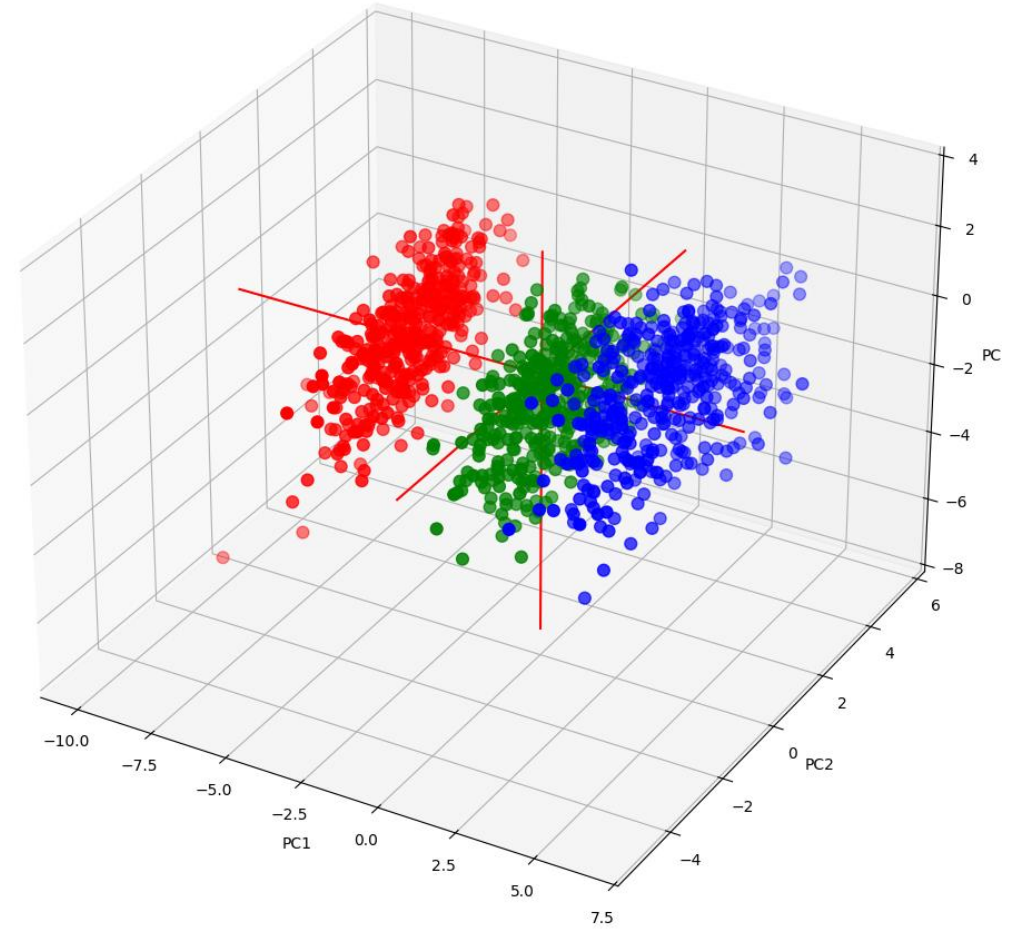
T-SNE vs PCA

Ejemplo: iris-extended dataset

t-SNE of Iris Extended Dataset



PCA of Iris Extended Dataset



T-SNE vs PCA

t-SNE (t-Distributed Stochastic Neighbor Embedding)

- **Uso:** Principalmente para **visualización** de datos complejos de alta dimensión en 2D o 3D.
- **Escenarios:**
 - **Visualización:** Explorar datos complejos (genómica, imágenes) para identificar clusters ocultos o estructuras locales.
 - **Relaciones no lineales:** Ideal cuando los datos tienen estructuras no lineales que PCA no puede captar.
- **Ventaja:** Conserva las relaciones locales (puntos cercanos en alta dimensión permanecen cercanos en baja).
- **Desventaja:** Computacionalmente costoso, no mantiene la estructura global y no se debe usar para preprocesamiento antes de entrenar un modelo.

- Análisis de Componentes Principales (PCA)
- Incrustación estocástica de vecinos distribuida en t (t-SNE)
- Análisis Discriminante Lineal (LDA)

Análisis Discriminante Lineal (LDA, Linear Discriminant Analysis)

Es un método de reducción de dimensionalidad y aprendizaje automático **supervisado**, que proyecta los datos en un subespacio de menor dimensión.

Su objetivo principal es encontrar una combinación lineal de características que maximice la separación entre dos o más clases.

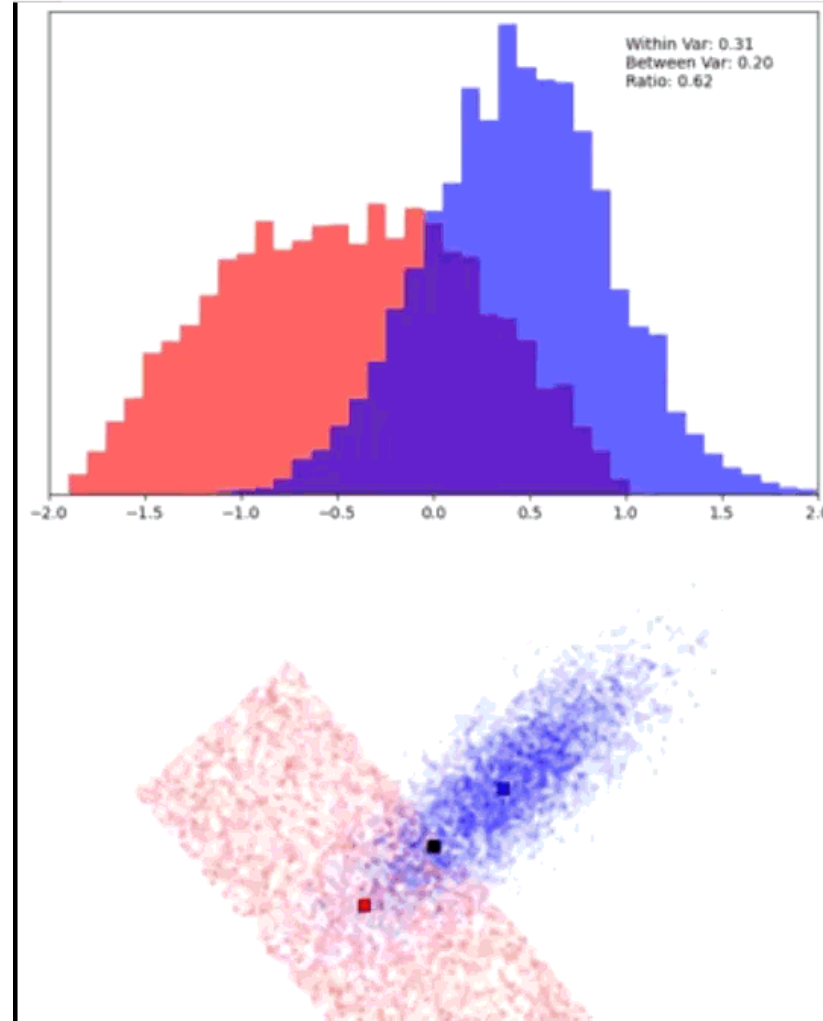


El LDA originariamente fue
desarrollado en 1936
por Ronald Fisher
Inglaterra

Análisis discriminante multiple
Calyampudi Radhakrishna Rao
(1950)
India



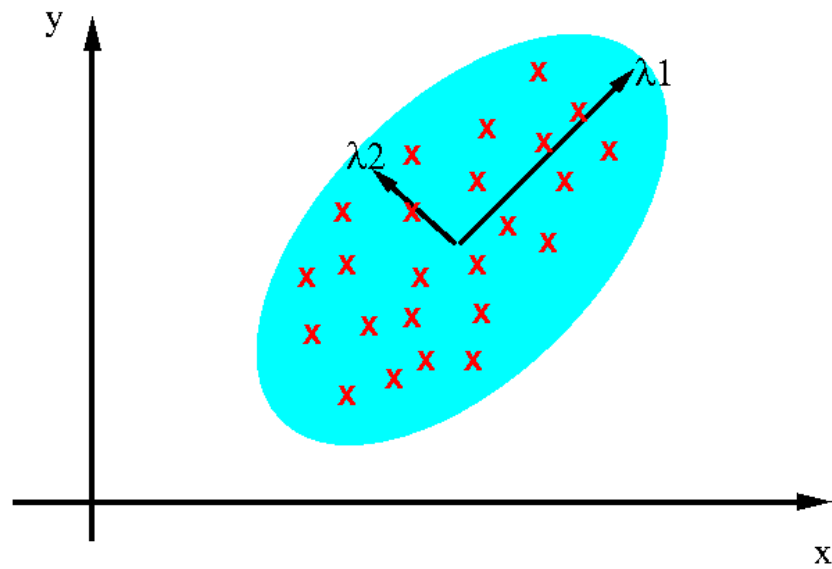
A diferencia de PCA (Análisis de Componentes Principales), que solo maximiza la varianza sin considerar las etiquetas de clase, LDA tiene en cuenta la relación entre las clases y encuentra las direcciones que mejor separan los datos de diferentes categorías.



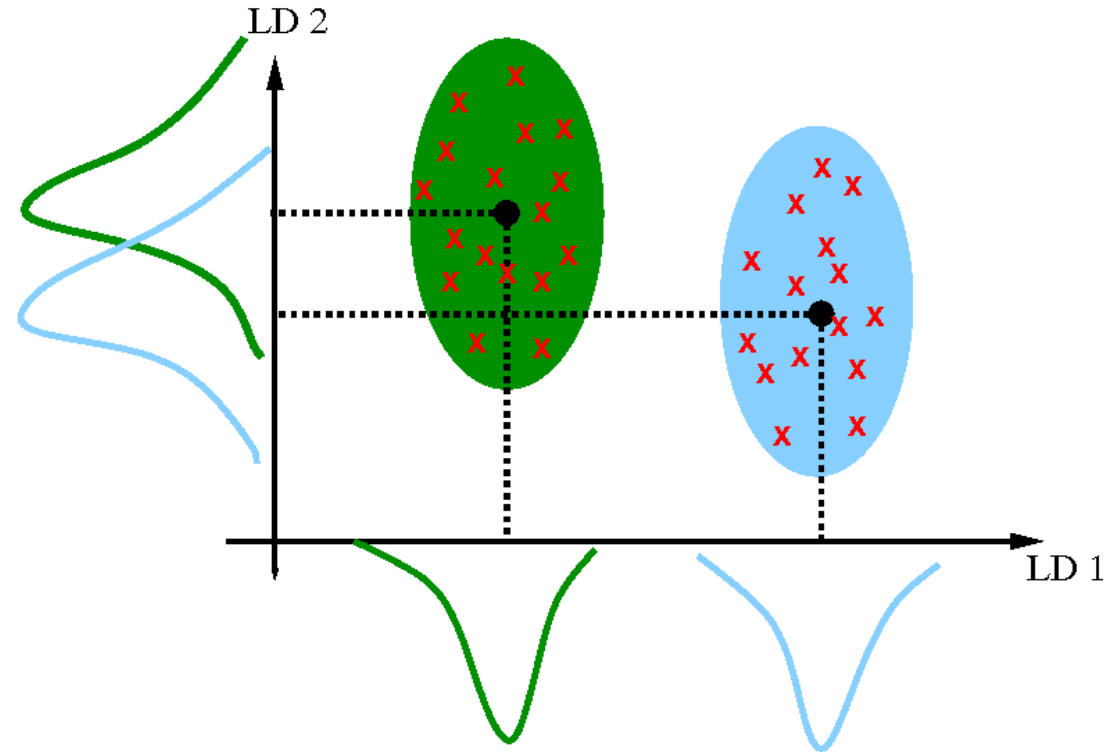
Animación de análisis discriminante lineal.

Dado un conjunto de datos con dos etiquetas, este se proyecta en una línea. La proyección óptima se obtiene cuando se maximiza la razón entre la varianza interclase y la varianza intraclase.

PCA: component axes that maximize the variance



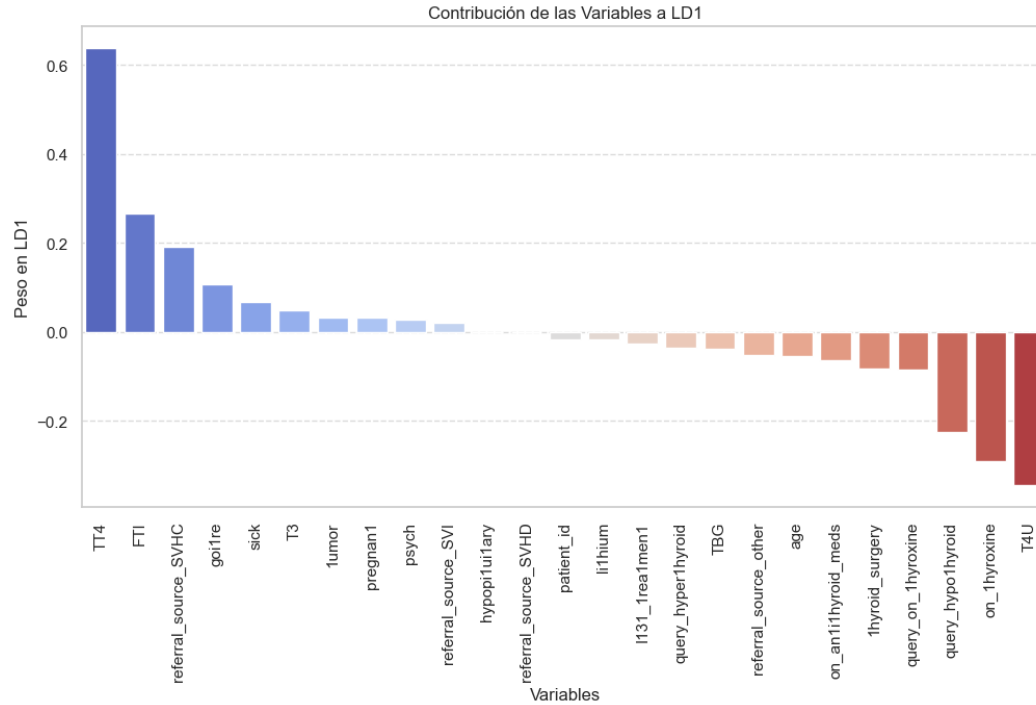
LDA: maximizing the component axes for class-separation



1. Estandarización de los datos: Normalizar los valores para que tengan media 0 y varianza 1.
2. Cálculo de la media de cada clase.
3. Construcción de SW y SB :
 - SW mide la dispersión dentro de cada clase.
 - SB mide la dispersión entre las medias de las clases.
4. Cálculo de los autovalores y autovectores: Se obtiene la transformación que maximiza la separabilidad.
5. Proyección de los datos al nuevo espacio de menor dimensión.

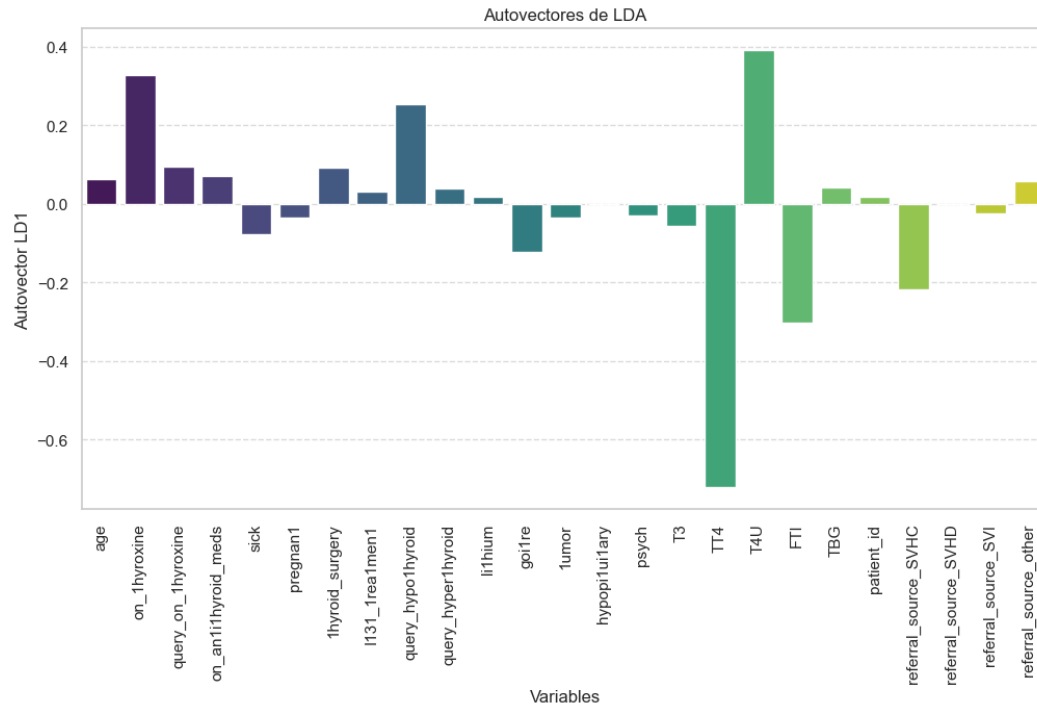
Supuestos para que LDA funcione eficazmente

- Distribución normal (gaussiana) de los datos
- Separabilidad lineal: los datos deben poder separarse mediante un plano.



Las cargas en LDA (lda.coef_): indican cuánto influye cada variable en la predicción de una clase específica. Se pueden ver como pesos de un modelo lineal para separar clases.

- Valores positivos (+): La variable contribuye a la clasificación en una clase específica.
- Valores negativos (-): La variable contribuye a la clasificación en la otra clase.
- Valores cercanos a 0: La variable tiene poco impacto en la separación.



Los autovectores en LDA (lda.scalings_):
representan las direcciones del espacio donde la separación entre clases es máxima. Es decir, indican cómo se deben transformar los datos para obtener la mejor separación.

Características:

- Son direcciones en el espacio transformado de LDA. No dependen directamente de la clasificación, sino de la estructura de los datos.
- Representan la nueva base del espacio de LDA donde las clases están mejor separadas.

Cómo interpretarlos: Valores altos en un autovector significan que la variable define la nueva dimensión de discriminación. Si una variable tiene un valor cercano a 0 en un autovector, su impacto en la separación de clases es bajo.

Ejemplo: breast-cancer dataset

Estandarizar los datos

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(breastcancer_numdata, features,
test_size=0.2, random_state=42, stratify=features)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Aplicar LDA

```
lda = LinearDiscriminantAnalysis()
# Entrenar LDA y transformar datos
X_train_lda = lda.fit_transform(X_train_scaled, y_train)
X_test_lda = lda.transform(X_test_scaled)
# Ver hiperparámetros usados en el modelo
print("🚀 Hiperparámetros del modelo:")
print(lda.get_params())
```

Hiperparámetro	Valor por defecto	Descripción
solver	'svd'	Método para calcular LDA: 'svd', 'lsqr', 'eigen'.
shrinkage	None	Regularización para evitar sobreajuste ('auto', float, None), para lsqr o eigen.
priors	None	Distribución a priori de las clases (por defecto, se calcula de los datos).
n_components	None	Número de dimensiones en las que reducir los datos ($\leq n_clases - 1$).
store_covariance	False	Guarda la matriz de covarianza (True = más memoria, útil para análisis).
tol	0.0001	Tolerancia para el algoritmo iterativo en lsqr y eigen.

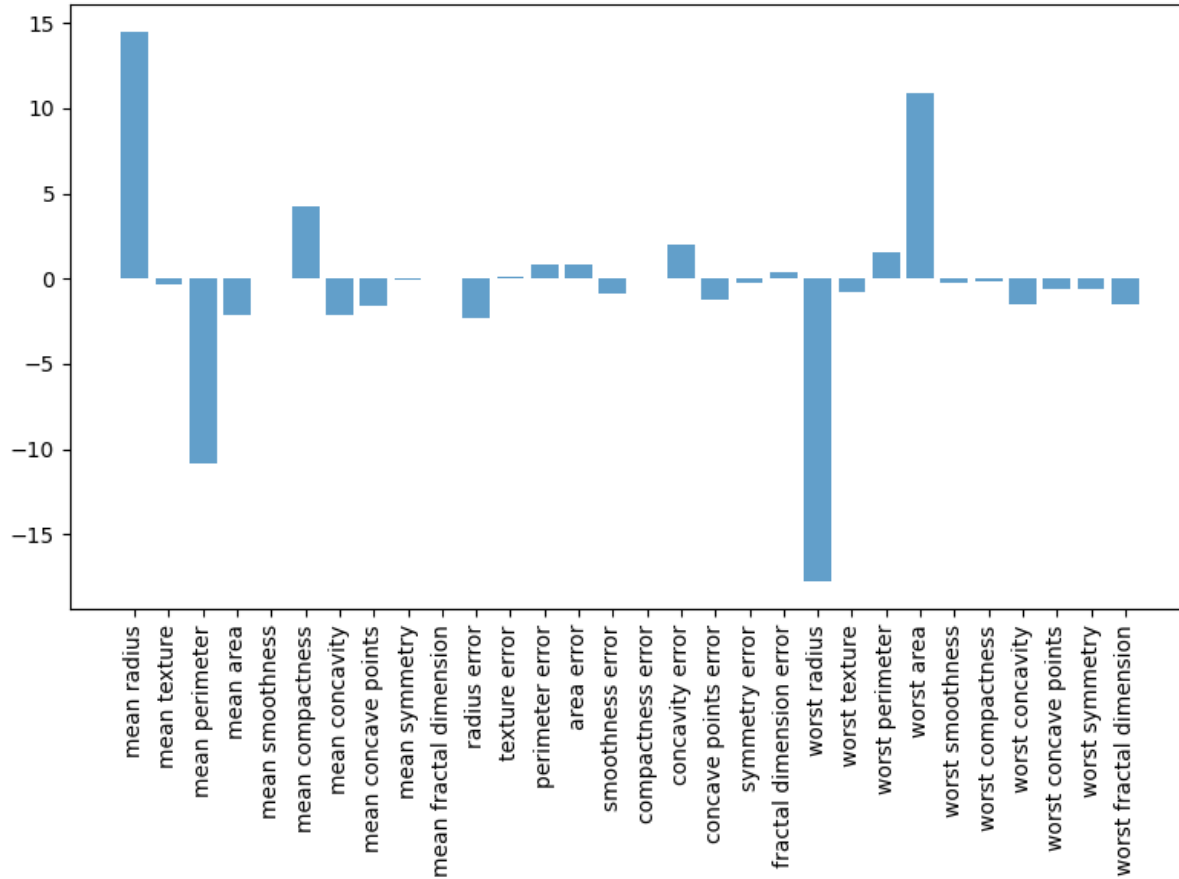
Ejemplo: breast-cancer dataset

```
# Print explained variance ratio
print("\nExplained variance ratio:",
lda.explained_variance_ratio_)
print("Cumulative explained variance:",
np.cumsum(lda.explained_variance_ratio_))
```

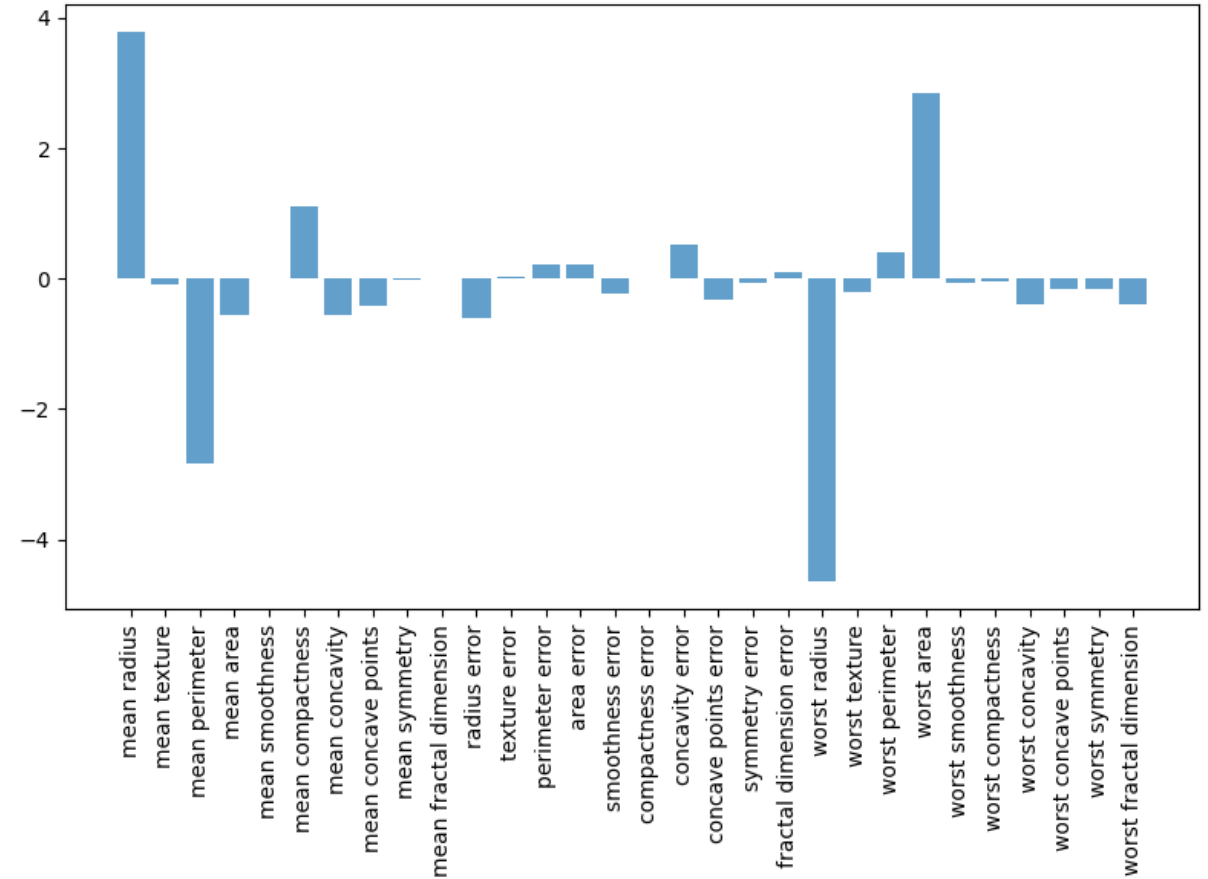
```
Explained variance ratio: [1.]
Cumulative explained variance: [1.]
```

El primer componente captura el 100% de la varianza.
Ambos componentes juntos capturan el 100% de la
información que separa las clases en los datos originales.

Ejemplo: breast-cancer dataset



Autovalores lda.coef_



Autovectores lda.scalings_

Solver: algoritmo de cálculo

'svd' (Singular Value Decomposition - Descomposición en Valores Singulares):

- Opción por defecto y recomendada para datos con un gran número de características (alta dimensionalidad).
- Es el método más estable numéricamente, ya que no calcula la matriz de covarianza directamente.
- Ideal cuando las características superan las muestras.

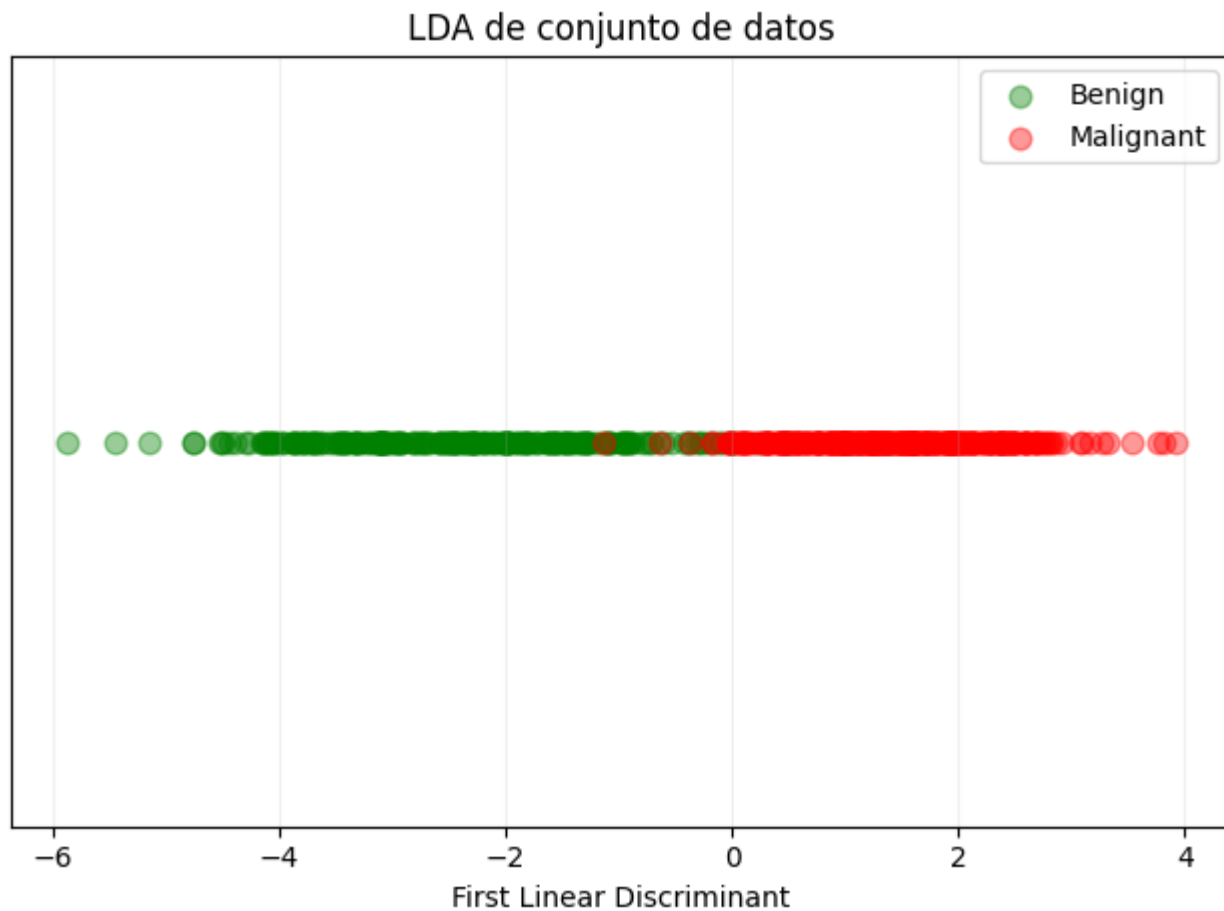
'lsqr' (Least Squares Solution - Solución por Mínimos Cuadrados):

- Es un solucionador iterativo.
- Rápido y eficiente en memoria, porque usa la solución de mínimos cuadrados.
- Ideal para problemas de clasificación de gran escala (muchas muestras y características).

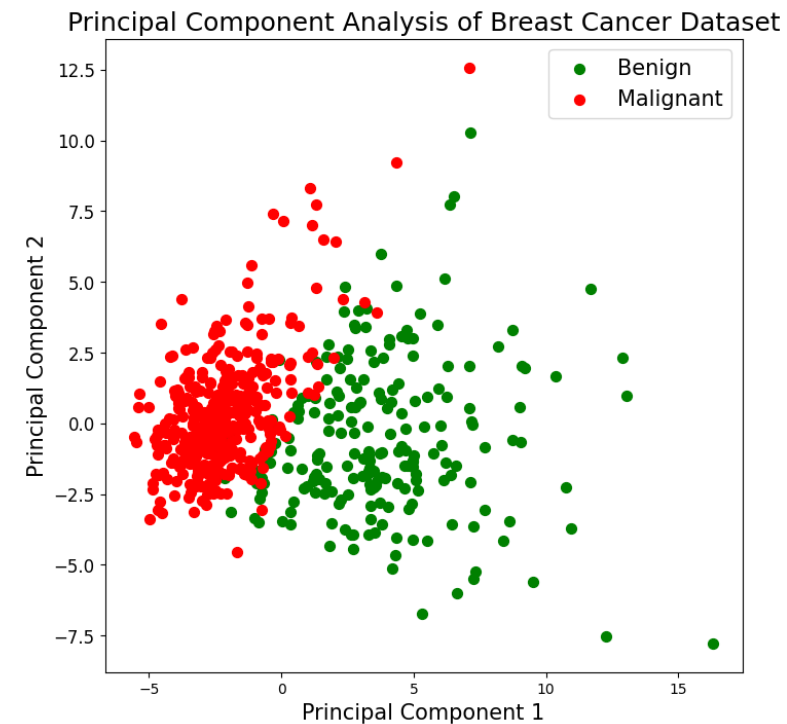
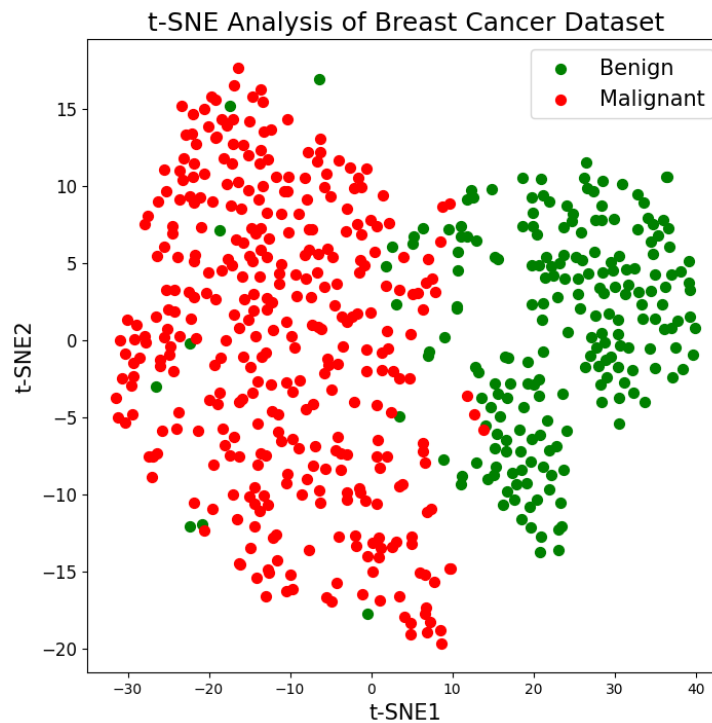
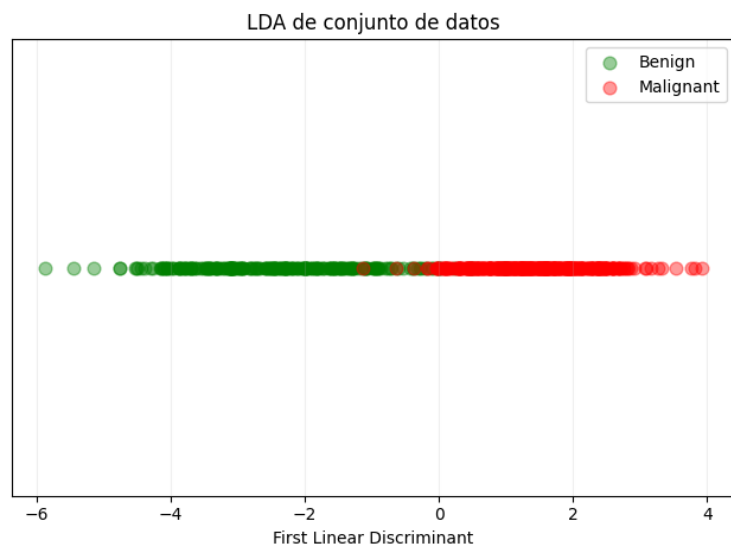
'eigen' (Eigenvalue Decomposition - Descomposición de Valores Propios):

- Basado en la descomposición de la matriz de covarianza, lo que permite interpretar los valores propios directamente.
- Útil cuando se desea un análisis detallado de los componentes, pero se recomienda menos en alta dimensionalidad.

Ejemplo: breast-cancer dataset



Ejemplo: breast-cancer dataset



LDA vs T-SNE vs PCA

Ejemplo: breast-cancer dataset

```
#LDA
lda = LinearDiscriminantAnalysis()
X_train_lda = lda.fit(X_train, y_train)
y_pred_lda = lda.predict(X_test)

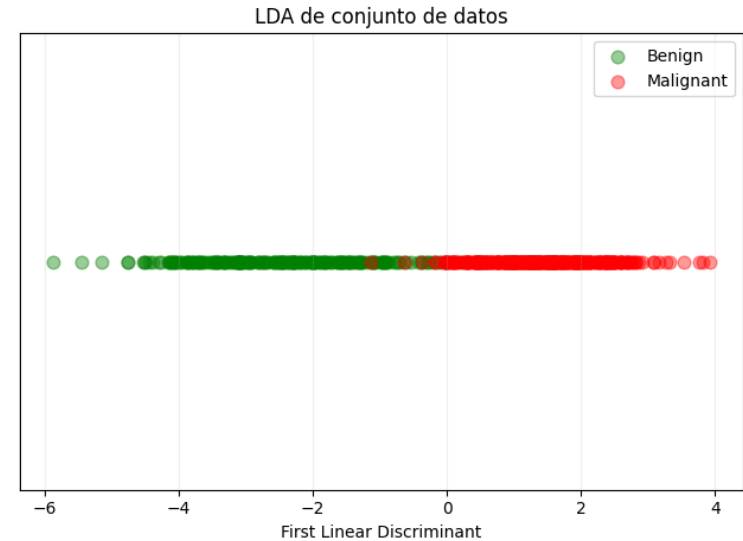
print("LDA Accuracy:", accuracy_score(y_test, y_pred_lda))
print("Classification Report:\n",
      classification_report(y_test, y_pred_lda))
```

```
LDA Accuracy: 0.935672514619883
Classification Report:
              precision    recall  f1-score   support

   Benign           0.98      0.84      0.91         64
  Malignant          0.91      0.99      0.95        107

   accuracy                   0.94         171
  macro avg           0.95      0.92      0.93         171
weighted avg           0.94      0.94      0.93         171
```

Entrenar el modelo => Clasificación



Ejemplo: iris-extended dataset

Estandarizar los datos

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(iris_numdata, features, test_size=0.2,
random_state=42, stratify=features)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Aplicar LDA

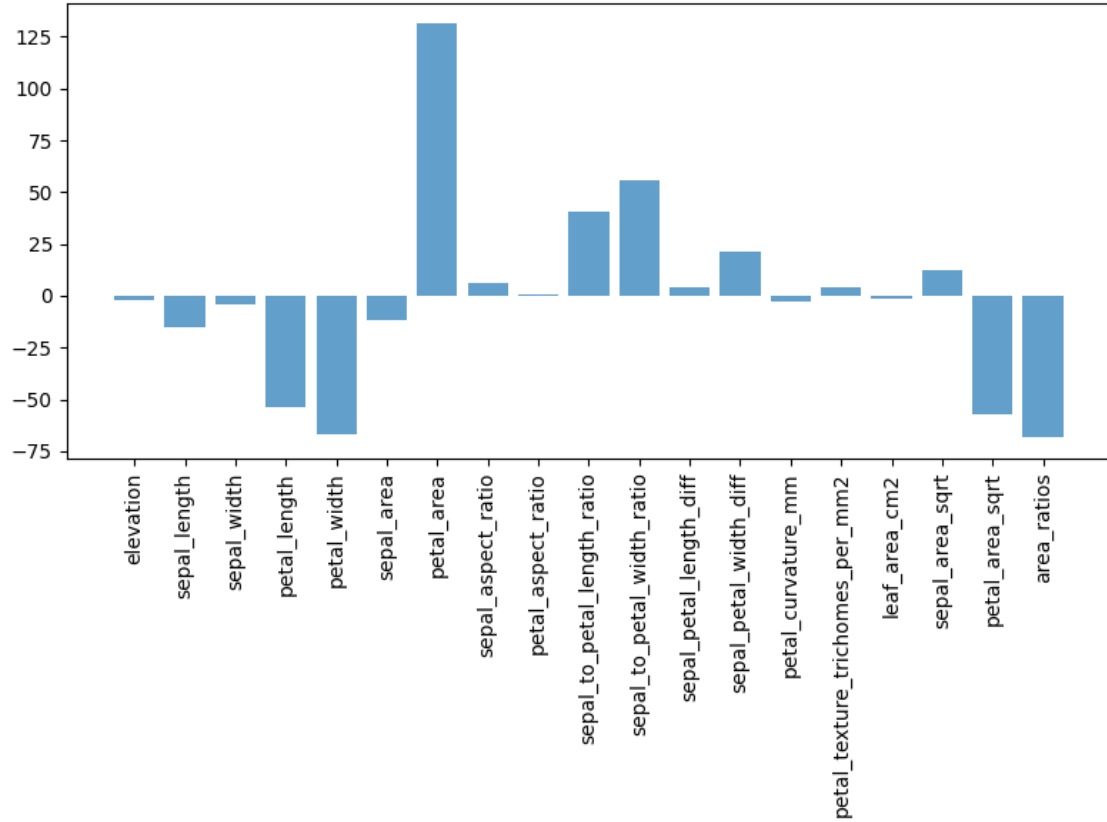
```
lda = LinearDiscriminantAnalysis()
# Entrenar LDA y transformar datos
X_train_lda = lda.fit_transform(X_train_scaled, y_train)
X_test_lda = lda.transform(X_test_scaled)
# Ver hiperparámetros usados en el modelo
print("🚀 Hiperparámetros del modelo:")
print(lda.get_params())
```

```
# Print explained variance ratio
print("\nExplained variance ratio:",
lda.explained_variance_ratio_)
print("Cumulative explained variance:",
np.cumsum(lda.explained_variance_ratio_))
```

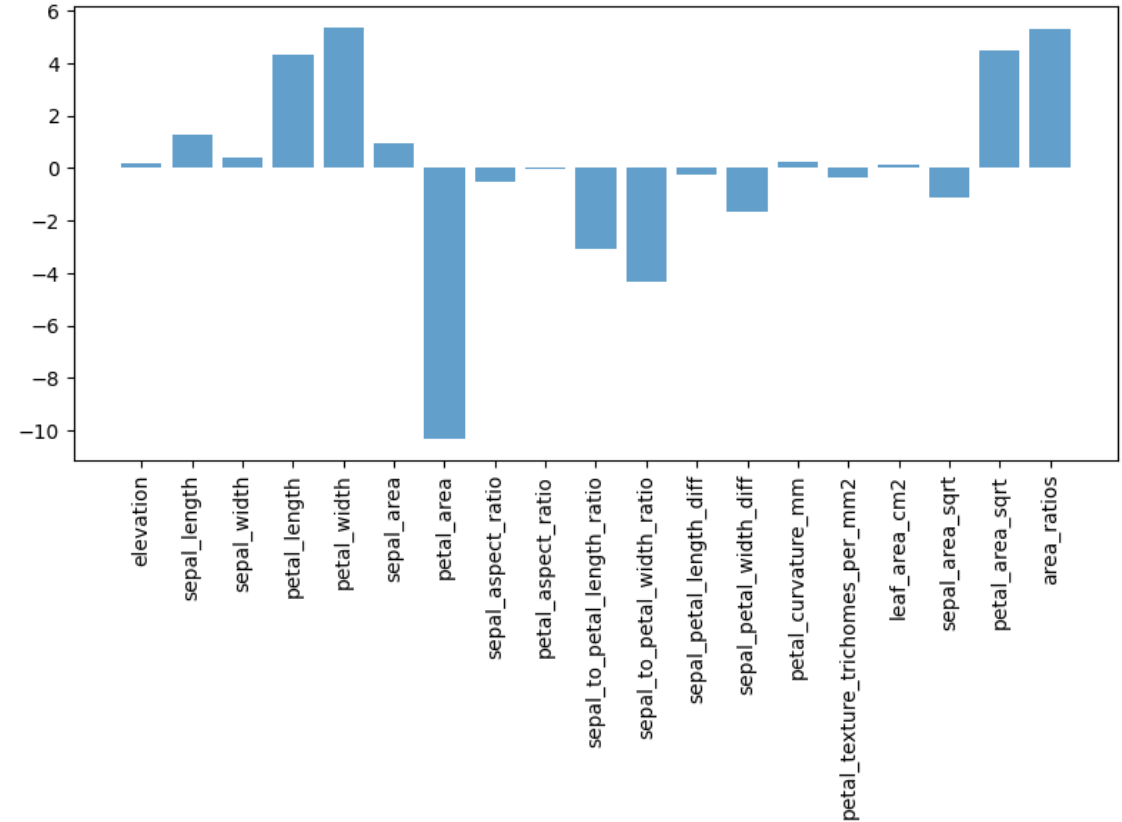
```
Explained variance ratio: [0.96512621 0.03487379]
Cumulative explained variance: [0.96512621 1.          ]
```

El primer componente captura el 96% de la varianza.
Ambos componentes juntos capturan el 100% de la información que separa las clases en los datos originales.

Ejemplo: iris-extended dataset

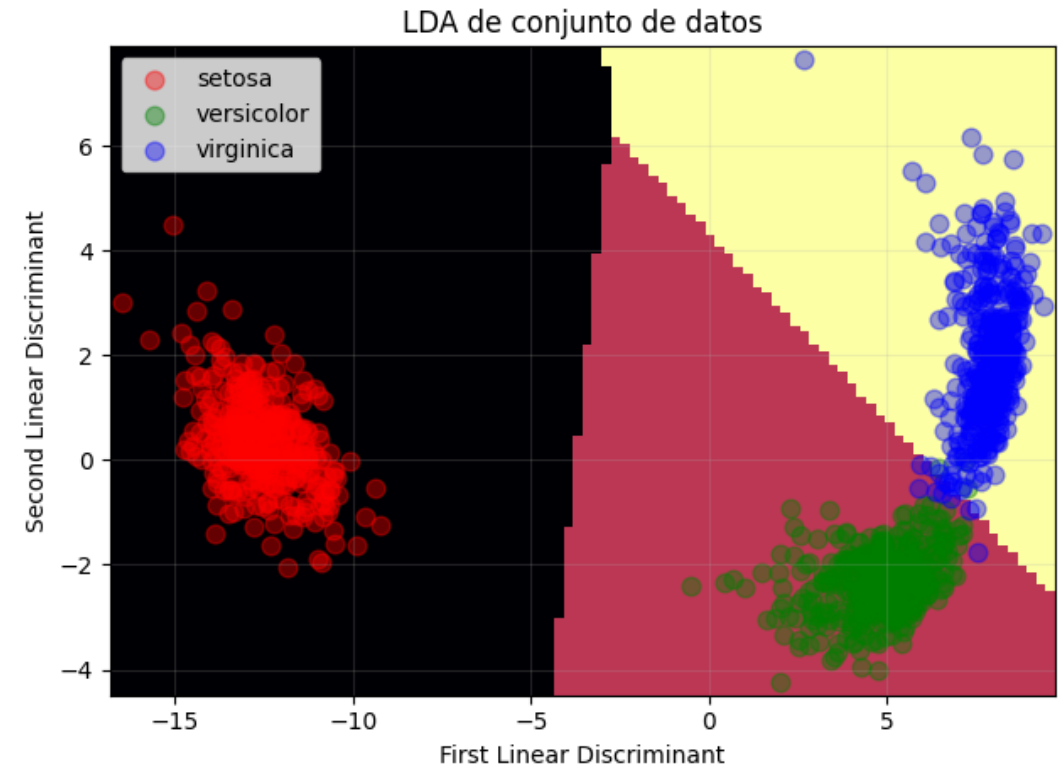
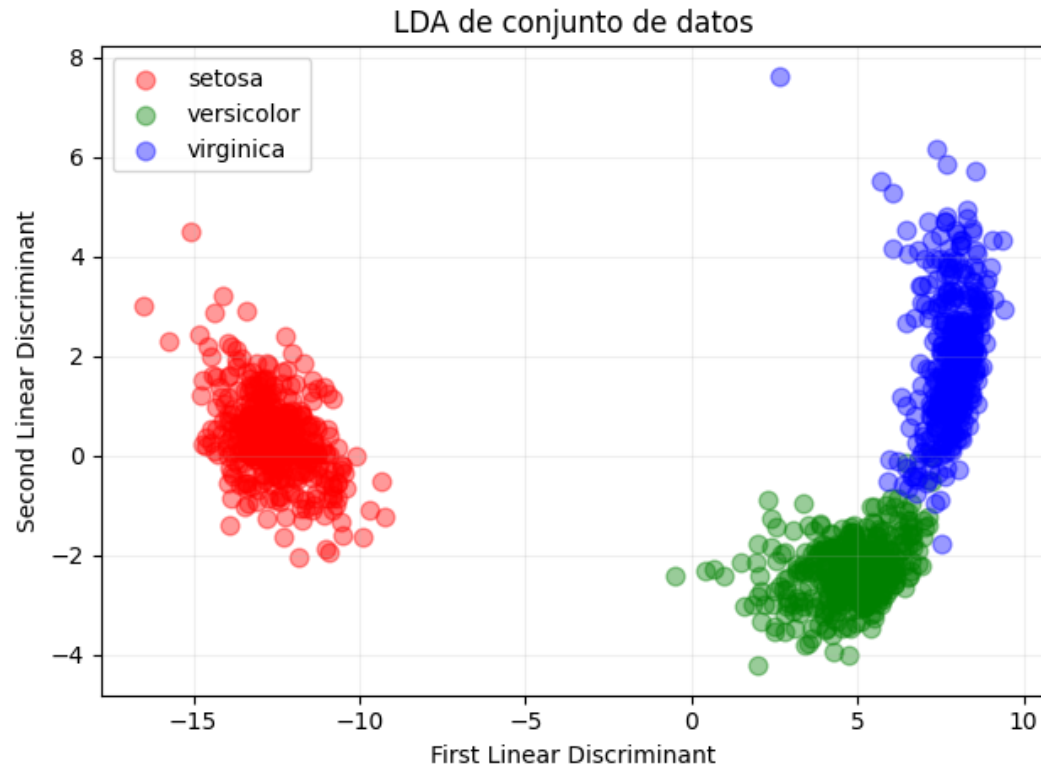


Autovalores lda.coef_



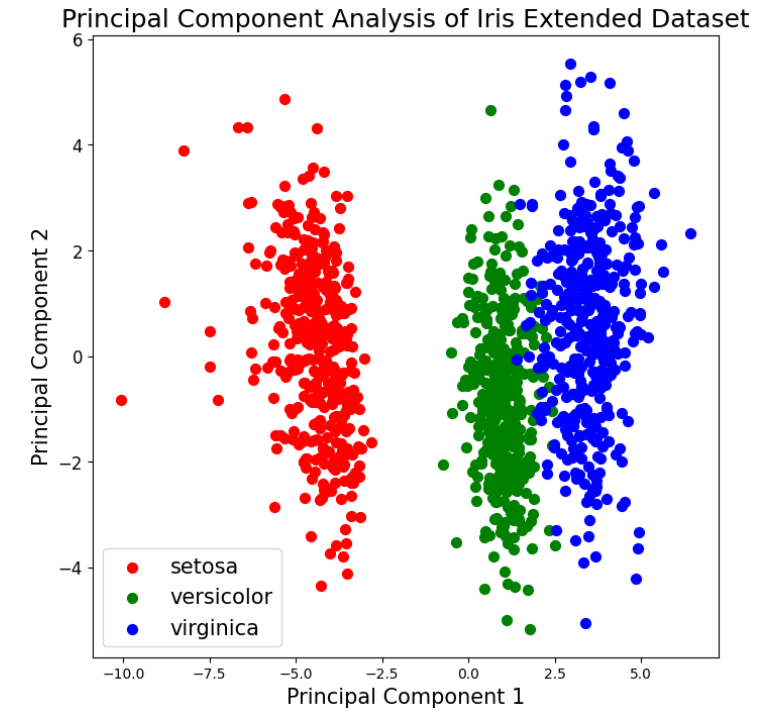
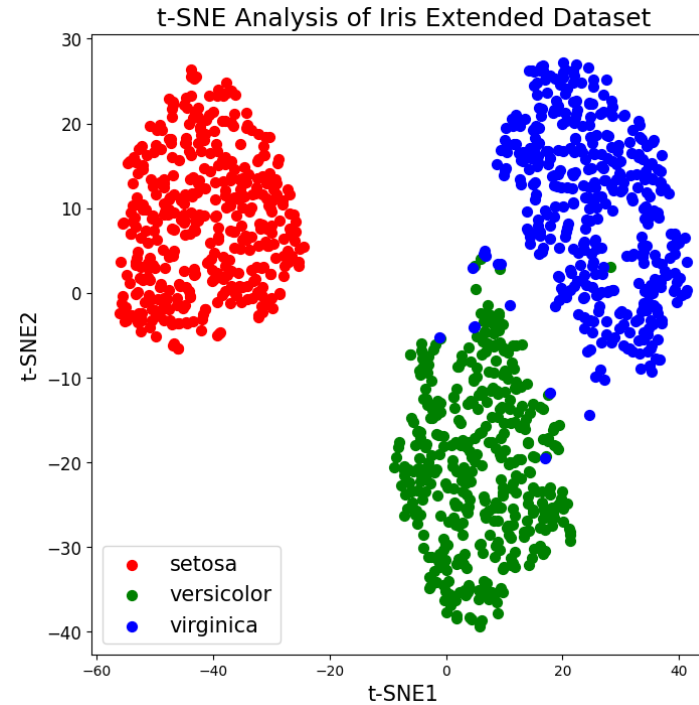
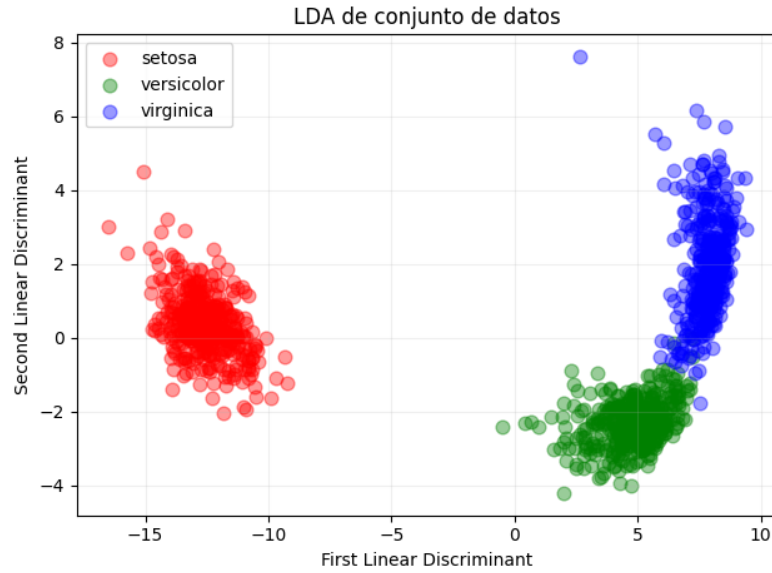
Autovectores lda.scalings_

Ejemplo: iris-extended dataset



Decission boundaries

Ejemplo: iris-extended dataset



LDA vs T-SNE vs PCA

Ejemplo: iris-extended dataset

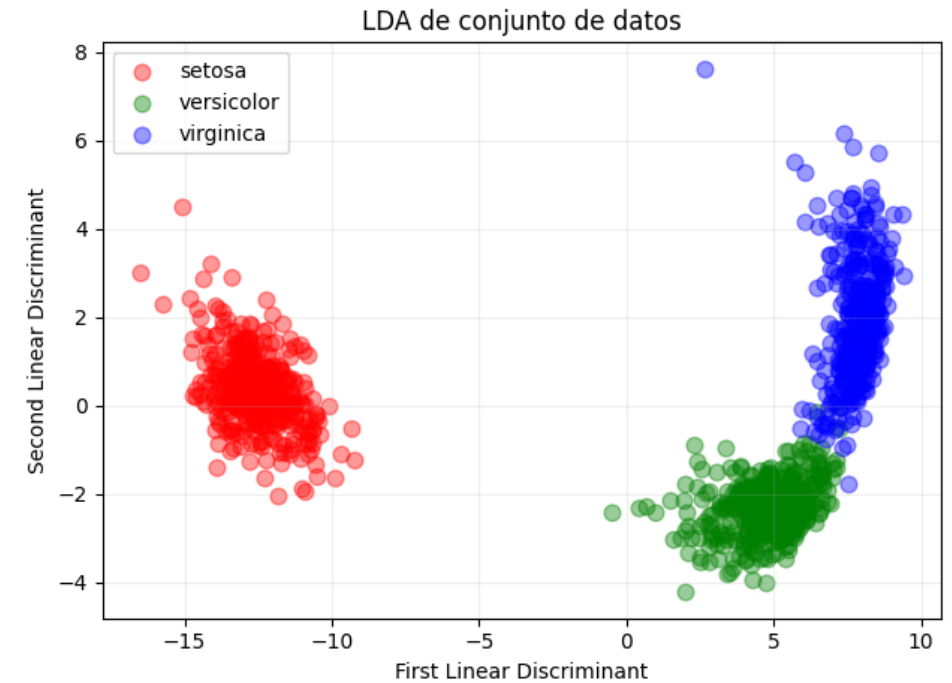
```
#LDA
lda = LinearDiscriminantAnalysis()
X_train_lda = lda.fit(X_train, y_train)
y_pred_lda = lda.predict(X_test)

print("LDA Accuracy:", accuracy_score(y_test, y_pred_lda))
print("Classification Report:\n",
      classification_report(y_test, y_pred_lda))
```

```
LDA Accuracy: 0.9861111111111112
Classification Report:
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	120
versicolor	0.98	0.98	0.98	120
virginica	0.98	0.97	0.98	120
accuracy			0.99	360
macro avg	0.99	0.99	0.99	360
weighted avg	0.99	0.99	0.99	360

Entrenar el modelo => Clasificación

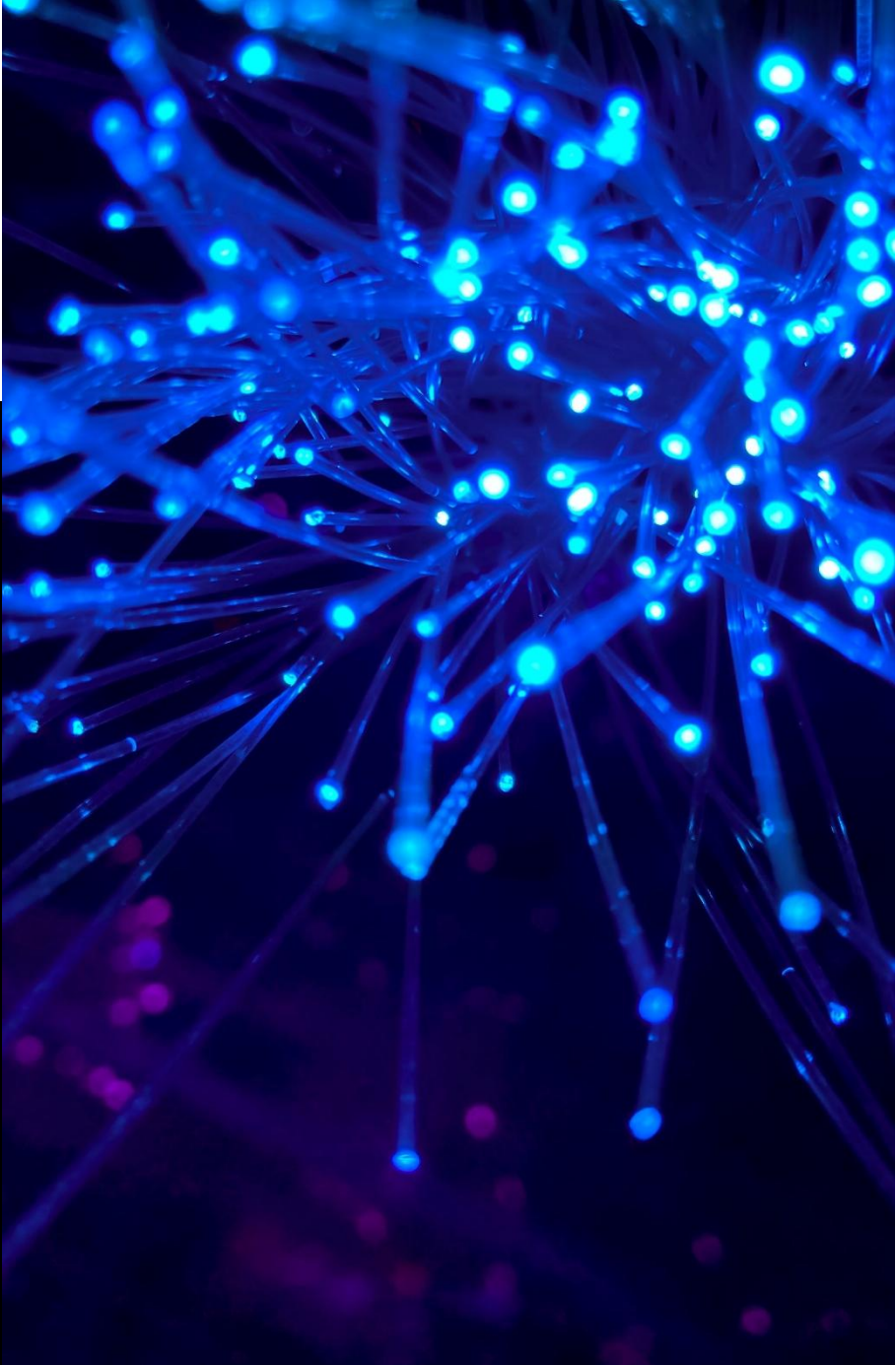


Resumen Comparativo

Característica 	PCA	LDA	t-SNE
Tipo	No supervisado	Supervisado	No supervisado
Linealidad	Lineal	Lineal	No lineal
Objetivo	Maximizar Varianza	Separar Clases	Visualización de Clusters
Velocidad	Rápido	Rápido	Lento (costoso)
Uso Principal	Preprocesamiento/PCA	Preprocesamiento/Clasificación	Exploración Visual

En resumen:

- ¿Datos sin etiquetas? → **PCA**.
- ¿Datos etiquetados para clasificar? → **LDA**.
- ¿Visualizar clusters complejos? → **t-SNE**. 



Clustering

Clustering o Agrupación

Es una técnica de aprendizaje automático **no supervisado**, que agrupa datos sin etiquetar en clústeres basándose en similitudes, como la densidad, la proximidad o características específicas.

Su objetivo principal es encontrar estructuras de segmentación oculta.

Ejemplos de uso: segmentación de clientes, reconocimiento de patrones, detección de anomalías, análisis de imágenes, ...

Los clústeres de calidad contienen al menos dos propiedades:

- Todos los puntos de datos de un clúster deben ser similares.
- Los clústeres deben ser distintos entre sí.

- Métodos de partición: **K-Means**
- Métodos basados en la densidad: **DBSCAN**

- Métodos de partición: **K-Means**
- Métodos basados en la densidad: **DBSCAN**

El algoritmo K-Means tiene como objetivo dividir un conjunto de datos en k grupos distintos y no superpuestos, basados en centroides.

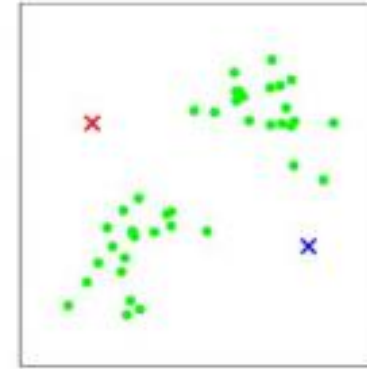
El fin es minimizar la varianza intra-clúster y maximizar la varianza inter-clúster.

1. Inicialización
2. Asignación
3. Actualización
4. Iteración -> Convergencia

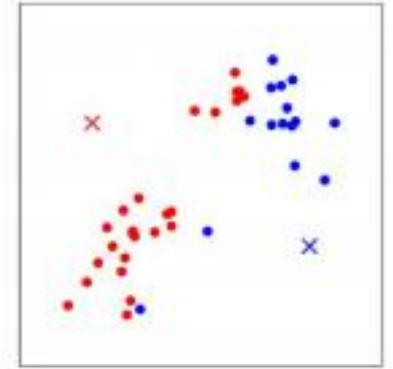
Centroide: es el punto central que representa el promedio de todos los puntos de datos pertenecientes a un grupo o clúster específico. Actúa como centro de masa o geométrico.



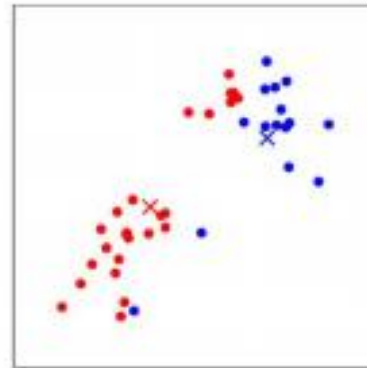
(a)



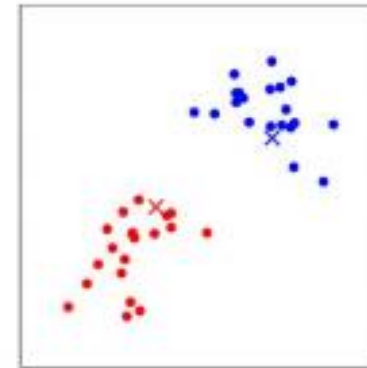
(b)



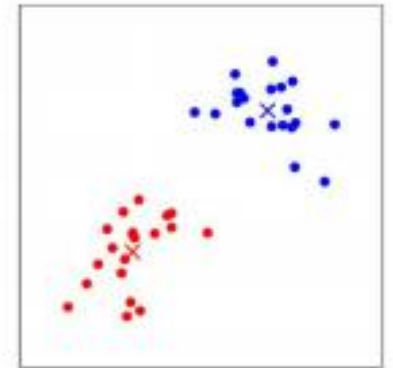
(c)



(d)



(e)



(f)

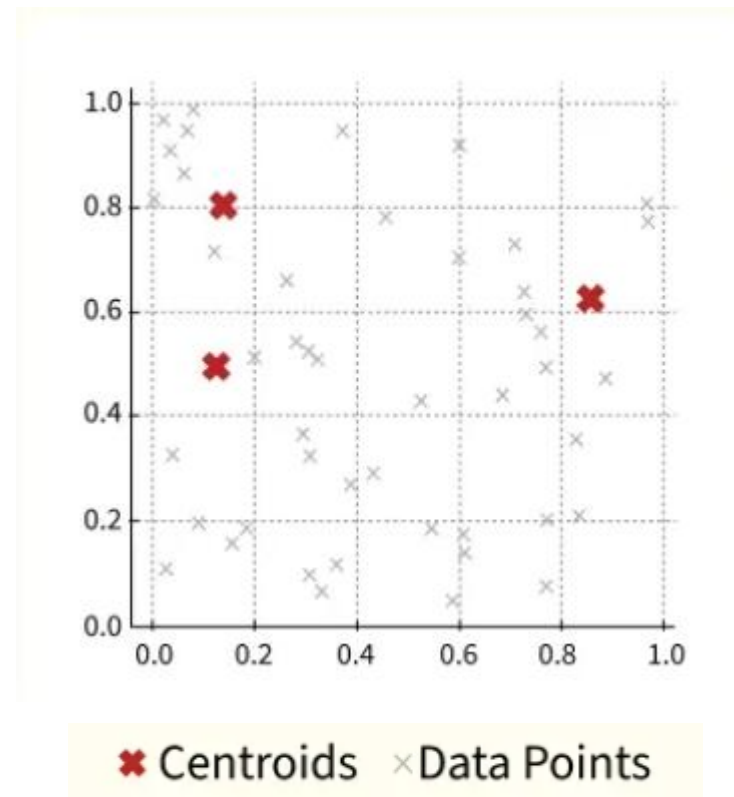
$$M = \arg \min \sum_{i=1}^k \frac{1}{|C_K|} \cdot \sum_{x \in C_K} \|x_i - \mu_i\|^2$$

El algoritmo busca dividir un conjunto de datos $X = [x_1, x_2, \dots, x_n]$ en K conjuntos $C = [C_1, C_2, \dots, C_K]$, para minimizar la suma de cuadrados dentro del clúster.

μ_i es el centroide promedio de los puntos en C_K .

1. Inicialización

Seleccionar K centroides iniciales.

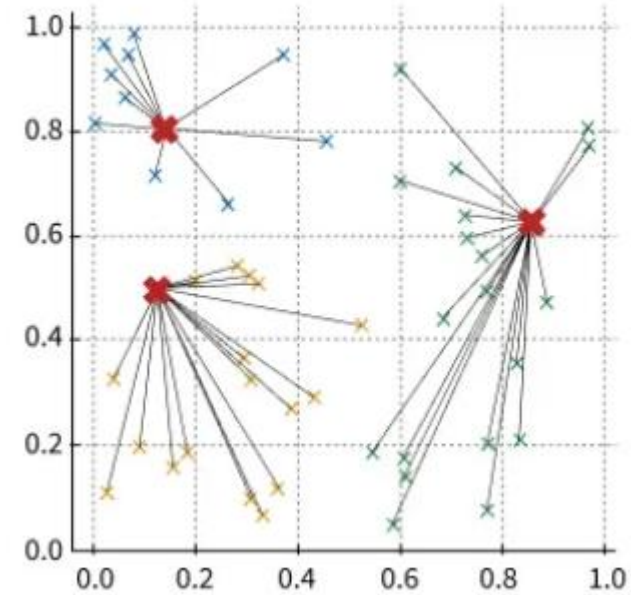


2. Asignación

Asignar los puntos de datos al clúster más cercano.

$$C_K^t = \left\{ x_i : \|x_i - \mu_i^t\|^2 \leq \|x_i - \mu_j^t\|^2 \forall j, 1 \leq j \leq k \right\}$$

Para cada punto x_i , se calcula la distancia Euclidiana a cada centroide y se asigna x_i al clúster cuyo centroide está más cercano.



3. Actualización

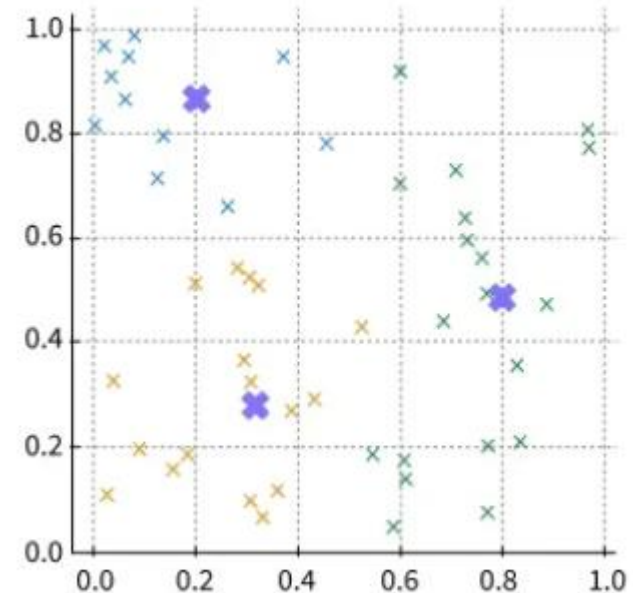
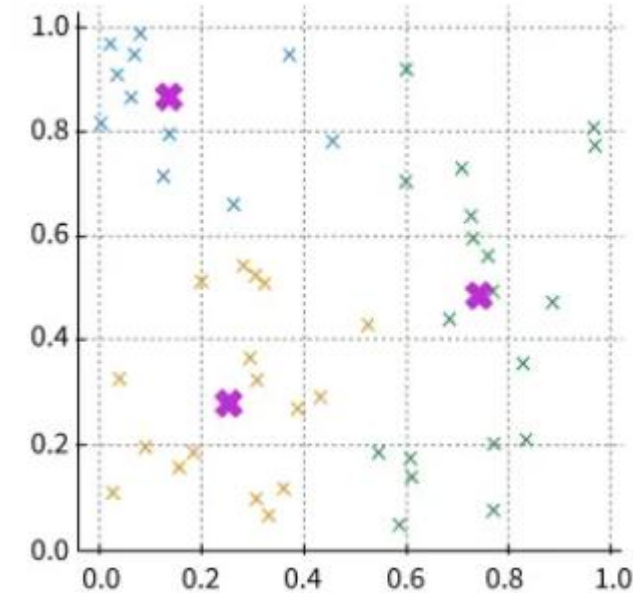
Actualización de los centroides para cada clúster.

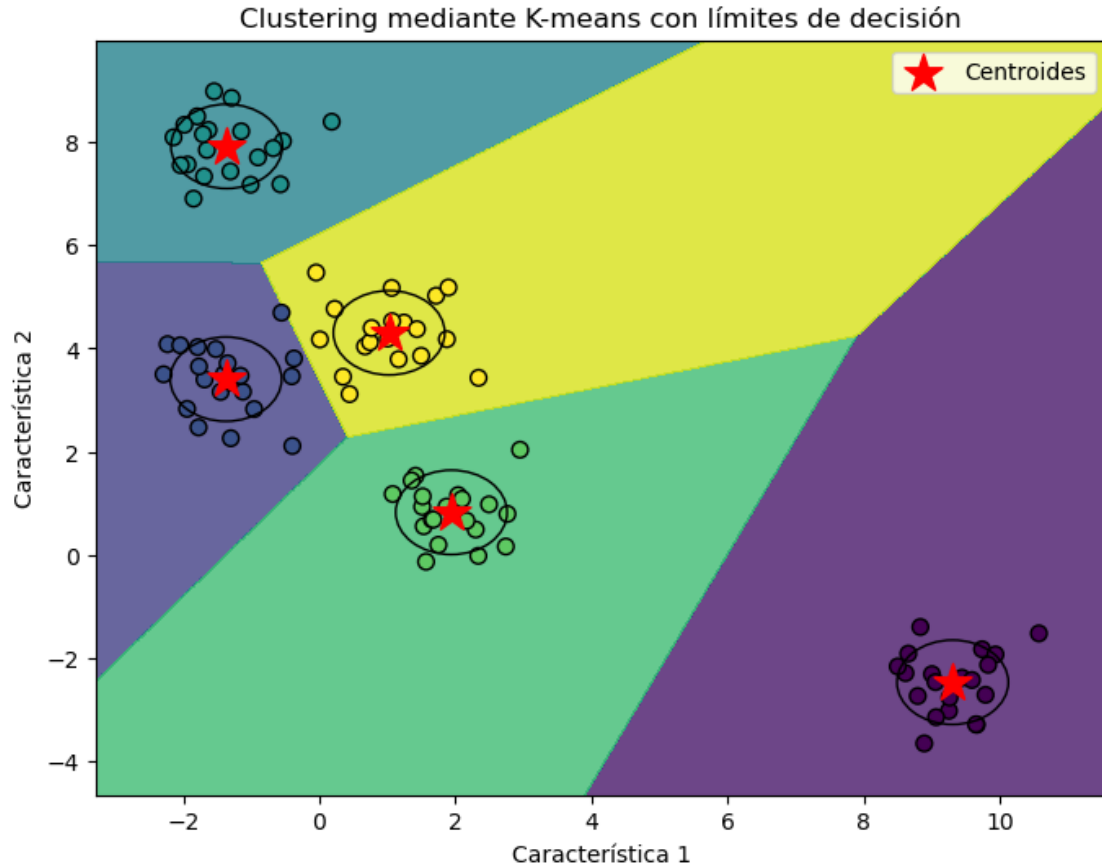
$$\mu_k^{t+1} = \frac{1}{|C_k^t|} \cdot \sum_{x \in C_K} x_i$$

Como el promedio de todos los puntos asignados a ese clúster.

4. Iteración (pasos 2,3) hasta Convergencia

El algoritmo convergerá cuando los centroides no cambien o cambien por debajo de un cierto umbral predefinido.





```
KMeans(n_clústers, init, n_init, max_iter, tol, algorithm)
```

n_clústers: Número de clústers que se desean encontrar.

init: Método de inicialización de los centroides. Puede ser k-means++, random, o un array de centroides iniciales.

n_init: Número de veces que se ejecutará el algoritmo K-means con diferentes centroides iniciales. El resultado final será el modelo con la inercia más baja.

max_iter: Número máximo de iteraciones en cada ejecución del algoritmo K-means.

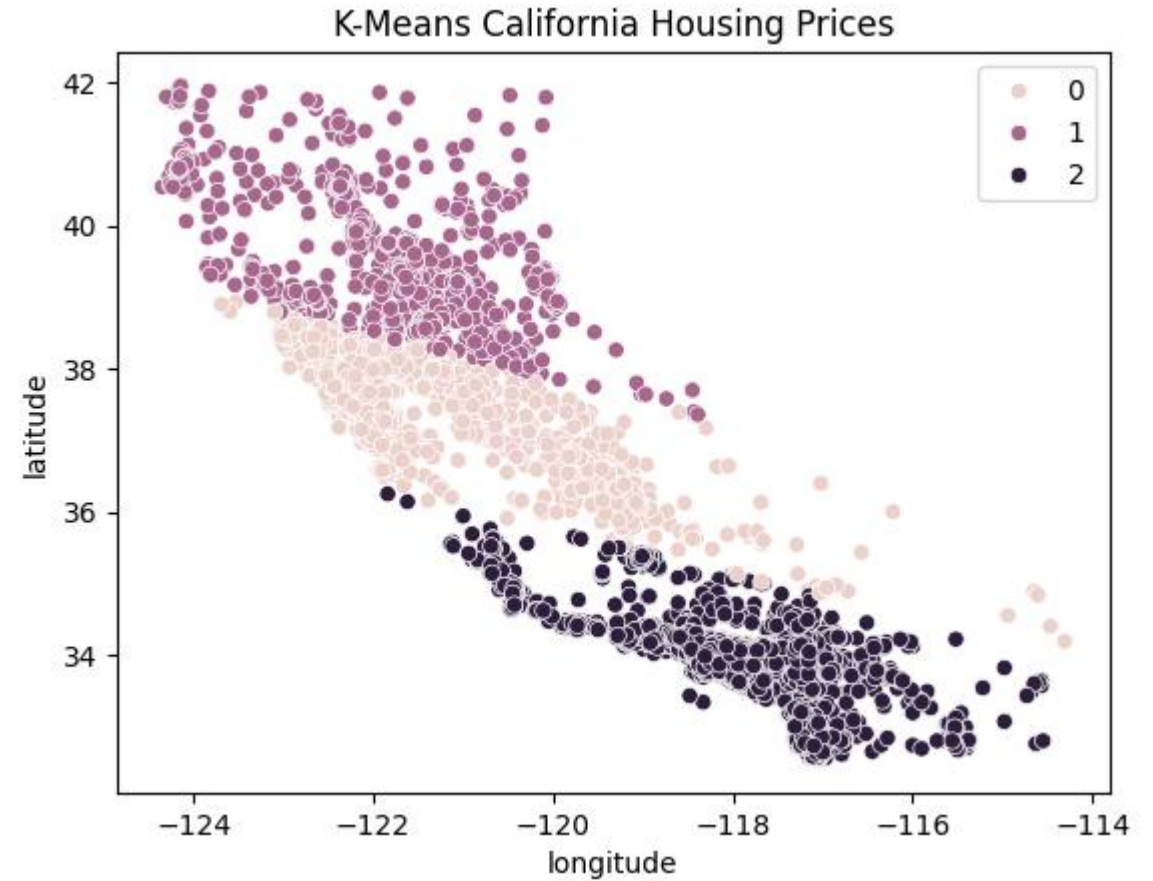
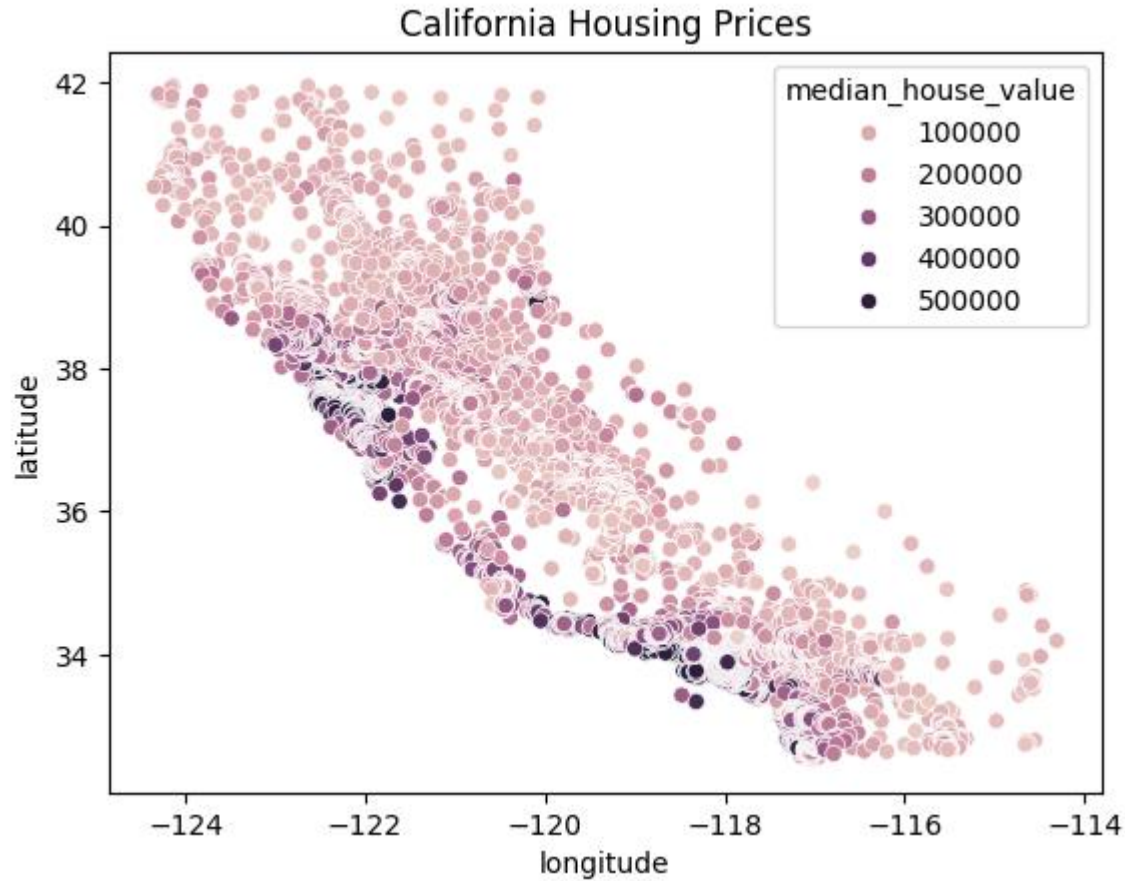
tol: Tolerancia para declarar la convergencia del algoritmo, es decir, el criterio de parada. Si la diferencia entre los centroides de dos iteraciones consecutivas es menor que tol, se considera que el algoritmo ha convergido.

algorithm: Algoritmo para calcular el K-means. Puede ser auto, full, elkan, entre otros.

Limitaciones del algoritmo K-Means:

- **Elegir el número correcto de clústeres:** Uno de los mayores desafíos es decidir cuántos clústeres utilizar.
- **Sensible a los centroides iniciales:** Los clústeres finales pueden variar dependiendo de la ubicación aleatoria inicial de los centroides.
- **Agrupaciones no esféricas:** K-Means asume que las agrupaciones son esféricas y de igual tamaño. Esto puede ser un problema cuando las agrupaciones reales en los datos tienen formas o densidades diferentes.
- **Valores atípicos:** K-Means es sensible a los valores atípicos, que pueden distorsionar el centroide y, en última instancia, los clústeres.

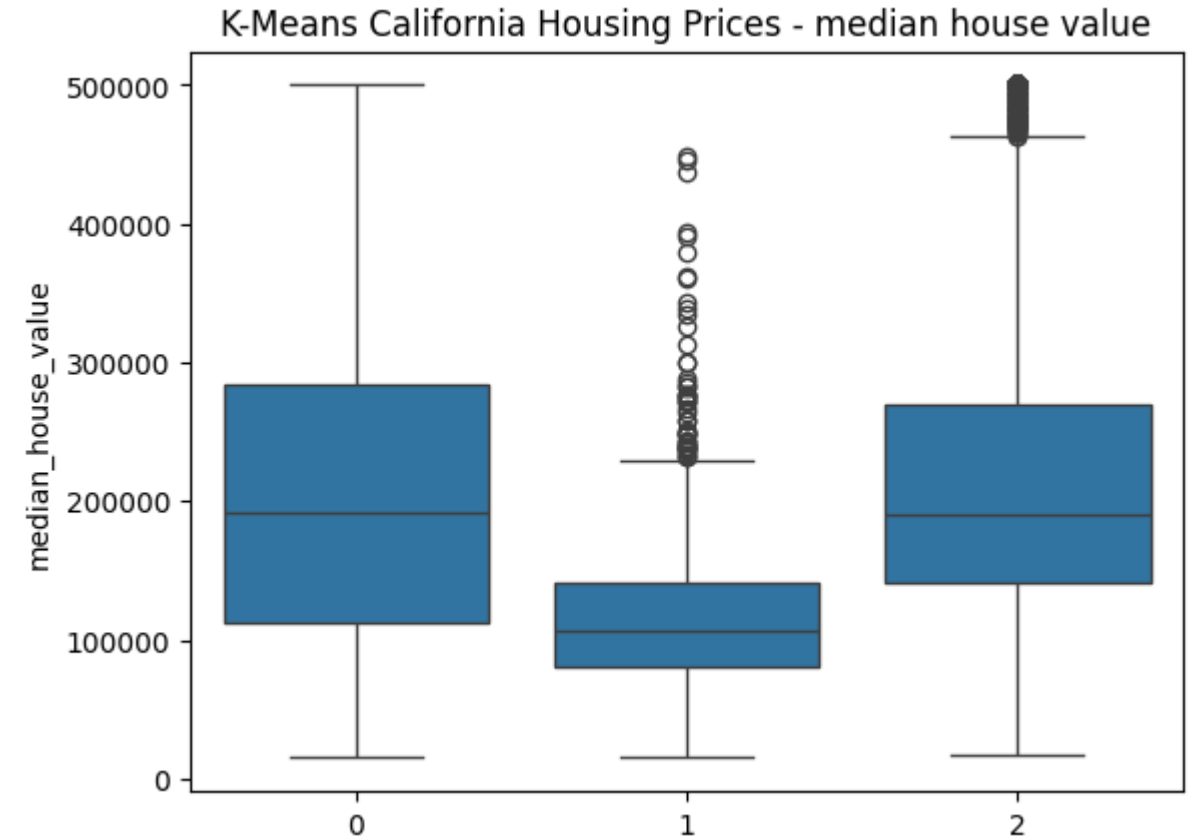
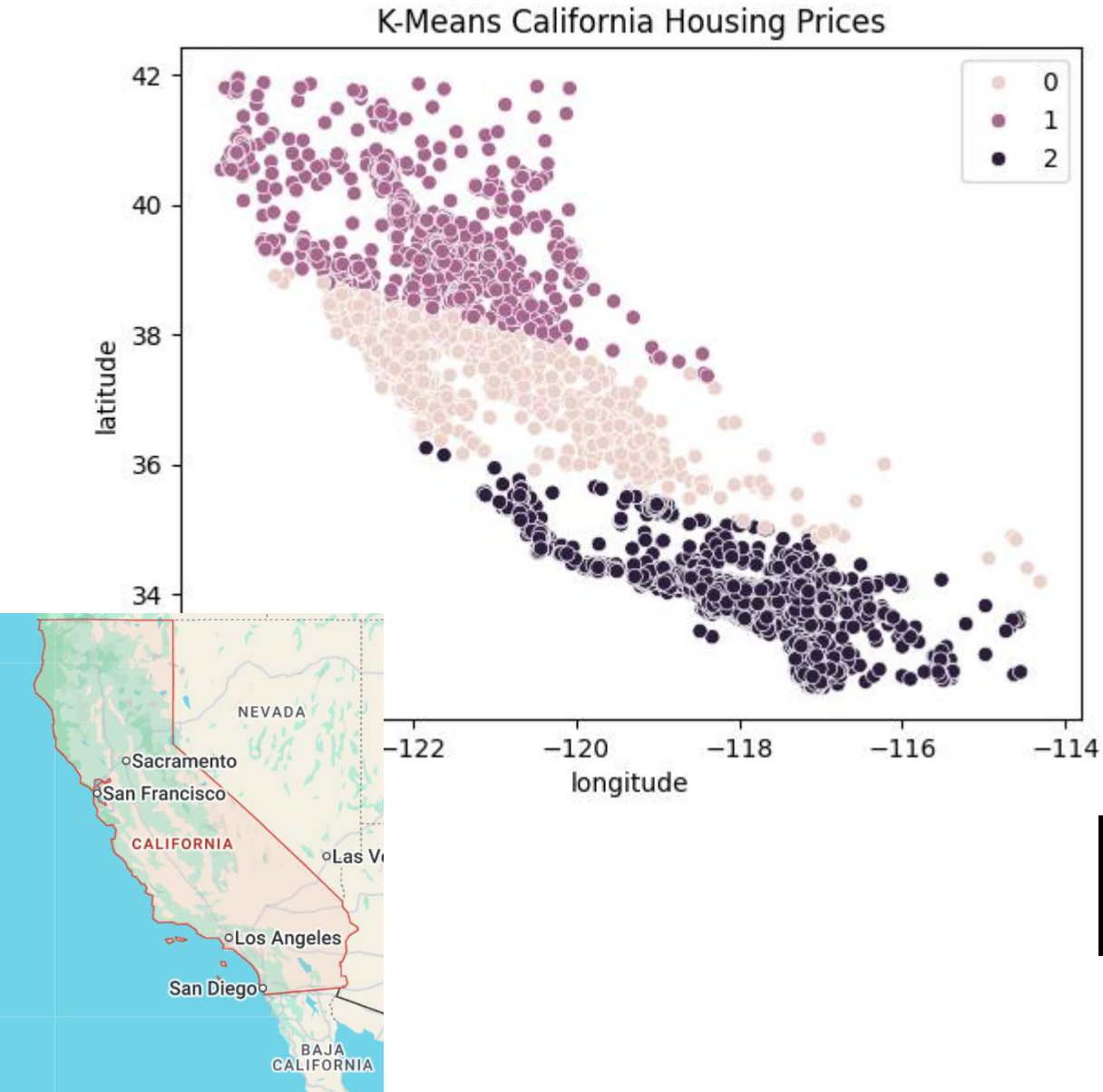
Ejemplo: California housing prices dataset



```
kmeans = KMeans(n_clusters = 3, random_state = 0, n_init='auto')
```

los datos ahora están claramente divididos en tres grupos distintos (norte de California, centro de California y sur de California)

Ejemplo: California housing prices dataset



los clústeres del norte y del sur presentan distribuciones similares (clústeres 0 y 2), con precios superiores a los del clúster central (clúster 1).

Elegir el número óptimo de clústeres

Debemos probar diferentes rangos de valores y decidir cuál es el mejor valor de k.

Normalmente, utilizamos el método del codo para determinar el número óptimo de clústeres, de manera que no se produzca sobreajuste con demasiados clústeres ni subajuste con muy pocos.

Se representa el análisis de inercia o silueta frente al número de clústeres.

La inercia cuantifica la dispersión de los puntos dentro de cada clúster, calculando la suma de las distancias al cuadrado de cada punto al centroide de su clúster correspondiente

$$I = \sum_{i=1}^k \sum_{x \in C_k} \|x - \mu_i\|^2$$

k: número total de clústeres

C_k : conjunto de puntos asignados al clúster k

x: punto de datos individual dentro del clúster

μ_i : centroide del clúster C_k

$\|x - \mu_i\|^2$: distancia euclidiana al cuadrado entre el punto x y el centroide

La silueta proporciona información detallada sobre cómo se agrupan los puntos dentro de cada clúster y cómo se separan entre sí los clústers adyacentes.

$$S(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

b(i): separación

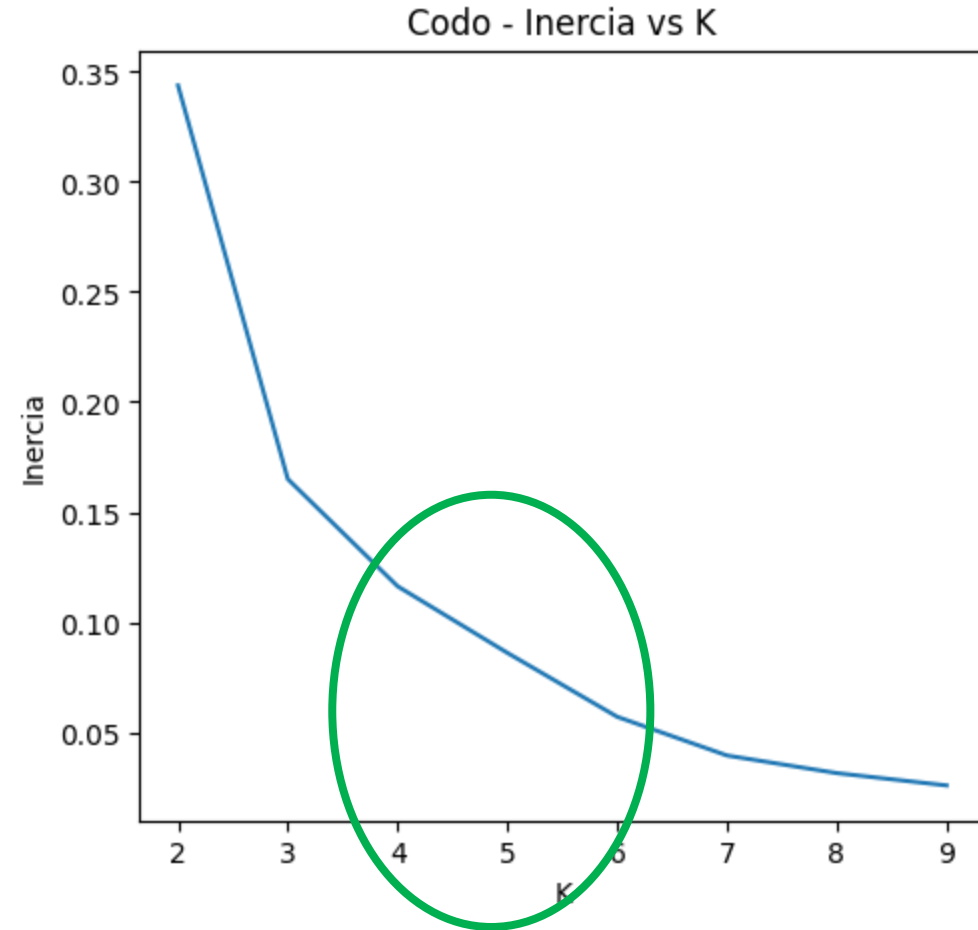
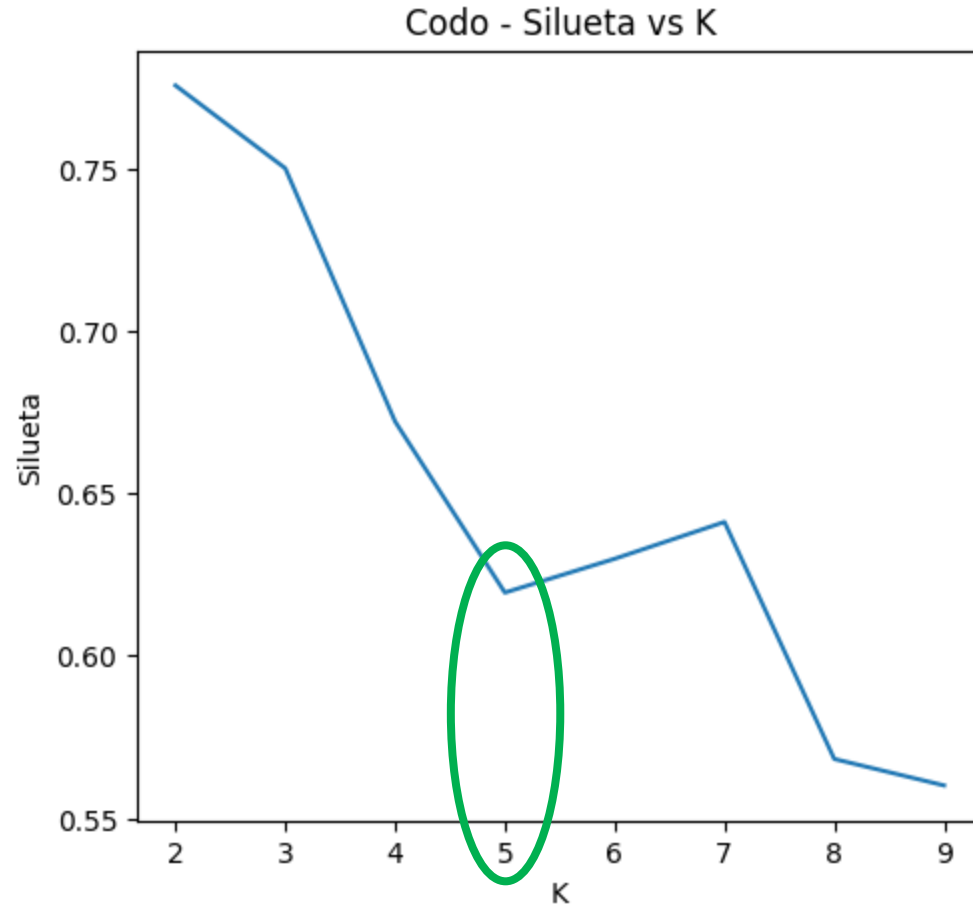
a(i): cohesión

$\max\{a(i), b(i)\}$: máximo par normalizar valores entre -1 y 1

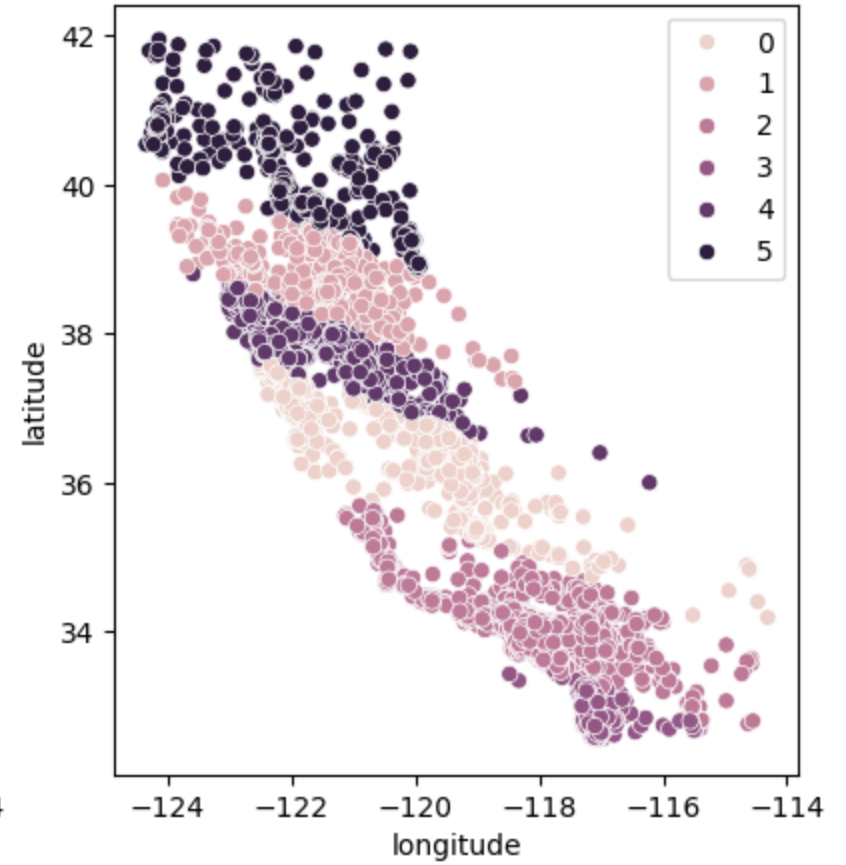
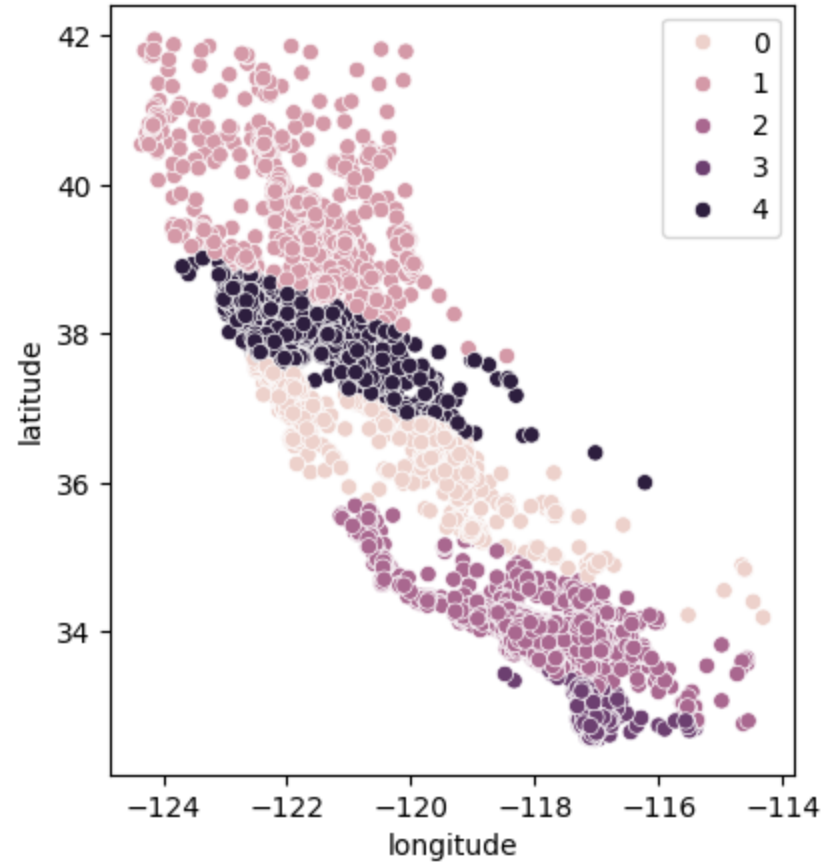
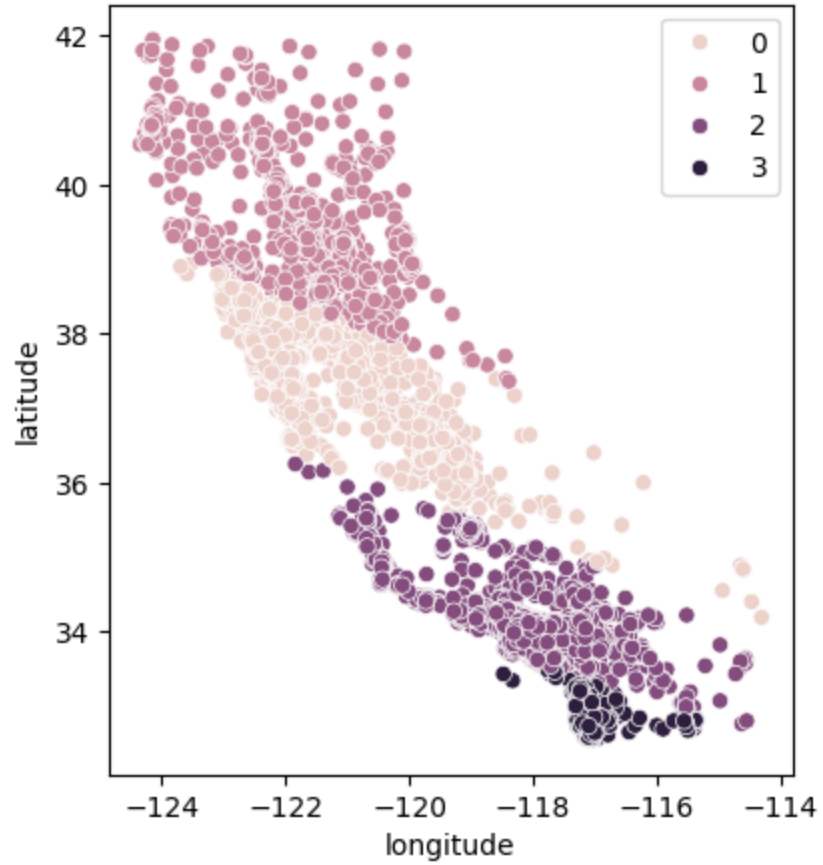
El índice de Davies-Bouldin (*Davies-Bouldin Index*) es una métrica que proporciona una medida de la "separación" entre los clústers, donde valores más bajos indican clústers mejor definidos y más separados entre sí.

$$DB = \frac{1}{k} \cdot \sum_{i=1}^N \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)} \right)$$

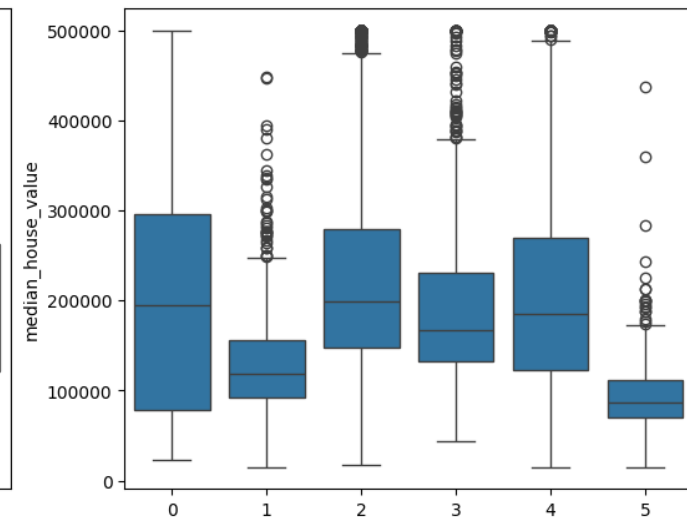
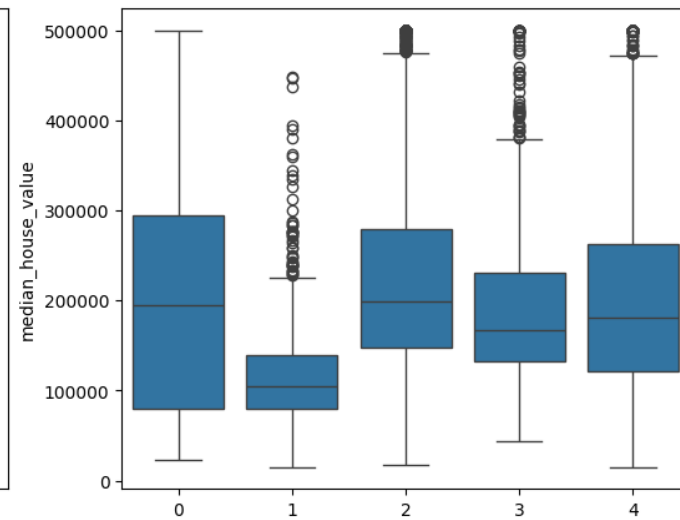
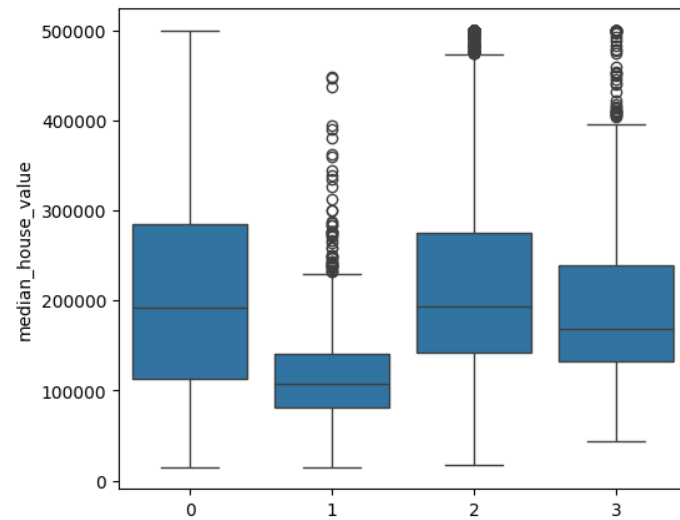
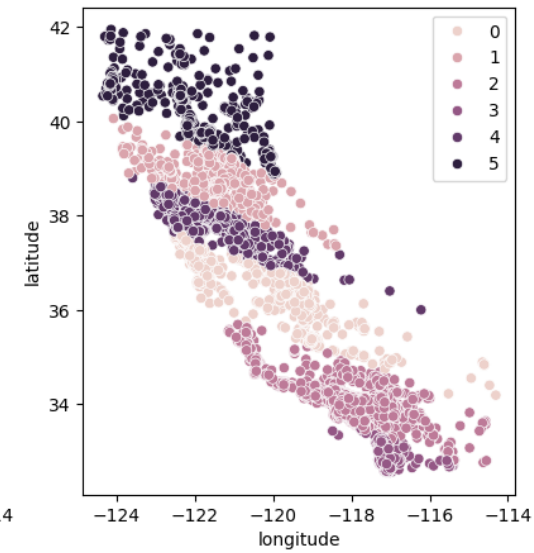
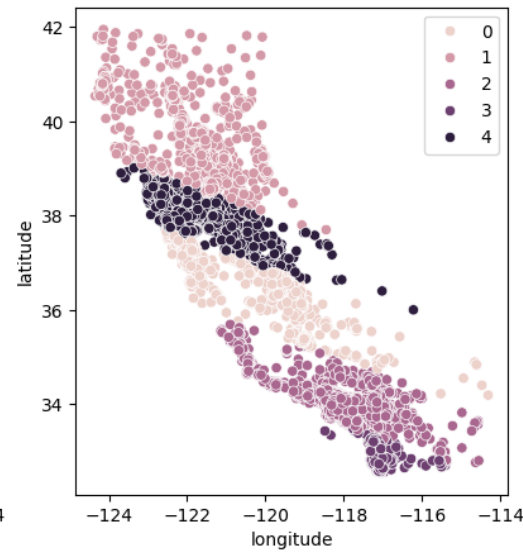
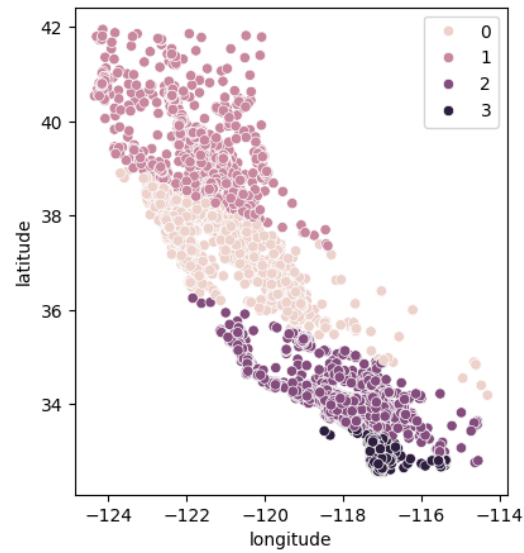
Ejemplo: California housing prices dataset



Ejemplo: California housing prices dataset



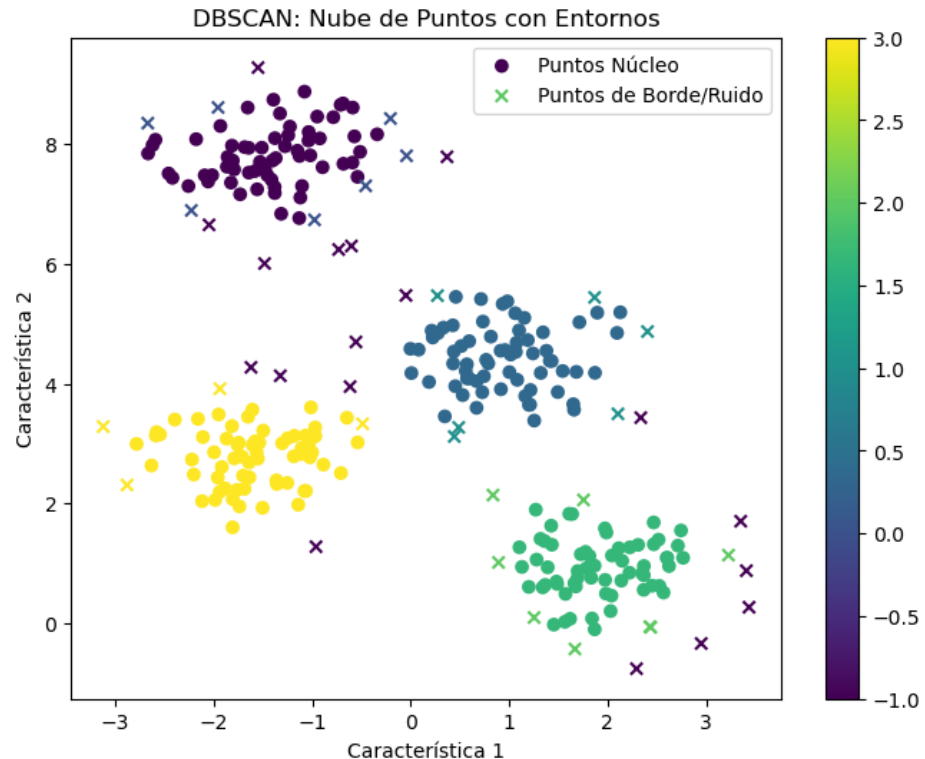
Ejemplo: California housing prices dataset



- Métodos de partición: **K-Means**
- Métodos basados en la densidad: **DBSCAN**

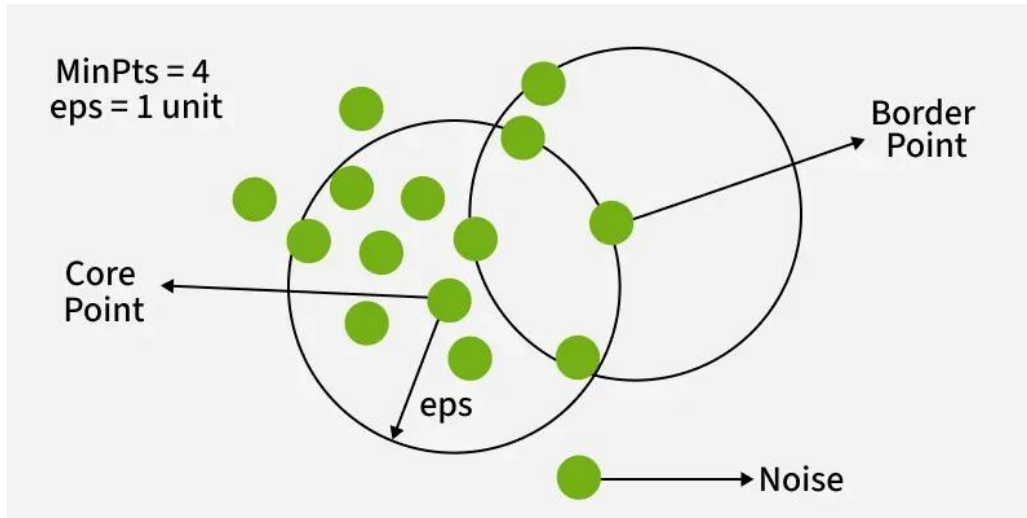
DBSCAN (Density-Based Spatial Clustering of Applications with Noise, agrupación espacial de aplicaciones basada en densidad con ruido) se basa en el concepto de densidad de puntos en el espacio de características.

Puntos más cercanos entre sí en este espacio tienen más probabilidades de pertenecer al mismo clúster.



El proceso de clústerización para cada punto p se realiza de la siguiente manera:

- Si p es un punto núcleo, se forma un nuevo clúster con él y todos los puntos alcanzables desde él.
- Si p es un punto de borde, se asigna al mismo clúster que un punto núcleo en su vecindario.
- Si p es un punto de ruido (valores atípicos), no se asigna a ningún clúster.



$$N(x) = \{p \in X : \text{dist}(p, x) \leq \epsilon\}$$

Dos puntos están conectados por densidad si existe una cadena de puntos donde cada punto se encuentra dentro del radio epsilon del siguiente y al menos un punto de la cadena es un punto central.

DBSCAN(eps, min_samples, metric, algorithm, leaf_size)

Ventajas: no requiere especificar el número de clústeres, maneja bien el ruido, e identifica clústeres de formas arbitrarias.

Desventajas: dificultades con grupos de densidad variable o datos con muy alta dimensionalidad.

eps o radio máximo del vecindario: La distancia máxima entre dos muestras para que una se considere como en el vecindario de la otra.

min_samples: El número mínimo de muestras en un vecindario para que un punto sea considerado como núcleo.

- pto. central: al menos min_samples dentro de eps
- pto. límite: dentro de eps, menos de min_samples
- pto. de ruido: ninguno de los anteriores

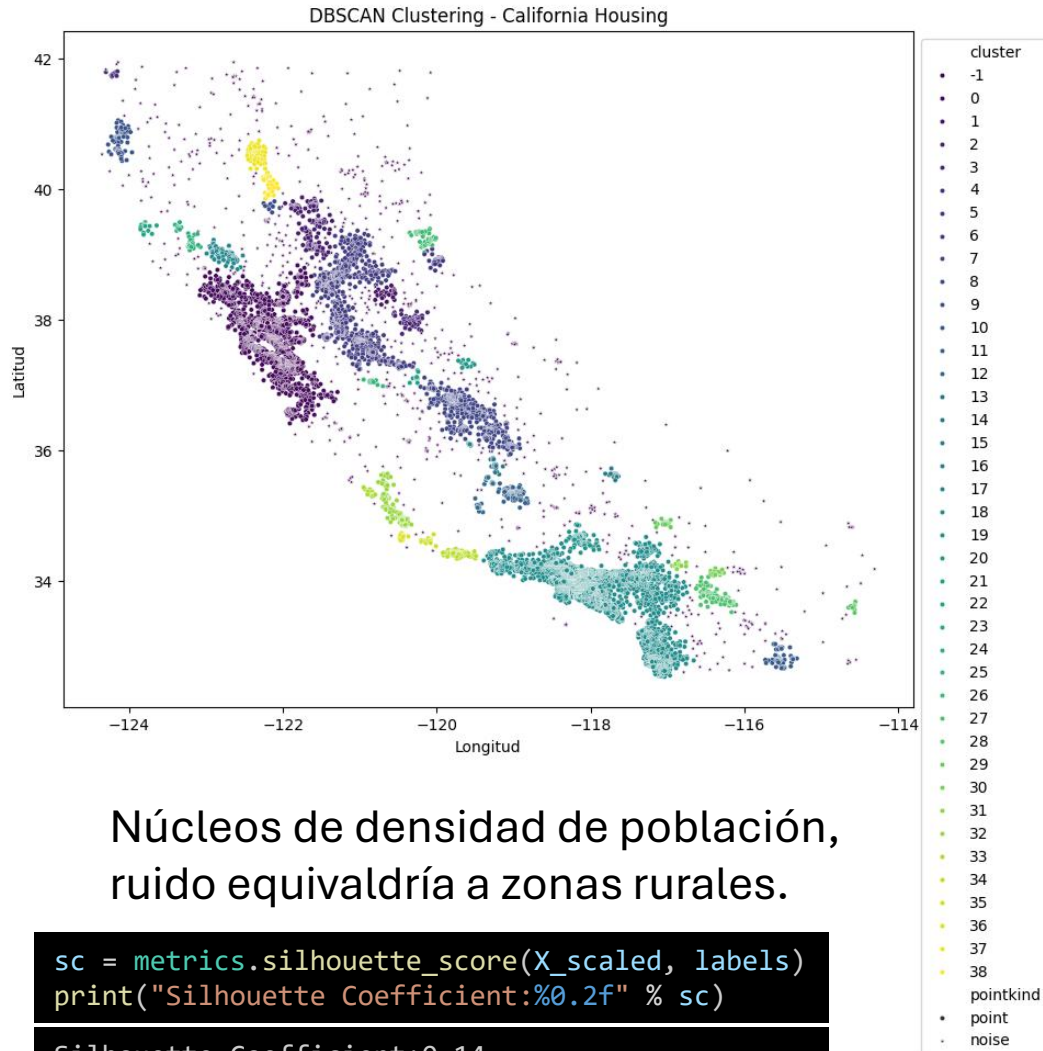
metric: La métrica de distancia utilizada para calcular las distancias.

algorithm: El algoritmo utilizado para calcular las distancias.

leaf_size: Tamaño de hoja pasado a los algoritmos de árbol.

Ejemplo: California housing prices dataset

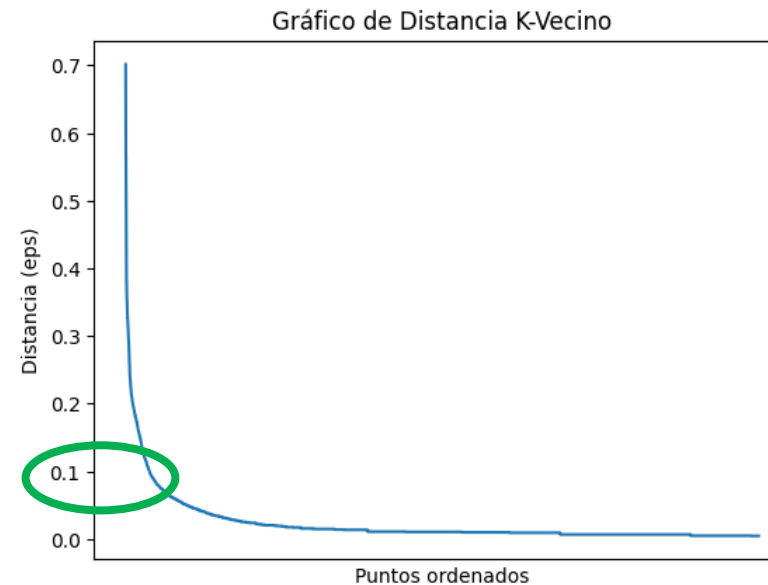
```
dbscan = DBSCAN(eps=0.05, min_samples=10)
```



Determinar los mejores valores:

- min_samples: $D+1$ a $2xD$, donde D es el número de dimensiones del conjunto de datos
- eps: calcular la distancia a k-ésimo vecino más cercano a cada punto de datos.
 - muy pequeño -> muchos puntos ruido
 - muy grande -> clústeres fusionados

```
neighbors = NearestNeighbors(n_neighbors=min_samples)
```

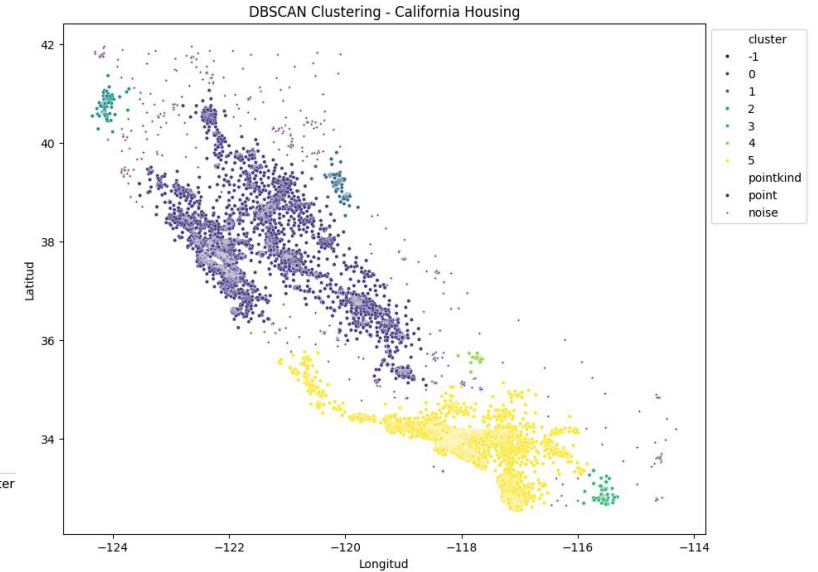
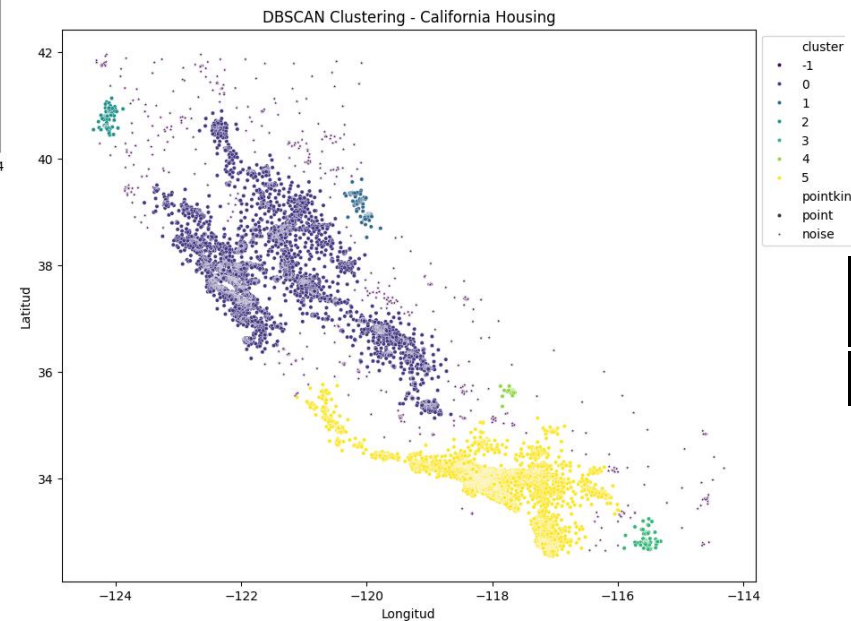
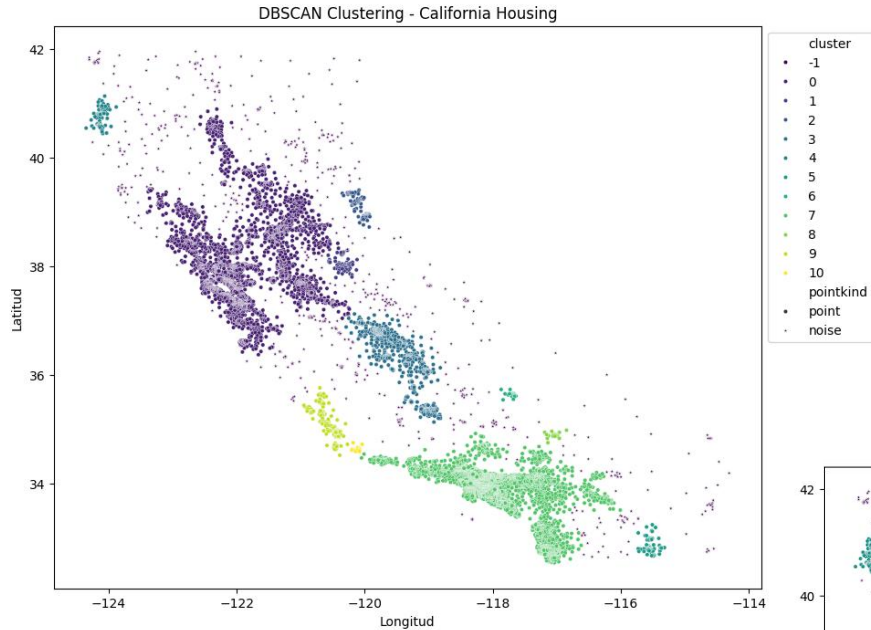


Ejemplo: California housing prices dataset

```
dbscan = DBSCAN(eps=0.08, min_samples=20)
```

```
dbscan = DBSCAN(eps=0.10, min_samples=20)
```

```
dbscan = DBSCAN(eps=0.12, min_samples=20)
```



```
sc = metrics.silhouette_score(X_scaled, labels)
print("Silhouette Coefficient:%0.2f" % sc)
```

Silhouette Coefficient:0.35

De -1 a 1, siendo 1 el mejor resultado. Valores cercanos a 0 indican clústeres superpuestos.

Comentarios y preguntas